14 haftalık yol haritasında 9. hafta

ALGORİTMA VE PROGRAMLAMA – II · DERS NOTU

HAFTA 09

Yığın ve Kuyruk

Soyut Veri Tipleri ve Algoritmik Uygulamaları

MODÜL III · DİNAMİK VERİ VE HIZLI ERİŞİM · H8–H10

> **HAFTANIN BÜYÜK SORUSU**
>
> *Hesap makineniz 3 + 4 × 2 ifadesinde önce çarpmayı yapması gerektiğini nereden bilir?*

Dr. Öğr. Üyesi Mehmet Sevri
Recep Tayyip Erdoğan Üniversitesi · Mühendislik ve Mimarlık Fakültesi · Bilgisayar Mühendisliği Bölümü
Bilgisayar Mühendisliği 1. sınıf · 2026–2027 Bahar yarıyılı · Sürüm 1.0 · Ekim 2026
`Ortam: gcc -std=c17 · g++ -std=c++17 (-Wall -Wextra -pedantic -O2) · Python 3.10+`

İçindekiler

1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları.....4
1.1 Öğrenme çıktıları.....5
2. Geçen Haftadan ve AP-1'den Bağlantı.....5
3. Kavramsal Anlatım: Soyut Veri Tipi, Yığın, Kuyruk ve İfade Gösterimleri.....6
3.1 Soyut veri tipi: işlemler, sözleşmeler ve opak tip.....6
3.2 Yığın: son giren ilk çıkar.....7
3.3 Kuyruk ve iki uçlu kuyruk.....8
3.4 İfade gösterimleri: araek, önek ve sonek.....8
3.5 Kaputun Altında: çağrı yığını, dairesel tampon ve CPython'un işlenen yığını.....9
4. Algoritmalar.....10
4.1 Yığın işlemleri: dizi ve liste tabanlı gerçekleştirim (Örnek 1–2).....10
4.2 Dairesel tampon kuyruk (Örnek 8).....12
4.3 Parantez denetimi (Örnek 3).....14
4.4 Sonek ifade değerlendirme (Örnek 4).....17
4.5 Shunting-yard: araekten soneke dönüşüm (Örnek 5).....19
4.6 Monotonik yığın: sonraki büyük eleman (Örnek 11).....21
4.7 Kayan pencere en büyüğü: monotonik iki uçlu kuyruk (Örnek 13).....23
4.8 Özyinelemeden açık yığına: hızlı sıralama ve ızgara dolaşması (Örnek 12, Kavram 4).....26
5. Çözümlü Örnekler.....28
5.1 Basit hesap makinesi: dönüşüm ve değerlendirme (Örnek 6).....28
5.2 Geri al / yinele: iki yığın (Örnek 7).....28
5.3 Liste tabanlı kuyruk ve katman katman işleme (Örnek 9).....29
5.4 Kayıt gişesi simülasyonu (Örnek 10).....30
6. Haftanın Deneyi: Gişe Sayısı, Yük Oranı ve Kuyruk Gerçekleştirimi.....31
6.1 Ölçüm düzeni.....31
6.2 Yük oranının etkisi.....32
6.3 Gişe sayısının etkisi.....32
6.4 Kuramla nitel karşılaştırma ve Little yasası.....33
6.5 Kuyruk gerçekleştirimlerinin maliyeti.....33
6.6 Yorum, sınırlılıklar ve kendi makinenizde.....35
7. Hata Avcısı.....35
8. Dil Penceresi: C · C++ · Python.....36
9. Mühendis Gözüyle: Arayüz, Sözleşme ve Taşma Politikası.....38
10. Mini-OBS 2 Adımı: v2.8 — Geri Al / Yinele ve Ders Kayıt Kuyruğu.....38

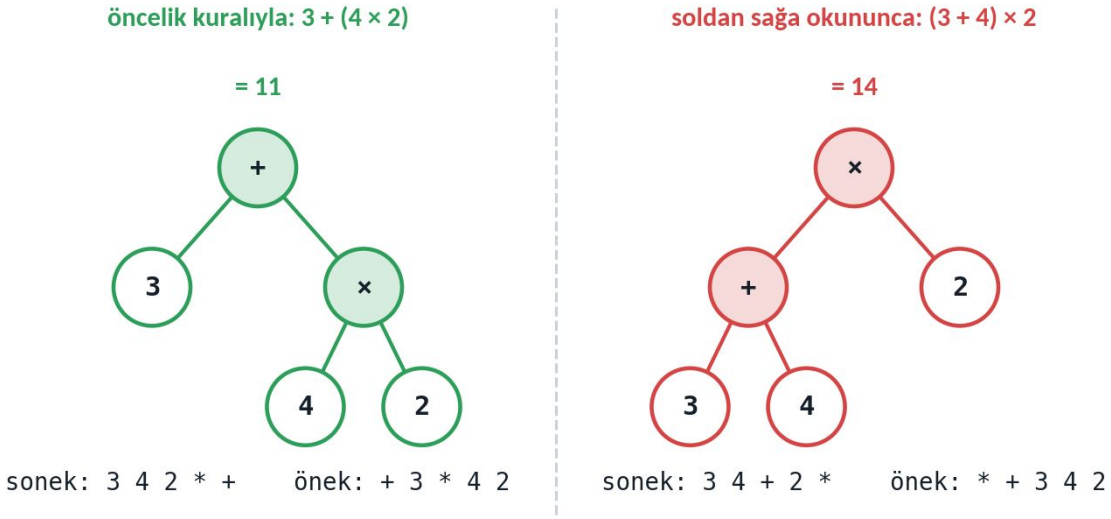
10.1 Gereksinimler ve ipuçları 38
10.2 Kabul ölçütleri ve test senaryoları 39
11. Alıştırmalar ve Atölyeler 40
11.1 İzleme ve çıktı tahmini (A) 41
11.2 Karmaşıklık (B) 41
11.3 Değişmez yazma (C) 42
11.4 Hata bulma (D) 42
11.5 Parsons (E) 43
11.6 Kod yazma (F) 43
11.7 Diller arası okuma (G) 44
11.8 Atölye 1 · Ray manevra oyunu: Shunting-yard kartlarla (sınıf, ≈ 15 dk) 45
11.9 Atölye 2 · Mini derleyici adımı: dört işlem, parantez ve üs destekli hesap makinesi (eşli, ≈ 30 dk) 45
11.10 Atölye 3 · Kuyruk simülasyonu: gişe sayısının ortalama beklemeye etkisi (gruplar, ≈ 20 dk). 45
12. Kendini Değerlendirme 46
13. İleri Bakış ve Merak Kutusu 46
14. Özet, Kavram Haritası ve Terim Sözlüğü 47
15. Kaynaklar 48
Ek A. Dil Karşılıkları 49
A.1 Örnek 1 · Dizi tabanlı yığın 49
A.2 Örnek 2 · Liste tabanlı yığın 50
A.3 Örnek 3 · Parantez denetimi 51
A.4 Örnek 4 · Sonek değerlendirme 52
A.5 Örnek 5 · Shunting-yard 54
A.6 Örnek 6 · Basit hesap makinesi 55
A.7 Örnek 7 · Geri al / yinele 56
A.8 Örnek 8 · Dairesel tampon kuyruk 57
A.9 Örnek 9 · Liste tabanlı kuyruk 58
A.10 Örnek 10 · Kayıt gişesi simülasyonu 59
A.11 Örnek 11 · Sonraki büyük eleman 61
A.12 Örnek 12 · Açık yığınlı hızlı sıralama 62
A.13 Örnek 13 · Kayan pencere en büyüğü 63
A.14 Örnek 14 · Histogramdaki en büyük dikdörtgen 65

1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları

Hesap makineniz 3 + 4 × 2 ifadesinde önce çarpmayı yapması gerektiğini nereden bilir?

Tuşlandığı sırayla, soldan sağa hesap yapan bir makine (3 + 4) × 2 = 14 bulurdu. Doğru yanıt 11'dir, çünkü çarpmanın **önceliği** (precedence) toplamanınkinden yüksektir (Şekil 9.1). Bir programın bu kuralı "bilmesi" iki adımdır ve ikisi de bu haftanın veri yapısını, **yığını** (stack) kullanır: ifade önce parantez ve öncelik kuralı gerektirmeyen **sonek** (postfix) gösterime çevrilir (Örnek 5: `3 + 4 * 2 -> 3 4 2 * +`), sonra bir işlenen yığınıyla soldan sağa tek geçişte hesaplanır (Örnek 6: `3 + 4 * 2 = 11`).

Şekil 9.1
3 + 4 × 2 ifadesinin iki okunuşu: öncelik kuralıyla 3 + (4 × 2) = 11 (solda) ve soldan sağa okununca (3 + 4) × 2 = 14 (sağda); her ağacın sonek ve önek yazılışı altında



araekte ikinci okuma için parantez gerekir; sonek ve önek gösterimde parantez ve öncelik kuralı yoktur

Bu hafta iki **soyut veri tipini** (ADT) işliyoruz: son giren ilk çıkar (LIFO) düzenindeki **yığın**, ilk giren ilk çıkar (FIFO) düzenindeki **kuyruk** (queue) ve ikisinin genellemesi **iki uçlu kuyruk** (deque). Yapılar birkaç satırdır; asıl konu onlarla çözülen problemlerdir: parantez denetimi, sonek değerlendirme ve araekten soneke dönüşüm (bir derleyicinin ilk adımları), geri al / yinele, her elemanın bir kez girip en çok bir kez çıktığı **amortize** Θ(n) algoritmalar, özyinelemenin açık yığınla yazılması ve bir kayıt gişesinin benzetimi.

> **GERÇEK DÜNYADAN · 1961: üç yönlü bir demiryolu kavşağı**
>
> Edsger W. Dijkstra, Mathematisch Centrum'da X1 bilgisayarı için bir ALGOL 60 çevirmeni yazarken ifade çevirisini Kasım 1961 tarihli MR 35/61 raporunda anlattı (Dijkstra, 1961): "The translation process shows much resemblance to shunting at a three way railroad junction" (çeviri, üç yönlü demiryolu kavşağındaki manevraya çok benzer). İşlenenler doğrudan çıktıya geçer, işleçler "translator stack"te bekler; her işlecin bir öncelik sayısı vardır (+ − 9; × / ve tekli eksi 10; ↑ 11) ve yığındakiler, öncelikleri gelen işlecinkinden "greater then or equal to" (sic) oldukça çıktıya aktarılır. **Shunting-yard** (manevra sahası) adı bu benzetmeden gelir; raporun bu bölümü algoritmaya ad vermez, onu kavşaktaki manevraya benzetir.
>
> "Büyük-eşit" kuralı bütün işleçleri **soldan birleşmeli** yapar ve ALGOL 60'la tutarlıdır: Revised Report'a göre 2↑n↑k, (2↑n)↑k demektir (Backus vd., 1963). Bu derste ^ matematikteki gibi **sağdan birleşmelidir** (2 ^ 3 ^ 2 = 512); kural bu yüzden değişir (Bölüm 4.5). Aynı yıllarda F. L. Bauer ve K. Samelson formül çevirisini **iki yığınla** yaptılar; patent metni bunlara "numbers cellar" ve "operations cellar" (sayı ve işlem mahzeni) der. Alman başvurusu 30 Mart 1957'dir; ABD patenti (başvuru 28 Mart 1958) 31 Temmuz 1962'de verildi (Bauer & Samelson, 1962). Yöntemi Samelson ve Bauer (1960) yayımladı.

> **GERÇEK DÜNYADAN · 1972: cebe sığan dört katlı yığın — HP-35**
>
> Hewlett-Packard'ın 1972'de çıkardığı HP-35'i tanıtan makaleye göre kullanıcının beş yazmacından dördü (X, Y, Z, T) bir "operational stack" (işlem yığını) oluşturur. ENTER↑ yığını iter: "x is copied into Y, y into Z, z into T, and t is lost" (Whitney vd., 1972, s. 3); iki işlenenli her işlemden sonra yığın bir düzey düşer. (3 × 4) + (5 × 6) için tuş dizisi 3, ENTER↑, 4, ×, 5, ENTER↑, 6, ×, + olarak verilir: parantez tuşu gerekmez. Makale HP-35'i ters Polonya gösteriminin üstünlüklerinden tam olarak yararlanan ilk bilimsel hesap makinesi olarak tanıtır; bu, üreticinin iddiasıdır.

1.1 Öğrenme çıktıları

- **ÖÇ 9.1** Soyut veri tipi kavramını arayüz–gerçekleştirim ayrımıyla açıklar; yığın ve kuyruğu dizi ve bağlı listeyle gerçekler. (Bölüm 3.1–3.3, 4.1, 5.3; Örnek 1, 2, 9)
- **ÖÇ 9.2** Dairesel tamponla sabit kapasiteli kuyruğu, dolu ve boş durumlarını doğru ayırt ederek gerçekler. (Bölüm 4.2; Örnek 8; Hata Avcısı 2)
- **ÖÇ 9.3** Yığın tabanlı algoritmalarla parantez denetimi, sonek ifade değerlendirme ve araek → sonek dönüşümü (Shunting-yard) yapar. (Bölüm 3.4, 4.3–4.5, 5.1; Örnek 3–6)
- **ÖÇ 9.4** Monotonik yığınla "sonraki büyük eleman" türü problemleri O(n) sürede çözer. (Bölüm 4.6–4.7, 13; Örnek 11, 13, 14)
- **ÖÇ 9.5** Özyinelemeli bir algoritmayı açık yığın kullanan yinelemeli biçime dönüştürür; kuyruğu simülasyon ve katman katman işlemede kullanır. (Bölüm 3.5, 4.8, 5.3–5.4, 6; Örnek 9, 10, 12)

2. Geçen Haftadan ve AP-1'den Bağlantı

H8'in iki yapısını kullanıyoruz: dolunca iki katına büyüyen **dinamik dizi** (ekleme **amortize** O(1)) ve başa ekleme ile baştan silmenin O(1) olduğu **tek bağlı liste**; sona eklemek için **son işaretçisi** gerekir. Bunlar sırasıyla dizi tabanlı yığının (Örnek 1, Kavram 1), liste tabanlı yığının (Örnek 2) ve baş + son işaretçili kuyruğun (Örnek 9) temelidir. AP-1 H9'da da özyinelemeyi **çağrı yığını** (call stack) üzerinden izlemiştik: her çağrı bir **çerçeve** (frame) iter, her dönüş tepedekini çıkarır.

> **TAHMİN ET · Geri çağırma: üç soru (yanıtlar aşağıda)**
>
> - **S1 (H8).** Kapasitesi 1'den başlayıp dolunca iki katına çıkan dinamik diziye 20 eleman ekleniyor. Kaç kez büyür ve toplam kaç eleman kopyalanır?
> - **S2 (H8).** Yalnızca baş işaretçisi olan tek bağlı listenin sonuna n = 1 000 eleman tek tek ekleniyor. Toplam kaç düğüm ziyaret edilir? Son işaretçisi tutulsaydı?
> - **S3 (AP-1 H9).** `faktoriyel(4)` çalışırken çağrı yığınında en çok kaç çerçeve bulunur ve ilk hangi çağrı döner?

Yanıtlar. S1: kapasite 1 → 2 → 4 → 8 → 16 → 32: 5 büyütme, 1 + 2 + 4 + 8 + 16 = 31 kopya (ekleme başına 1,55; 10^6 eklemede 1,049; H8 Örnek 1). S2: n(n − 1)/2 = 499 500 ziyaret, son işaretçisiyle 0 (H8 Örnek 4). S3: dört çerçeve; ilk dönen en son çağrılan `faktoriyel(1)`'dir: son giren ilk çıkar (AP-1 H9 Örnek 6).

3. Kavramsal Anlatım: Soyut Veri Tipi, Yığın, Kuyruk ve İfade Gösterimleri

3.1 Soyut veri tipi: işlemler, sözleşmeler ve opak tip

Bir **soyut veri tipi** (abstract data type, ADT) bir değerler kümesini, üzerindeki işlemleri ve her işlemin **sözleşmesini** tanımlar: işlem hangi koşulda çağrılabilir (önkoşul), sonunda ne doğrudur (sonkoşul), koşul sağlanmazsa ne olur (hata bildirimi). Verinin bellekte nasıl tutulduğunu söylemez. Yığın, kuyruk ve iki uçlu kuyruk, elemanlarına yalnızca uçlarından erişilen **doğrusal listelerin** üç kısıtlı biçimidir (Knuth, 1997). Bu haftanın bütün örneklerinde her işlem başarısını `bool` olarak döndürür ve başarısız işlem yapıyı değiştirmez (Tablo 9.1).

Tablo 9.1
Yığın ve kuyruk soyut veri tipleri: işlemler ve sözleşmeler

İşlem	Yığın (LIFO)	Kuyruk (FIFO)	Sözleşme (bu haftanın C arayüzü)
ekle	`yigin_it(y, x)`: x tepeye	`kuyruk_ekle(q, x)`: x sona	O(1); sabit kapasitede yapı doluysa **taşma** (overflow): false, yapı değişmez
çıkar	`yigin_cikar(y, &x)`: tepedeki	`kuyruk_cikar(q, &x)`: baştaki	O(1); yapı boşsa **alt taşma** (underflow): false, `*x` değişmez
bak	`yigin_tepe(y, &x)`	baştaki (Örnek 13: `deque_on`)	O(1); boş yapıda false
oluştur, yok et	`yigin_olustur`, `yigin_yok_et`	`kuyruk_olustur`, `kuyruk_yok_et`	oluşturan yok eder (sahiplik); yok etme bütün belleği bırakır

C'de arayüzü gerçekleştirimden ayırmanın aracı **opak tiptir** (opaque type): başlık yapının yalnızca adını bildirir; kullanıcı kodu bu **eksik tipe** (incomplete type) yalnızca işaretçi tutar ve onu ancak arayüzün fonksiyonlarıyla değiştirir. Kavram 1 çok dosyalıdır: `yigin.h` arayüz, `yigin_dizi.c` (dolunca iki katına büyüyen dizi) ve `yigin_liste.c` (tek bağlı liste) iki gerçekleştirim, `main.c` yalnızca başlığı bilen kullanıcı kodudur:

C **kavram/H09_K01_opak_yigin/yigin.h · satır 15–28**

```c
typedef struct Yigin Yigin;  /* opak tip: yalnızca işaretçisi kullanılır */

/* Boş yığın oluşturur; bellek yetmezse NULL döner.
 * Sahiplik: dönen yığın çağıranındır; yigin_yok_et ile serbest bırakılır. */
Yigin *yigin_olustur(void);

/* Yığını ve tuttuğu bütün belleği serbest bırakır (NULL güvenlidir). */
void yigin_yok_et(Yigin *y);

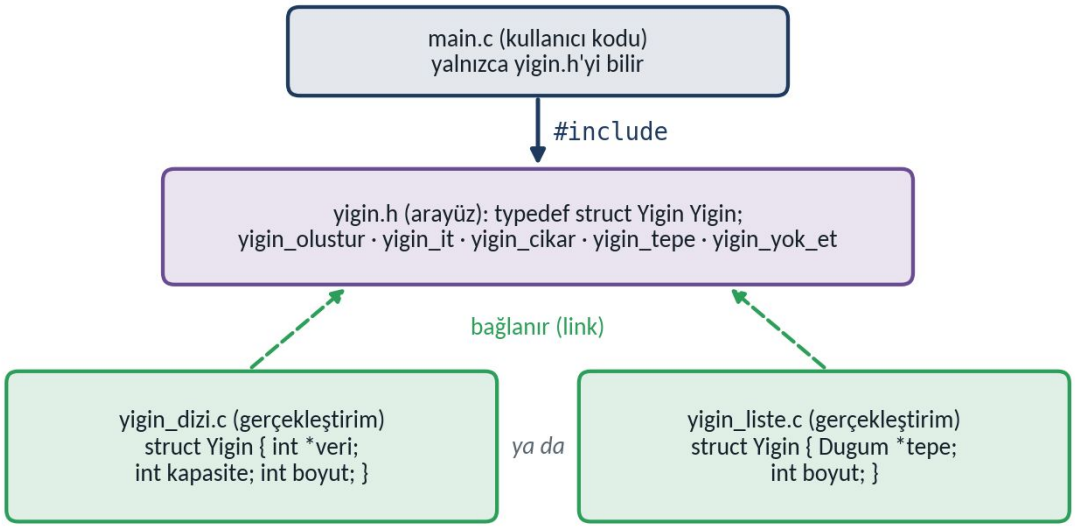
/* x'i tepeye iter; yalnızca bellek yetmezse false döner (sınırsız). */
bool yigin_it(Yigin *y, int x);

/* Tepeyi çıkarıp *x'e yazar; yığın boşsa false döner, *x değişmez. */
bool yigin_cikar(Yigin *y, int *x);
```

Alanlara erişmeye ya da yapının boyutunu almaya çalışan kod derlenmez (`opak_ihlal.c`: `error: invalid use of incomplete typedef ‘Yigin’`; `error: invalid application of ‘sizeof’ to incomplete type ‘Yigin’`). Aynı `main.c` iki gerçekleştirimle ayrı ayrı bağlanır; ikisi de `10 kare itildi: boyut = 10, tepe = 100` ve `3 kez çıkar: 100 81 64; boyut = 7`

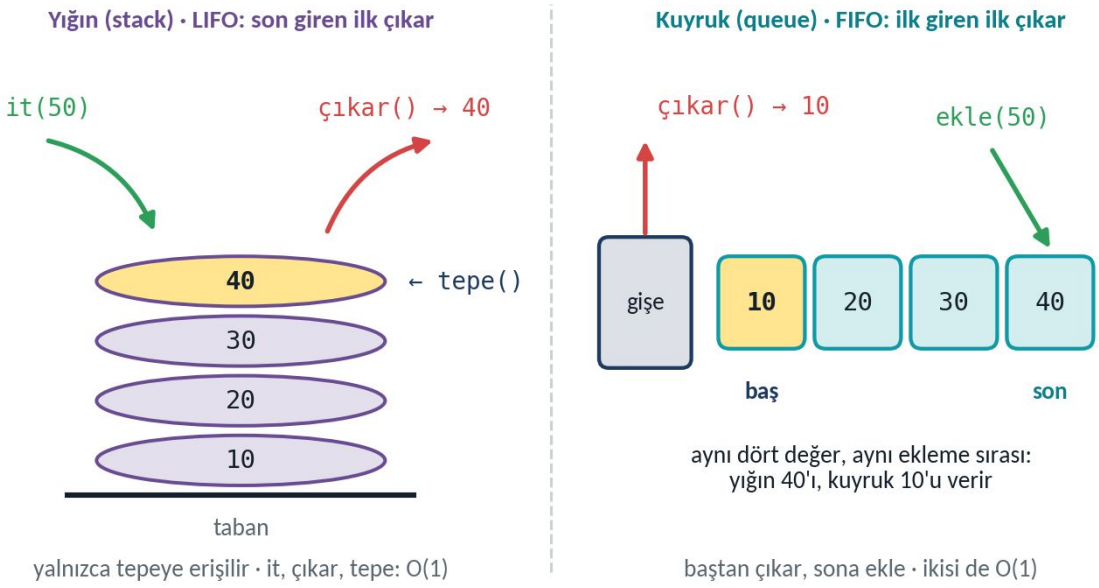
satırlarını yazar ve iki standart çıktı bayt bayt aynıdır (`./yigin_dizi > dizi.txt; ./yigin_liste > liste.txt; cmp dizi.txt liste.txt` → `fark yok: iki gerçekleştirimin stdout'u bayt bayt aynı`). Arayüz sabit kaldıkça gerçekleştirim değişebilir (Şekil 9.2; Bölüm 9); bedeli, yapının öbekte ayrılması ve her erişimin bir fonksiyon çağrısı olmasıdır.

Şekil 9.2
Opak tip (Kavram 1): kullanıcı kodu yalnızca yigin.h'yi görür; struct Yigin'in alanları gerçekleştirim dosyalarındadır ve iki gerçekleştirimden biri bağlanır



3.2 Yığın: son giren ilk çıkar

Şekil 9.3
Yığın (dikey tabak yığını, LIFO) ve kuyruk (yatay sıra, FIFO): aynı dört değer aynı sırayla eklenince yığından 40, kuyruktan 10 çıkar



Yığında bütün işlemler **tepede** (top) yapılır: **it** (push) ekler, **çıkar** (pop) tepedekini alır, **tepe** (peek) çıkarmadan bakar; son giren ilk çıkar (LIFO), bir tabak yığını gibi (Şekil 9.3). **Dizi tabanlı** yığında `veri[0..boyut − 1]` doludur, tepe `veri[boyut − 1]`'dir; bellek bitişiktir, kapasite sabitse taşma olabilir. **Liste tabanlı** yığında tepe baş düğümdür; kapasite yoktur ama her it bir `malloc` ister. Gündelik örnekler "en son yapılanı geri al" düzenindedir: düzenleyicinin geri al / yinele düğmeleri (Örnek 7), tarayıcının geri ve ileri düğmeleri. Yeni bir sayfaya gidilince **ileri yığını boşaltılır**, çünkü dallanan geçmişin "ilerisi" tanımsızdır (Kavram 9: `ileri şimdiki: b.com geri: [ana.sayfa a.com] ileri: [c.com]` → `git d.com şimdiki: d.com geri: [ana.sayfa a.com b.com] ileri: []`).

3.3 Kuyruk ve iki uçlu kuyruk

Kuyrukta **ekle** (enqueue) sona, **çıkar** (dequeue) baştan yapılır: ilk giren ilk çıkar (FIFO). Dizide her çıkarmada kalanları sola kaydırmak çıkarmayı Θ(n) yapar (Bölüm 6.5); çözüm, sondaki indisi başa saran **dairesel tampondur** (Bölüm 4.2). Liste tabanlı kuyruk baş ve son işaretçisiyle iki işlemi de O(1) yapar (Bölüm 5.3). Kuyruğun iki büyük kullanımı **bekleme sıraları** (gişe, yazıcı, ağ paketleri; Örnek 10) ve **katman katman işlemedir**: Örnek 9 1'den başlayıp her x için 2x ve 2x + 1'i ekleyince elemanlar düzey düzey çıkar (`düzey 2: 4 5 6 7`); H11'deki genişlik öncelikli arama (BFS) bu döngüdür. **İki uçlu kuyrukta** (double-ended queue, deque) iki uçtan da eklenip çıkarılır; yığın ve kuyruk onun özel durumlarıdır (Knuth, 1997). Örnek 13 onu dairesel tamponla gerçekler; C++'ta `std::deque`, Python'da `collections.deque` iki uçta da O(1)'dir.

3.4 İfade gösterimleri: araek, önek ve sonek

Günlük yazımda işleç işlenenlerin **arasındadır** (**araek**, infix) ve okumak için üç ek bilgi gerekir: **öncelik**, **birleşme yönü** (associativity: 1 − 2 − 3 soldan, (1 − 2) − 3; 2 ^ 3 ^ 2 sağdan, 2 ^ (3 ^ 2)) ve bunları değiştiren **parantez**. İşleç işlenenlerden **önce** yazılırsa **önek** (prefix), **sonra** yazılırsa **sonek** (postfix) gösterim elde edilir; ikisinde de bu üç bilgiye gerek yoktur (Tablo 9.2). Önek gösterimini Jan Łukasiewicz mantık formülleri için geliştirdi; kendi ifadesiyle, işlevleri bağımsız değişkenlerinden önce yazarak ayraçlardan kurtulan bu simgeleri 1929'dan beri kullanıyordu (Łukasiewicz, 1957, s. 78). Önek gösterime bu yüzden **Polonya**, sonek gösterime **ters Polonya gösterimi** (reverse Polish notation, RPN) denir (Whitney vd., 1972).

Tablo 9.2
Aynı ifadelerin üç gösterimi (sonek ve değer Örnek 5 ve 6'nın çıktılarından; ~ tekli eksidir)

Araek	Önek	Sonek	Değer
3 + 4 * 2	+ 3 * 4 2	3 4 2 * +	11
(3 + 4) * 2	* + 3 4 2	3 4 + 2 *	14
2 ^ 3 ^ 2	^ 2 ^ 3 2	2 3 2 ^ ^	512
(2 ^ 3) ^ 2	^ ^ 2 3 2	2 3 ^ 2 ^	64
1 - 2 - 3 - 4	- - - 1 2 3 4	1 2 - 3 - 4 -	−8
-2 ^ 2	~ ^ 2 2	2 2 ^ ~	−4
(a + b) * (c - d)	* + a b - c d	a b + c d - *	—
a + b * c ^ d ^ e - f	- + a * b ^ c ^ d e f	a b c d e ^ ^ * + f -	—

Sonekte **işlenenlerin sırası araektekiyle aynıdır**; yalnızca işleçler yer değiştirir ve uygulanacakları anda işlenenlerinin hemen arkasında dururlar. Bu yüzden sonek ifade bir yığınla tek geçişte hesaplanır

(Bölüm 4.4) ve araekten soneke dönüşüm işleçleri bir yığında bekletmekten ibarettir (Bölüm 4.5). Öncelik ve birleşme yönü sonekte **işleç sırasına** dönüşür: `2 3 2 ^ ^` ile `2 3 ^ 2 ^` farklı ifadelerdir.

3.5 Kaputun Altında: çağrı yığını, dairesel tampon ve CPython'un işlenen yığını

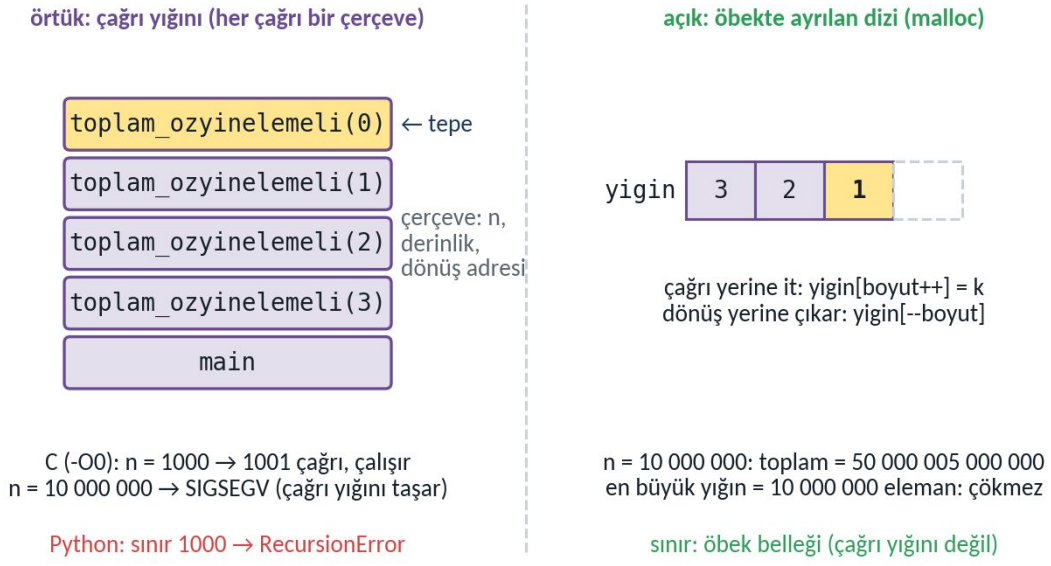
KAPUTUN ALTINDA · **Özyineleme örtük bir yığındır**

Her çağrı çağrı yığınına bir çerçeve iter (bağımsız değişkenler, yereller, dönüş adresi). Kavram 3 aynı toplamı iki yolla hesaplar: `toplam_ozyinelemeli` kendini çağırır, `toplam_yiginla` "çağrıları" öbekte ayrılmış bir diziye iter, sonra çıkararak toplar (aşağıda). n = 1000'de ikisi aynı sonucu verir (`özyinelemeyle : toplam = 500500, en büyük derinlik = 1001 çağrı`). n = 10^7'de açık yığın çalışır (`açık yığınla : toplam = 50000005000000, en büyük yığın = 10000000 eleman (öbekte)`), özyinelemeli sürüm bu makinede, bu çalıştırmada (-O0, yığın sınırı 8 MiB) çağrı yığınını taşırıp SIGSEGV ile sonlandı (Şekil 9.4); -O2'de derleyici bu özyinelemeyi döngüye çevirebilir, ama buna güvenilmez. Python bir sınır koyar (`n = 900; özyineleme sınırı sys.getrecursionlimit() = 1000`); n = 5000'de `özyinelemeyle : RecursionError: maximum recursion depth exceeded (ulaşılan derinlik 998)`. Derinliği girdiye bağlı özyineleme bu yüzden **açık yığınla** yazılabilmelidir (Bölüm 4.8).

kavram/H09_K03_cagri_yigini.c · satır 40–46

```
40      for (long k = n; k > 0; k--) {
41          yigin[boyut++] = k;      /* çağrı yerine: çerçeveyi it */
42      }
43      *en_buyuk = boyut;
44      while (boyut > 0) {
45          sonuc += yigin[--boyut]; /* dönüş yerine: çerçeveyi çıkar */
46      }
```

Şekil 9.4
Çağrı yığını ve açık yığın (Kavram 3): solda özyinelemenin çerçeveleri (8 MiB'lık çağrı yığınında), sağda öbekte ayrılan yığın dizisi; n = 10^7'de yalnızca açık yığın çalışır



KAPUTUN ALTINDA · **Dairesel tampon gerçek sistemlerde; mod yerine bit maskesi**

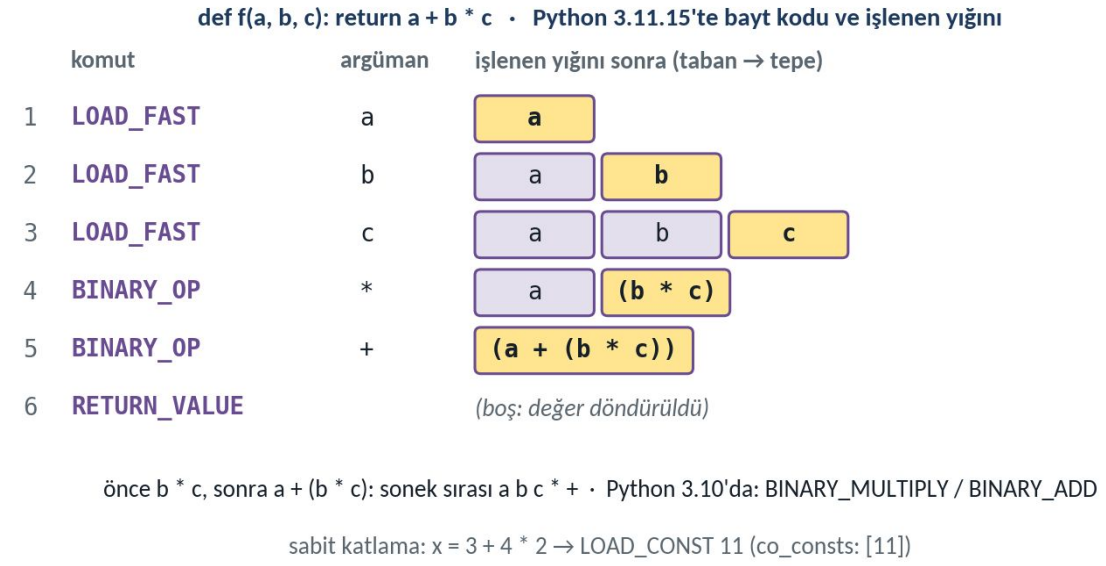
Dairesel tampon (ring buffer), üreticinin bir uçtan yazdığı, tüketicinin öbür uçtan okuduğu sabit boyutlu bir dizidir: klavye tamponları tuşları, ses akışları örnekleri, ağ kartları paket tanımlayıcılarını böyle halkalarda tutar; bellek bir kez ayrılır, eleman kaydırılmaz. İndis `(i + 1) % kapasite` ile ilerler; kapasite 2^k ise

kapasite − 1'in ikilik yazımı k tane 1'dir ve `(i + 1) & (kapasite - 1)` aynı sonucu bölmesiz verir. Kavram 5 bunu 0 … 999 999 aralığında denetler: kapasite 8, 16, 1024'te farklı sonuç 0, 0, 0; kapasite 6, 10, 1000'de 666 668, 800 000, 992 064. Örnek 8 kapasite 2'nin kuvvetiyse maskeyi kullanır. Geriye sarmada `-1 % 8` C'de −1, Python'da 7'dir; C'de `(i - 1 + k) % k` yazılır (Bölüm 8).

KAPUTUN ALTINDA · **CPython bir yığın makinesidir (Python 3.11.x'te)**

CPython kaynağı **bayt koduna** derler; yorumlayıcı komutları bir **işlenen yığını** üzerinde yürütür: `LOAD_FAST` bir yereli iter, `BINARY_OP` iki değeri çıkarıp sonucu iter, `RETURN_VALUE` tepedekini döndürür. `dis` modülüyle a + b * c'nin komut sırası `a b c * +`, yani ifadenin **sonek** yazılışıdır (Şekil 9.5; Kavram 2, Python 3.11.15'te üretildi, bayt kodu sürüme bağlıdır). Python 3.10'da ikili işlemlerin ayrı komutları vardı (bu makinede 3.10.20: `BINARY_MULTIPLY`, `BINARY_ADD`); 3.11 bunları tek `BINARY_OP` komutunda topladı. Sabit ifadeler derleme anında katlanır: `x = 3 + 4 * 2` doğrudan `LOAD_CONST` 11 olur (co_consts içindeki tamsayılar: [11]).

Şekil 9.5
*def f(a, b, c): return a + b * c işlevinin CPython bayt kodu ve her komuttan sonraki işlenen yığını (Python 3.11.15; Kavram 2)*



4. Algoritmalar

Yedi amiral algoritmayı on bir adımlı standart anlatımla işliyoruz (problem, fikir, elle izleme, sözde kod, doğruluk, analiz ve kimlik kartı, C gerçekleştirimi, üç dil, test, deney, varyantlar). 4.1–4.2 yapıların kendisidir, 4.3–4.5 ifade işleme, 4.6–4.7 her elemanın bir kez girip en çok bir kez çıktığı **monotonik** yapılar; 4.8 özyinelemeyi açık yığınla yazar. Yapılar hep elle yazılmıştır; kütüphane karşılıkları yalnızca kâhin olarak ve Bölüm 8'de kullanılır. C++ ve Python sürümlerinin öteki fonksiyonları Ek A'dadır.

4.1 Yığın işlemleri: dizi ve liste tabanlı gerçekleştirim (Örnek 1–2)

1 · Problem. Tablo 9.1 içindeki sözleşmeyle yığını sabit kapasiteli bir dizide (Örnek 1) ve sınırsız bir listede (Örnek 2) gerçeklemek. **2 · Fikir.** Dizide `boyut` tepenin bir üstündeki boş hücredir: it `veri[boyut++] = x`, çıkar `x = veri[--boyut]`. Listede tepe baş düğümdür: it başa bağlar, çıkar

baş düğümü ayırıp serbest bırakır. **3 · Elle izleme.** Kapasite 4'te 10…50 itilir: `it 50 -> HAYIR (taşma: yığın dolu)`, yığın değişmez; dört çıkarma 40, 30, 20, 10'u verir, beşincisi `çıkar -> HAYIR (alt taşma: yığın boş)`. Örnek 2 aynı komutlarda 50'yi de kabul eder.

SÖZDE KOD · 4 · Sözde kod: dizi tabanlı yığın

```
algoritma İT(y, x)                          ▷ y.veri[0 .. y.boyut − 1] yığındır
    eğer y.boyut = y.kapasite ise döndür YANLIŞ      ▷ taşma: y değişmez
    y.veri[y.boyut] ← x;  y.boyut ← y.boyut + 1;  döndür DOĞRU
algoritma ÇIKAR(y)                          ▷ çıktı: (başarı, değer)
    eğer y.boyut = 0 ise döndür (YANLIŞ, —)          ▷ alt taşma: y değişmez
    y.boyut ← y.boyut − 1;  döndür (DOĞRU, y.veri[y.boyut])
algoritma TEPE(y)                           ▷ ÇIKAR gibi, ama boyut değişmez
```

5 · Doğruluk. Değişmez: veri[0..boyut−1] yığının tabandan tepeye içeriğidir; tepe veri[boyut−1]; 0 ≤ boyut ≤ kapasite. İt ve çıkar onu korur; başarısız işlem hiçbir alana dokunmaz. Denetimsiz çıkar `veri[-1]`'i okur, denetimsiz it bloğun ötesine yazar (Hata Avcısı 1 ve 6). **6 · Analiz.** Her işlem O(1). Dizi O(kapasite), liste O(n) bellek ister ama düğüm başına bir işaretçi ve her it için bir `malloc` öder. Dolunca iki katına çıkan dinamik dizide (Kavram 1) tek bir it O(n) kopya yapabilir; H8'in amortize çözümlemesiyle it başına **amortize** O(1) olur (Tablo 9.3).

Tablo 9.3
Yığın kimlik kartı

ALGORİTMA KİMLİK KARTI **Yığın: it / çıkar / tepe** *(stack: push / pop / peek)*	
Strateji	Soyut veri tipi (LIFO, son giren ilk çıkar): yalnızca tepeden erişim; dizi (boyut indisi) ya da bağlı liste (baş düğüm) ile gerçekleştirim, arayüz aynı
Girdi önkoşulu	Çıkar ve tepe için yığın boş olmamalı; sabit kapasiteli dizide it için boyut < kapasite
Zaman (en iyi · ortalama · en kötü)	O(1) · O(1) · O(1)*
Ek bellek	O(n)
Özellikler	Taşma ve alt taşma dönüş koduyla bildirilir. Liste tabanlıda kapasite yoktur, her it bir düğüm ayırır
Değişmez / doğruluk	veri[0..boyut−1] yığının tabandan tepeye içeriğidir; tepe veri[boyut−1]; 0 ≤ boyut ≤ kapasite
Kütüphane karşılığı	C: yok · C++: `std::stack` · Python: `list.append` / `list.pop()`
Ne zaman kullanılır	Son giren ilk çıkar düzeni: geri al / yinele, tarayıcı geri tuşu, parantez ve ifade işleme, özyinelemenin açık yığınla yazılması
Ne zaman kullanılmaz	Elemanlar geliş sırasıyla işlenecekse ya da ortadaki elemana erişim gerekiyorsa

7 · C gerçekleştirimi. (Örnek 1; `yigin_it` simetriktir: önce `yigin_dolu_mu` denetlenir, sonra `veri[boyut++] = x`, 48–60. satırlar):

C c/H09_01_dizi_yigin.c · satır 63–72

```
63 bool yigin_cikar(Yigin *y, int *x)
64 {
65     if (yigin_bos_mu(y)) {
66         return false;
67     }
```

```
68      y->boyut--;
69      *x = y->veri[y->boyut];
70      cikar_sayisi++;
71      return true;
72  }
```

8 · Üç dil karşılaştırması. Liste tabanlı çıkarma (Örnek 2) baş düğümü önce bir değişkende tutar, tepeyi ilerletir, düğümü **ondan sonra** bırakır: C'de `free`, C++'ta `delete`; Python'da bırakma yoktur (başvuru sayımı). Değer C'de `int *x`, C++'ta başvuru, Python'da `(başarı, değer)` demetiyle döner:

C++ **H09_02_liste_yigin.cpp · satır 81–86**

```
81      Dugum *d = y.tepe;
82      x = d->deger;
83      y.tepe = d->sonraki;
84      delete d;
85      serbest_dugum++;
86      y.boyut--;
```

Python **H09_02_liste_yigin.py · satır 73–77**

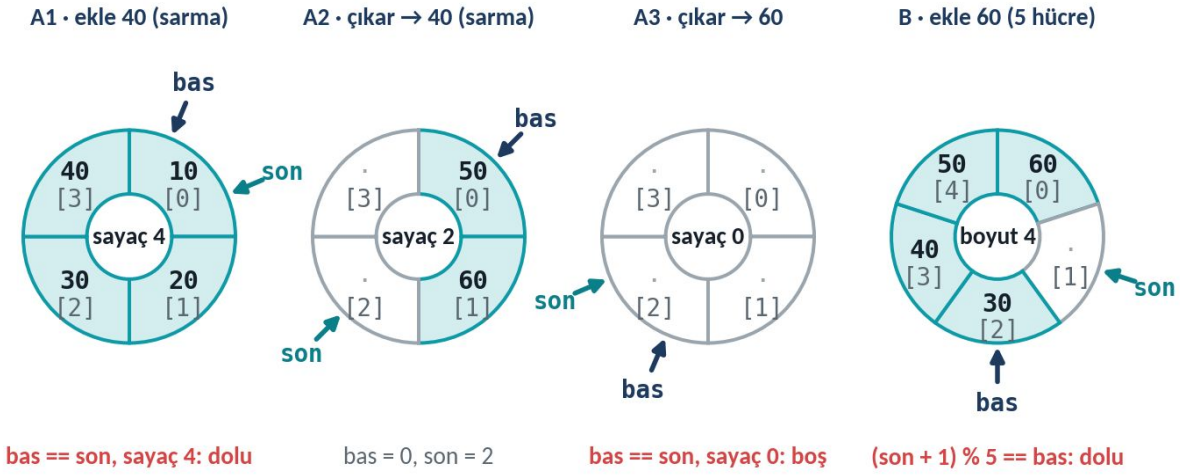
```
73      d = y.tepe
74      x = d.deger
75      y.tepe = d.sonraki
76      serbest_dugum += 1
77      y.boyut -= 1
```

9 · Test. 100 000 rastgele it/çıkar (tohum 42, kapasite 8): 5 688 taşma, 5 482 alt taşma, `son boyut = 2; it - çıkar = 2; tutarlı mı? evet`; Örnek 2 kâhine karşı: `kâhinle uyuşmazlık = 0`, `toplam: ayrılan = 150109, serbest = 150109`. **10 · Deney.** Süre ölçülmedi (her işlem O(1)); 100 000 it ve yok etmeden sonra `boş mu? evet; serbest bırakılan = 100005; fark = 0`. **11 · Varyantlar.** Dolunca büyüyen dizi (Kavram 1); tek dizide uçlardan büyüyen iki yığın; en küçüğü de O(1)'de veren yığın.

4.2 Dairesel tampon kuyruk (Örnek 8)

1 · Problem. Sabit kapasiteli FIFO kuyruğunda iki işlemi de O(1)'de, eleman kaydırmadan yapmak ve dolu/boş durumunu her zaman doğru bilmek. **2 · Fikir.** Dizi bir halkadır: `bas` ilk elemanın, `son` sıradaki boş hücrenin indisidir; ikisi de yalnız ileri gider ve sonda başa **sarar**: i ← (i + 1) mod kapasite. Sorun: boş kuyrukta `bas == son`'dur, ama dolu kuyrukta da `son` sarıp `bas`'a yetişir. Çözüm (A) ayrı bir **sayaç** (dolu: sayaç = kapasite) ya da (B) kapasite + 1 hücre ayırıp **bir hücreyi boş bırakmak**tır (dolu: (son + 1) mod hücre = bas). **3 · Elle izleme.** A izinde (Şekil 9.6) 40 eklenince `son` 3'ten 0'a sarar: `bas = 0, son = 0, sayaç = 4` (bas == son, sayaç 4: dolu); kuyruk boşalınca yine `bas = son = 2`, ama sayaç 0'dır. İki durumu yalnız sayaç ayırır.

Şekil 9.6
Dairesel tampon halkası (Örnek 8): yöntem A'da (sayaçlı, kapasite 4) son'un sardığı dolu durum, bas'ın sarması ve boş durum; sağda yöntem B'nin (5 hücre, biri boş) dolu durumu. bas == son hem doluda hem boşta doğrudur



> **TAHMİN ET · Son nereye sarar?**
>
> A kuyruğunda `bas = 2, son = 0, sayaç = 2`, hücreler [. . 30 40] iken 50 eklenir. Hangi hücreye yazılır, `son` ne olur? (Yanıt: `son` önceki eklemede 0'a sarmıştı; 50 hücre 0'a yazılır, `son = 1`, sayaç 3: [50 . 30 40]; sıra baştan başlar: 30 40 50.)

SÖZDE KOD · 4 · Sözde kod: dairesel tampon (yöntem A, sayaçlı)

```
algoritma EKLE(q, x)                          ▷ q.veri[0 .. kap − 1], bas, son, sayaç
    eğer q.sayaç = q.kap ise döndür YANLIŞ          ▷ dolu: reddet
    q.veri[q.son] ← x;  q.son ← (q.son + 1) mod q.kap;  q.sayaç ← q.sayaç + 1
algoritma ÇIKAR(q)
    eğer q.sayaç = 0 ise döndür (YANLIŞ, —)         ▷ boş
    x ← q.veri[q.bas];  q.bas ← (q.bas + 1) mod q.kap
    q.sayaç ← q.sayaç − 1;  döndür (DOĞRU, x)
```

5 · Doğruluk. Değişmez: Kuyruk sırasıyla veri[bas], veri[(bas + 1) % kap], …, veri[(bas + sayaç − 1) % kap]; 0 ≤ sayaç ≤ kap; ayrıca son = (bas + sayaç) mod kap. Ekleme sırayı sondan uzatır, çıkarma baştan kısaltır; mod yalnızca indisleri dizide tutar. Sayaçsız (yalnız `bas == son`) kuyrukta dolu kuyruk boş görünür ve eklemeler eskilerin üzerine yazar (Hata Avcısı 2). **6 · Analiz.** O(1) işlemler, O(kapasite) bellek; kapasite 2'nin kuvvetiyse mod yerine `& (kap - 1)` (Bölüm 3.5).

Tablo 9.4
Dairesel tampon kuyruk kimlik kartı

ALGORİTMA KİMLİK KARTI **Kuyruk (dairesel tampon)** *(circular buffer queue; ring buffer)*	
Strateji	FIFO (ilk giren ilk çıkar): bas'tan çıkarılır, (bas + sayaç) mod kapasite konumuna eklenir; indisler dizinin sonunda başa sarar, eleman kaydırılmaz
Girdi önkoşulu	Sabit kapasite; dolu ve boş durumunu ayırmak için sayaç ya da bir boş hücre
Zaman (en iyi · ortalama · en kötü)	O(1) · O(1) · O(1)
Ek bellek	O(kapasite)

Özellikler	bas == son hem boş hem dolu kuyrukta doğrudur: ayrımı sayaç ya da kapasite + 1 hücre yapar
Değişmez / doğruluk	Kuyruk sırasıyla veri[bas], veri[(bas + 1) % kap], …, veri[(bas + sayaç − 1) % kap]; 0 ≤ sayaç ≤ kap
Kütüphane karşılığı	C: yok · C++: `std::queue` · Python: `collections.deque`
Ne zaman kullanılır	Üst sınırı bilinen akış tamponları; benzetimde bekleme sırası; katman katman işleme
Ne zaman kullanılmaz	Üst sınır bilinmiyorsa; dolunca ne yapılacağı seçilmeden

7 · C gerçekleştirimi. (Örnek 8; `ilerle(q, i)` kapasite 2'nin kuvvetiyse `(i + 1) & (q->kapasite - 1)`, değilse `(i + 1) % q->kapasite` döndürür ve sarmaları sayar, 61–70. satırlar; çıkarma simetriktir):

c/H09_08_dairesel_kuyruk.c · satır 73–83

```c
bool kuyruk_ekle(Kuyruk *q, int x)
{
    if (kuyruk_dolu_mu(q)) {
        return false;
    }
    q->veri[q->son] = x;
    q->son = ilerle(q, q->son);
    q->sayac++;
    ekle_sayisi++;
    return true;
}
```

8 · Üç dil karşılaştırması. Python'da çıkarma `(başarı, değer)` döndürür; `%` yalnız negatif olmayan indislerle kullanıldığından üç dil aynı sonucu verir (negatif sayılarda farklıdır: Bölüm 8):

Python H09_08_dairesel_kuyruk.py · satır 70–76

```python
    if kuyruk_bos_mu(q):
        return False, -1
    x = q.veri[q.bas]
    q.bas = ilerle(q, q.bas)
    q.sayac -= 1
    cikar_sayisi += 1
    return True, x
```

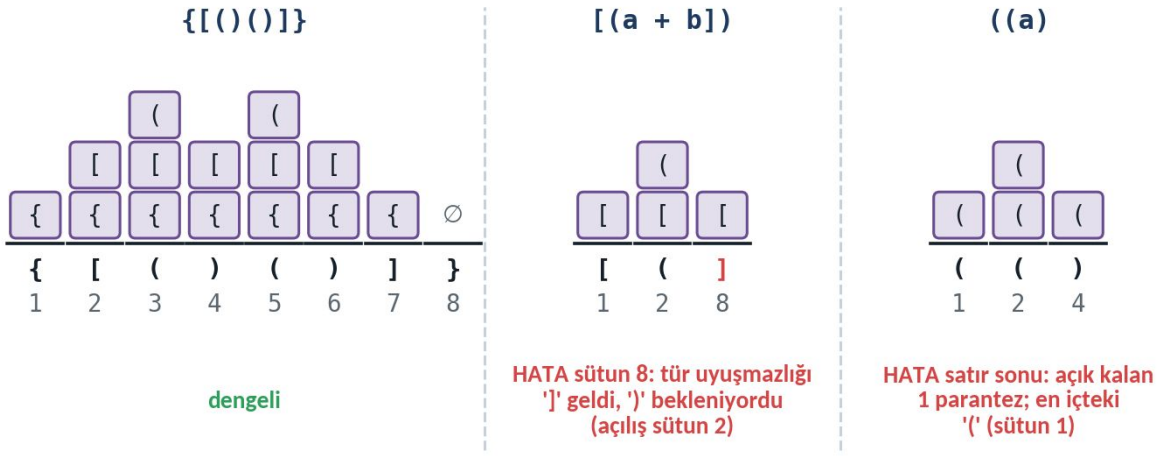
9 · Test. 100 000 rastgele işlem (%55 ekleme), A ve B kâhine karşı (C'de sarmayan dizi, C++'ta `std::queue`, Python'da `collections.deque`): kapasite 4'te 19 576 sarma, 1024'te 87; hepsinde uyuşmazlık 0. **10 · Deney.** Kaydırarak çıkaran dizi kuyruğu n = 10^5 için C -O2'de 204,03 kat yavaştır (Bölüm 6.5). **11 · Varyantlar.** Yöntem B (`bh_ekle`/`bh_cikar`, 123–141. satırlar; dört eklemeden sonra `(son + 1) % 5 == bas: dolu`); liste tabanlı kuyruk (Örnek 9); iki uçlu kuyruk (Örnek 13); üzerine yazan ya da bekleten tampon (Bölüm 9).

4.3 Parantez denetimi (Örnek 3)

1 · Problem. ( ) [ ] { } ayraçlarının dengeli olup olmadığını bulmak; değilse **ilk** hatanın sütununu ve türünü bildirmek: eşleşmeyen kapanış, tür uyuşmazlığı, açık kalan. **2 · Fikir.** En son açılan ilk kapanmalıdır. Tek tür ayraç için bir sayaç yeter; türler karışınca yetmez: `[()]`'de iki sayaç da dengelidir ama ayraçlar

çaprazdır. Yığın açık ayraçları açılış sırasıyla tutar, her kapanış tepedekiyle eşleşmelidir. **3 · Elle izleme.** Şekil 9.7: `{[()()]}` en çok üç derinliğe çıkıp boşalır; `[(a + b])`'de sütun 8'deki `]` gelince tepede `(` vardır (`HATA sütun 8: tür uyuşmazlığı: ']' bulundu, ')' bekleniyordu (açılış sütun 2)`); `((a)`'de girdi biter, yığında bir ayraç kalır.

Şekil 9.7
Parantez denetiminde yığın durumları (Örnek 3): dengeli, tür uyuşmazlığı ve açık kalan; her sütunun altında okunan ayraç ve sütun numarası



SÖZDE KOD · 4 · Sözde kod: parantez denetimi (ayraç olmayan karakterler atlanır)

```
algoritma PARANTEZ-DENETLE(s[1 .. n])
    Y ← boş yığın                              ▷ (tür, sütun) çiftleri
    her i ← 1 … n için
        eğer s[i] açılış ise Y'ye (s[i], i) it
        değilse eğer s[i] kapanış ise
            eğer Y boş ise döndür HATA(i, eşleşmeyen kapanış)
            (t, j) ← Y'den çıkar
            eğer t, s[i]'nin eşi değil ise döndür HATA(i, tür uyuşmazlığı, j)
    eğer Y boş değil ise döndür HATA(satır sonu, açık kalan, tepedeki)
    döndür DENGELİ
```

5 · Doğruluk. Değişmez: Yığın, okunan önekte henüz kapanmamış açılışları açılış sırasıyla tutar (tepede en içteki). Bir kapanış ancak en içteki açık ayraçla eşleşebilir; sonda yığın boşsa her açılış eşini bulmuştur. **6 · Analiz.** Her ayraç bir kez itilir, en çok bir kez çıkarılır: Θ(n); ilk hatada durulduğu için en iyi durum O(1) (ilk karakter `)`: `işlenen karakter = 1`). Bellek iç içelik derinliği kadardır; 100 000 karakterlik rastgele dengeli girdide `en büyük derinlik = 337` (Tablo 9.5).

Tablo 9.5
Parantez denetimi kimlik kartı

ALGORİTMA KİMLİK KARTI **Parantez denetimi** *(balanced brackets check)*	
Strateji	Yığınla eşleme: açılış itilir; kapanışta tepe çıkarılır ve türü karşılaştırılır; sonda yığın boş olmalı
Girdi önkoşulu	Yok — herhangi bir karakter dizisi; ( ) [ ] { } dışındaki karakterler atlanır
Zaman (en iyi · ortalama · en kötü)	O(1) · Θ(n) · Θ(n)

Ek bellek	O(n)
Özellikler	Üç hata türü: eşleşmeyen kapanış, tür uyuşmazlığı, açık kalan
Değişmez / doğruluk	Yığın, okunan önekte henüz kapanmamış açılışları açılış sırasıyla tutar (tepede en içteki)
Kütüphane karşılığı	C: yok · C++: yok (`std::stack<char>` ile kurulur) · Python: yok
Ne zaman kullanılır	Derleyici ve düzenleyicilerde ayraç eşleme; HTML/XML etiket dengesi
Ne zaman kullanılmaz	Dizgi ve yorum içindeki parantezler ayıklanmadan; tam sözdizimi denetimi için tek başına yetmez

7 · C gerçekleştirimi. (Örnek 3; yığın elemanı ayracın türünü ve sütununu taşır, böylece ileti açılışın yerini de söyler; döngüden sonra yığın boş değilse en içteki `y.veri[y.boyut - 1]` bildirilir):

C `c/H09_03_parantez_denetimi.c · satır 114–129`

```c
    for (int i = 0; i < n && r.durum == DENGELI; i++) {
        char c = s[i];
        islenen++;
        if (c == '(' || c == '[' || c == '{') {
            yigin_it(&y, (Acilis){c, i + 1});
        } else if (c == ')' || c == ']' || c == '}') {
            Acilis a;
            if (!yigin_cikar(&y, &a)) {
                r = (Sonuc){ESLESMEYEN_KAPANIS, i + 1, c, 0, 0, 0};
            } else if (a.tur != acilis_karsiligi(c)) {
                r = (Sonuc){TUR_UYUSMAZLIGI, i + 1, c,
                            kapanis_karsiligi(a.tur), a.sutun, 0};
            }
        } else {
            continue;  /* parantez olmayan karakterler atlanır */
        }
```

8 · Üç dil karşılaştırması. Python'da `c in "([{"` üç karşılaştırmanın yerini alır; erken çıkış koşulu `while`'ın koşulundadır:

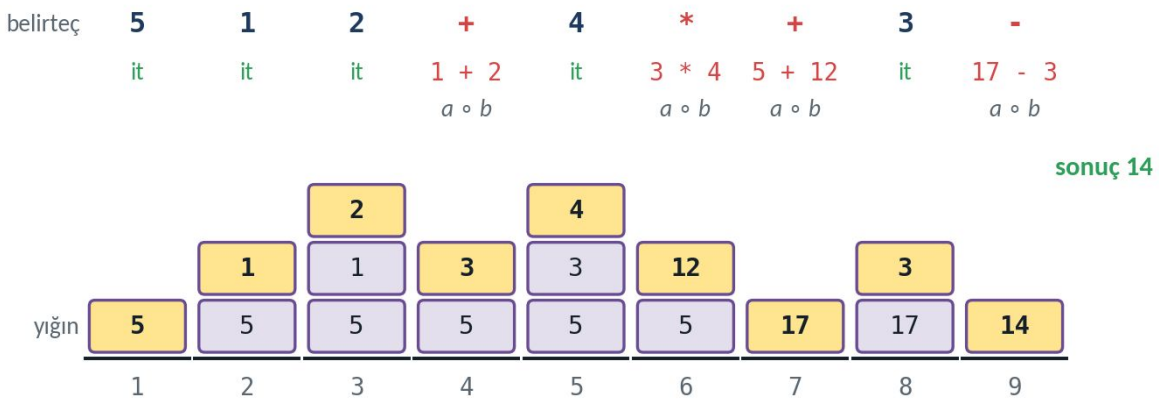
Python `H09_03_parantez_denetimi.py · satır 101–112`

```python
    while i < n and r.durum == DENGELI:
        c = s[i]
        islenen += 1
        if c in "([{":
            yigin_it(y, Acilis(c, i + 1))
        elif c in ")]}":
            tamam, a = yigin_cikar(y)
            if not tamam:
                r = Sonuc(ESLESMEYEN_KAPANIS, i + 1, c, "", 0, 0)
            elif a.tur != acilis_karsiligi(c):
                r = Sonuc(TUR_UYUSMAZLIGI, i + 1, c,
                          kapanis_karsiligi(a.tur), a.sutun, 0)
```

9 · Test. Girdi dosyasının on iki satırı üç hata türünü ve sınırları (boş satır, on düzey iç içe, ayraç sayılmayan `< >`) kapsar: `toplam 12 satır: dengeli 7, hatalı 5`. **10 · Deney.** Süre ölçülmedi (tek geçiş). **11 · Varyantlar.** HTML/XML etiket dengesi (Kavram 8: `<ul><li>bir</li><li>iki</ul>` → `</ul> bulundu, </li> bekleniyordu`); dizgi ve yorumlardaki ayraçları atlamak.

4.4 Sonek ifade değerlendirme (Örnek 4)

Şekil 9.8
*Sonek değerlendirmede yığın durumları (Örnek 4): 5 1 2 + 4 * + 3 − için her belirteçten sonra yığın; işleç sütununda yapılan işlem (a ∘ b)*



1 · Problem. Boşlukla ayrılmış bir sonek ifadeyi (tamsayılar, + − * / ^, tekli eksi ~) hesaplamak; hatada sütunu ve türü bildirmek. **2 · Fikir.** Sonekte işleç geldiğinde işlenenleri hesaplanmış ve hemen solundadır. Sayılar yığına itilir; ikili işleç iki değer çıkarır, sonucu iter (HP-35'in işlem yığını, Bölüm 1). Sıra önemlidir: önce çıkan **tepedeki** değer ikinci işlenendir (b), sonra çıkan birincidir (a); itilen a ∘ b'dir. **3 · Elle izleme.** Şekil 9.8: + işleci 1 ile 2'yi 3'e, * işleci 3 ile 4'ü 12'ye indirger; sonda tek değer kalır: `5 1 2 + 4 * + 3 - = 14 [it 9, çıkar 8, derinlik 3]` (5 sayı ve 4 sonuç itildi, 4 ikili işleç 8 değer çıkardı, derinlik en çok 3).

SÖZDE KOD · 4 · Sözde kod: sonek değerlendirme (sıfıra bölme, taşma ve negatif üs a ∘ b'de denetlenir)

```
algoritma SONEK-DEĞERLENDİR(t₁ t₂ … tₘ)
    Y ← boş yığın
    her t için (soldan sağa)
        eğer t sayı ise Y'ye t'yi it
        değilse eğer t = ~ ise  a ← Y'den çıkar;  Y'ye −a'yı it
        değilse eğer t ikili işleç ∘ ise
            eğer Y'de ikiden az değer var ise döndür HATA(eksik işlenen)
            b ← Y'den çıkar;  a ← Y'den çıkar      ▷ önce b (tepe), sonra a
            Y'ye a ∘ b'yi it
        değilse döndür HATA(geçersiz belirteç)
    eğer Y'de tek değer var ise döndür o değer
    döndür HATA(boş ifade ya da fazla işlenen)
```

5 · Doğruluk. Değişmez: Yığın, okunan önekteki tamamlanmış alt ifadelerin değerlerini soldan sağa sırayla tutar. İkili işleç en sağdaki iki tamamlanmış alt ifadeyi birleştirir, bu yüzden tepedeki iki değer onun işlenenleridir; geçerli bir ifadenin sonunda tek değer kalır. **6 · Analiz.** Θ(n) zaman, O(n) yığın; her belirteç tam bir it yapar (sayı kendini, işleç sonucunu iter). Hatalar ilk görüldükleri sütunda bildirilir: `1 0 / -> HATA sütun 5: sıfıra bölme`; `1 + -> HATA sütun 3: eksik işlenen`; `1 2 -> HATA ifade sonu: fazla işlenen (yığında 2 değer kaldı)`; `2 31 ^ -> HATA sütun 6: taşma (int aralığı dışında)`; `3 x + -> HATA sütun 3: geçersiz belirteç`; `2 1 ~ ^ -> HATA sütun 7: negatif üs` (Tablo 9.6).

Tablo 9.6
Sonek değerlendirme kimlik kartı

ALGORİTMA KİMLİK KARTI	**Sonek (RPN) değerlendirme** *(postfix evaluation; reverse Polish notation)*
Strateji	İşlenen yığını: sayı itilir; ikili işleç önce b'yi sonra a'yı çıkarır ve a ∘ b'yi iter; sonda yığında tek değer kalır
Girdi önkoşulu	Geçerli sonek ifade; sıfıra bölme ve taşma denetimi
Zaman (en iyi · ortalama · en kötü)	Θ(n) · Θ(n) · Θ(n)
Ek bellek	O(n)
Özellikler	Parantez ve öncelik gerekmez: soldan sağa tek geçiş. Örnek 4: 5 1 2 + 4 * + 3 - = 14
Değişmez / doğruluk	Yığın, okunan önekteki tamamlanmış alt ifadelerin değerlerini soldan sağa sırayla tutar
Kütüphane karşılığı	C: yok · C++: yok · Python: yok
Ne zaman kullanılır	RPN hesap makineleri; yığın tabanlı sanal makineler; Shunting-yard çıktısının hesaplanması
Ne zaman kullanılmaz	Kullanıcı araek yazıyorsa doğrudan değil, önce Shunting-yard ile dönüştürülerek

7 · C gerçekleştirimi. (Örnek 4, ikili işleç dalı; `islem_uygula` sonucu `long long`'da hesaplayıp `int` aralığını denetler, 99–124. satırlar). `||` kısa devre yapar: b çıkarılamazsa a denenmez:

C `c/H09_04_sonek_degerlendirme.c · satır 182–190`

```c
        } else if (uz == 1 && strchr("+-*/^", c) != NULL) {
            if (!yigin_cikar(&y, &b) || !yigin_cikar(&y, &a)) {
                hata = EKSIK_ISLENEN;  /* önce b (tepe), sonra a */
            } else {
                hata = islem_uygula(c, a, b, &r);
                if (hata == HATA_YOK) {
                    yigin_it(&y, r);
                }
            }
```

8 · Üç dil karşılaştırması. Python iki çıkarmayı ayrı satırlarda aynı sırayla yazar; bölme C ve C++'ta sıfıra doğru keser, Python'da `kesik_bol` aynı tanımı kurar (`7 2 ~ / = -3`; Bölüm 8):

Python `H09_04_sonek_degerlendirme.py · satır 146–154`

```python
            tamam, b = yigin_cikar(y)  # önce b (tepe), sonra a
            if tamam:
                tamam, a = yigin_cikar(y)
            if not tamam:
                hata = EKSIK_ISLENEN
            else:
                hata, r = islem_uygula(c, a, b)
                if hata == HATA_YOK:
                    yigin_it(y, r)
```

9 · Test. 20 ifade bütün hata türlerini kapsar (`toplam 20 ifade: geçerli 11, hatalı 9`); `2 30 ^ 1 - 2 * 1 +` tam `INT_MAX`'tır; üç dilin çıktısı bayt bayt aynıdır; asıl kâhin Örnek 6'dadır. **10 · Deney.**

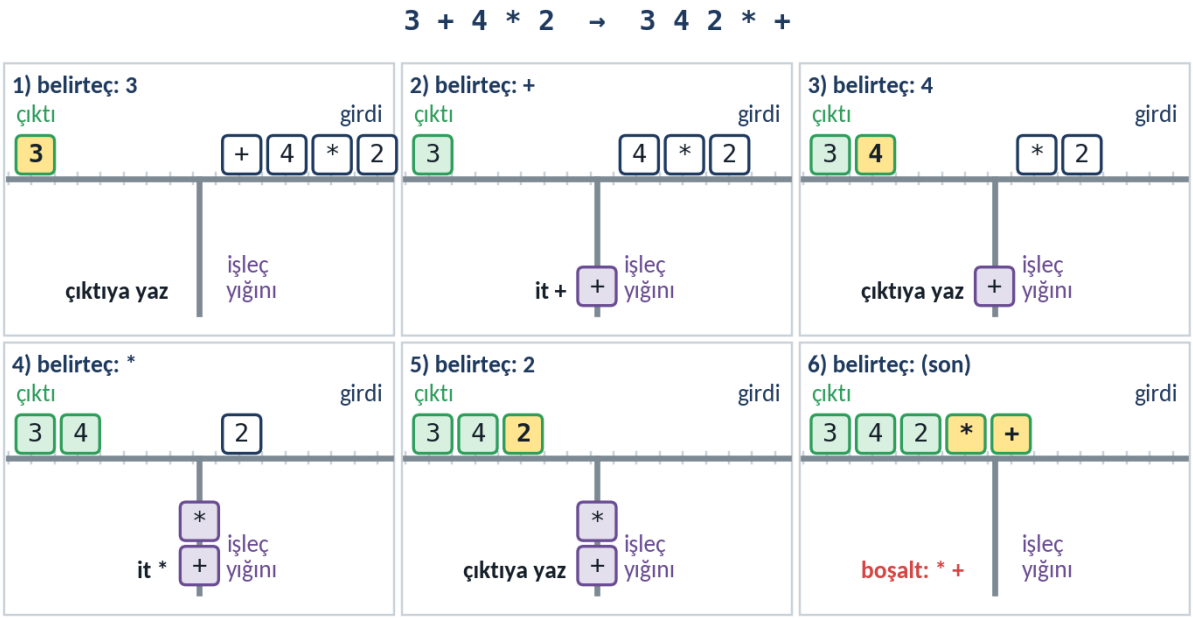
Sayaçlar Θ(n)'i gösterir: it = belirteç sayısı, çıkar = 2 × ikili + tekli işleç sayısı. **11 · Varyantlar.** Önek ifadeyi sağdan sola değerlendirmek; yığınla ifade ağacı kurmak.

4.5 Shunting-yard: araekten soneke dönüşüm (Örnek 5)

1 · Problem. Araek ifadeyi (sayılar, tek harfli adlar, + − * / ^, parantez, tekli eksi) öncelik ve birleşme yönüne uyarak sonek gösterime çevirmek; sözdizimi hatalarını sütunuyla bildirmek. **2 · Fikir.** Dijkstra'nın kavşağı (Bölüm 1): işlenenler doğrudan çıktıya gider (sonekte sıraları değişmez); işleçler **işleç yığınında** bekler. Yeni işleç gelince tepede ondan **önce uygulanması gereken** işleçler (daha yüksek öncelikli ya da eşit öncelikli ve gelen soldan birleşmeliyse) çıktıya aktarılır, sonra yeni işleç itilir. `(` bariyerdir, `)` eşine kadar boşaltır; girdi bitince kalanlar aktarılır. Öncelikler: + 1 (soldan), - 1 (soldan), * 2 (soldan), / 2 (soldan), ~ 3 (sağdan), ^ 4 (sağdan).

> **TAHMİN ET · Sıradaki belirteçte ne olur?**
>
> `3 + 4 * 2`'de üç belirteçten sonra çıktı `3 4`, yığın [+]; sıradaki `*`. + çıktıya mı aktarılır, * üstüne mi itilir? (Yanıt: it *: * işlecinin önceliği (2) tepedekinden (1) yüksektir; + beklemelidir, çünkü sağ işleneni 4 * 2'nin tamamıdır. Sonuç `3 + 4 * 2 -> 3 4 2 * +`.)

Şekil 9.9
*Shunting-yard manevra sahası, 3 + 4 * 2 (Örnek 5): her karede sağda kalan girdi, ortada işleç yığını (çıkmaz hat), solda çıktı ve yapılan eylem*



3 · Elle izleme. Şekil 9.9 içindeki son karede yığın tepeden boşaltılır: önce *, sonra +. Eşit öncelikte birleşme yönü karar verir: `2 ^ 3 ^ 2`'nin ikinci ^ işlecinde eylem "it ^"dir (tepedeki ^ bekler; sonuç `2 ^ 3 ^ 2 -> 2 3 2 ^ ^`), `a - b - c`'nin ikinci - işlecinde "çıkar: -; it -" (a − b önce hesaplanır; `a - b - c -> a b - c -`).

SÖZDE KOD · 4 · Sözde kod: Shunting-yard (işlenen/işleç sırası belirteçlemede denetlenir)

```
algoritma SHUNTING-YARD(belirteçler)
    Y ← boş işleç yığını;  Ç ← boş çıktı
    her t için (soldan sağa)
        eğer t işlenen ise Ç'ye t'yi ekle
```

```
        değilse eğer t = ( ya da t = ~ ise Y'ye t'yi it  ▷ ~ bir şey çıkarmaz
        değilse eğer t = ) ise
            iken Y'nin tepesi ( değil yap  Ç'ye Y'den çıkarılanı ekle
            eğer Y boş ise döndür HATA(eşleşmeyen ));  Y'den çıkar      ▷ ( atılır
        değilse                                                ▷ t ikili işleç
            iken tepe ü işleç ve ÖNCE-ÇIKAR(ü, t) yap  Ç'ye Y'den çıkarılanı ekle
            Y'ye t'yi it
    iken Y boş değil yap
        eğer tepe ( ise döndür HATA(kapanmamış ();  Ç'ye Y'den çıkarılanı ekle
    döndür Ç
fonksiyon ÖNCE-ÇIKAR(ü, t)
    döndür öncelik(ü) > öncelik(t) veya (öncelik(ü) = öncelik(t) ve t soldan
birleşmeli)
```

5 · Doğruluk. Değişmez: Çıktı, okunan önekteki tamamlanmış alt ifadelerin sonekidir; yığında aynı parantez düzeyinde her ikili işleç altındakinden daha yüksek öncelikli ya da eşit öncelikli sağdan birleşmelidir. Bir işleç ancak sağ işleneni tamamlanınca çıktıya geçebilir; sağ işlenen, ondan önce uygulanmayacak bir işleç, `)` ya da girdi sonu gelince tamamlanır: `ÖNCE-ÇIKAR` tam bu koşuldur. Dijkstra'nın "büyük-eşit" kuralı ^ için `2 3 ^ 2 ^` üretir (Hata Avcısı 3). **6 · Analiz.** Her işleç bir kez itilir, en çok bir kez çıkarılır: Θ(n), O(n) yığın; `a + b * c ^ d ^ e - f -> a b c d e ^ ^ * + f - [belirteç 11, it 5, çıkar 5, derinlik 4]` (Tablo 9.7).

Tablo 9.7
Shunting-yard kimlik kartı

ALGORİTMA KİMLİK KARTI **Shunting-yard: araek → sonek** *(shunting-yard algorithm; infix to postfix)*	
Strateji	İşleç yığını: işlenen doğrudan çıktıya; ikili işleç gelince tepedeki daha yüksek öncelikli (ya da eşit öncelikli ve soldan birleşmeli) işleçler çıktıya aktarılır, sonra işleç itilir; '(' bariyerdir, ')' eşini bulana kadar boşaltır
Girdi önkoşulu	Öncelik ve birleşme yönü tablosu: + − 1; * / 2; tekli eksi ~ 3 sağdan; ^ 4 sağdan
Zaman (en iyi · ortalama · en kötü)	Θ(n) · Θ(n) · Θ(n)
Ek bellek	O(n)
Özellikler	Örnek 5: 3 + 4 * 2 → 3 4 2 * +; 2 ^ 3 ^ 2 → 2 3 2 ^ ^; a - b - c → a b - c -
Değişmez / doğruluk	Çıktı, okunan önekteki tamamlanmış alt ifadelerin sonekidir; yığında aynı parantez düzeyinde her ikili işleç altındakinden daha yüksek öncelikli ya da eşit öncelikli sağdan birleşmelidir
Kütüphane karşılığı	C: yok · C++: yok · Python: yok (`ast`/`eval` algoritmayı göstermez)
Ne zaman kullanılır	Hesap makinesi ve yorumlayıcılarda araek ifade ayrıştırma; formül ve sorgu dilleri
Ne zaman kullanılmaz	İşleç dışı yapılar eklenmeden; bütün bir programlama dili için özyinelemeli iniş ayrıştırıcısı

7 · C gerçekleştirimi. (Örnek 5, ikili işleç dalı; yığında belirteç indisleri tutulur, `eylem` metni iz içindir; `once_cikar_mi(ust, gelen)` = `oncelik(ust) > oncelik(gelen) || (oncelik(ust) == oncelik(gelen) && !sag_birlesmeli(gelen))`, 108–112. satırlar):

C `c/H09_05_shunting_yard.c · satır 240–249`

```c
            while (yigin_tepe(&y, &j) && b[j].tur == ISLEC &&
                   once_cikar_mi(b[j].islec, t.islec)) {
                yigin_cikar(&y, &j);
                cikti[(*k)++] = j;
                strcat(eylem, eylem[0] == '\0' ? "çıkar: " : " ");
                belirtec_ekle(eylem, s, &b[j]);
            }
            strcat(eylem, eylem[0] == '\0' ? "it " : "; it ");
            belirtec_ekle(eylem, s, &t);
            yigin_it(&y, i);
```

8 · Üç dil karşılaştırması. Öncelik tablosu C'de `if` zinciri (94–103. satırlar), Python'da da `if` zinciridir; üyelik sınaması `op in "+-"` iki karşılaştırmanın yerini alır:

Python `H09_05_shunting_yard.py · satır 77–83`

```python
def oncelik(op):
    """Öncelik ve birleşme yönü tablosu"""
    if op in "+-":
        return 1
    if op in "*/":
        return 2
    return 3 if op == "~" else 4  # ~ tekli eksi, ^ üs
```

9 · Test. 22 ifade (`toplam 22 ifade: dönüştürülen 14, hatalı 8`), sözdizimi hataları dahil: `3 + 4 * (2 - 1 → HATA sütun 9: kapanmamış '('`; `3 + 4) * 2 → HATA sütun 6: eşleşmeyen ')'`; Örnek 6'nın kâhini dönüşümü değerlendirmeyle birlikte doğrular. **10 · Deney.** ^ soldan işlenirse `2 ^ 3 ^ 2` 64 olur (Hata Avcısı 3). **11 · Varyantlar.** Tekli eksi bir şey çıkarmadan itilir ve önceliği ^'ninkinden düşüktür: `-2 ^ 2 -> 2 2 ^ ~`; fonksiyon çağrısı ve virgül; doğrudan değer üretmek (iki yığın, Örnek 6).

4.6 Monotonik yığın: sonraki büyük eleman (Örnek 11)

1 · Problem. Her a[i] için sağındaki ilk büyük değeri bulmak (yoksa −1); kaba kuvvet en kötü durumda n(n − 1)/2 karşılaştırma yapar. **2 · Fikir.** Cevabı **henüz bulunmamış** elemanlar yığında bekler; gelen a[i], tepedeki daha küçük bekleyenlerin cevabıdır: hepsi çıkarılıp yanıtlanır, sonra a[i] beklemeye girer (benzetme: daha sıcak bir günü bekleyen günler). Bekleyenler tabandan tepeye hiç artmaz, bu yüzden **monotonik yığın**. **3 · Elle izleme.** a = [3 8 4 1 2 6 5 9 7] (Şekil 9.10): i = 1'de 8 gelince 3 yanıtlanır, i = 5'te 6 gelince 2 ve 4; sonda kalanların cevabı −1'dir: `sonraki büyük: 8 9 6 2 6 9 9 -1 -1`.

Şekil 9.10
Monotonik yığın şeridi (Örnek 11): her adımda dizi (mor: yığında bekleyen, yeşil: cevabı bulunan), çıkarılanlar ve cevapları, yığının değerleri; alt satırda sonuç



> TAHMİN ET · **9 gelince kaç eleman çıkar?**
>
> i = 6'dan sonra yığın [8 6 5], a[7] = 9. (Yanıt: 3 eleman, 5, 6, 8; cevapları 9; yığın [9]. Bu üçü bir daha hiç çıkarılmayacak.)

SÖZDE KOD · 4 · Sözde kod: sonraki büyük eleman

```
algoritma SONRAKİ-BÜYÜK(a[0 .. n − 1])
    Y ← boş yığın                                  ▷ cevabı bulunmamış indisler
    her i ← 0 … n − 1 için
        iken Y boş değil ve a[tepe(Y)] < a[i] yap  j ← Y'den çıkar;  sonuç[j] ← a[i]
        Y'ye i'yi it
    iken Y boş değil yap  sonuç[Y'den çıkar] ← −1      ▷ sağda büyüğü yok
```

5 · Doğruluk. Değişmez: yığındaki öğelerin sonraki büyüğü henüz bulunmamıştır ve yığın tabandan tepeye azalan (eşitlere izin veren) sıradadır. j, i adımında çıkıyorsa a[i] onun **ilk** büyüğüdür: arada a[k] > a[j] olan bir k bulunsaydı j o adımda çıkarılırdı (üstündekiler a[j]'den büyük değildir, yani a[k]'dan küçüktür ve önce onlar çıkar). **6 · Analiz.** İç döngü tek adımda n − 1 kez dönebilir, yine de toplam Θ(n)'dir. **Amortize argüman** (toplama yöntemi; Cormen vd., 2022): her indis bir kez itilir, en çok bir kez çıkarılır, dolayısıyla it + çıkar ≤ 2n ve karşılaştırma ≤ 2n − 1. n = 1000'de artan girdide 999 karşılaştırma ve derinlik 1; azalan girdide yine 999, derinlik 1 000; kaba kuvvet azalan girdide 499 500, rastgelede 7 254 (Tablo 9.8).

Tablo 9.8
Sonraki büyük eleman kimlik kartı

ALGORİTMA KİMLİK KARTI **Sonraki büyük eleman (monotonik yığın)** *(next greater element; monotonic stack)*	
Strateji	Monotonik yığın: cevabı henüz bulunmamış indisler yığında bekler; yeni eleman kendinden küçük tepeleri çıkarır ve onların cevabı olur

Girdi önkoşulu	Yok
Zaman (en iyi · ortalama · en kötü)	Θ(n) · Θ(n) · Θ(n)
Ek bellek	O(n)
Özellikler	Her indis bir kez itilir, en çok bir kez çıkarılır: it + çıkar ≤ 2n
Değişmez / doğruluk	Yığındaki değerler tabandan tepeye artmayan sıradadır ve hiçbirinin sonraki büyüğü henüz görülmemiştir
Kütüphane karşılığı	C: yok · C++: yok · Python: yok
Ne zaman kullanılır	Hisse senedi aralığı, "daha sıcak güne kaç gün", histogramdaki en büyük dikdörtgen, yağmur suyu tutma
Ne zaman kullanılmaz	Soru 'sonraki büyük' değil de keyfi aralık sorgusuysa

7 · C gerçekleştirimi. (Örnek 11; `kucuk_mu` karşılaştırmaları sayar, `coz` iz metnidir):

C `c/H09_11_sonraki_buyuk.c · satır 80–88`

```c
    for (int i = 0; i < n; i++) {
        coz[0] = '\0';
        while (y.boyut > 0 && kucuk_mu(a[y.veri[y.boyut - 1]], a[i])) {
            int j = yigin_cikar(&y);
            sonuc[j] = a[i];  /* a[j]'nin cevabı bulundu */
            snprintf(coz + strlen(coz), sizeof coz - strlen(coz), "%s%d",
                     coz[0] != '\0' ? " " : "", a[j]);
        }
        yigin_it(&y, i);
```

8 · Üç dil karşılaştırması. Python sürümü satır satır aynıdır; iz metni liste + `join` ile kurulur:

Python `H09_11_sonraki_buyuk.py · satır 65–71`

```python
    for i in range(n):
        coz = []
        while y.boyut > 0 and kucuk_mu(a[y.veri[y.boyut - 1]], a[i]):
            j = yigin_cikar(y)
            sonuc[j] = a[i]  # a[j]'nin cevabı bulundu
            coz.append(str(a[j]))
        yigin_it(y, i)
```

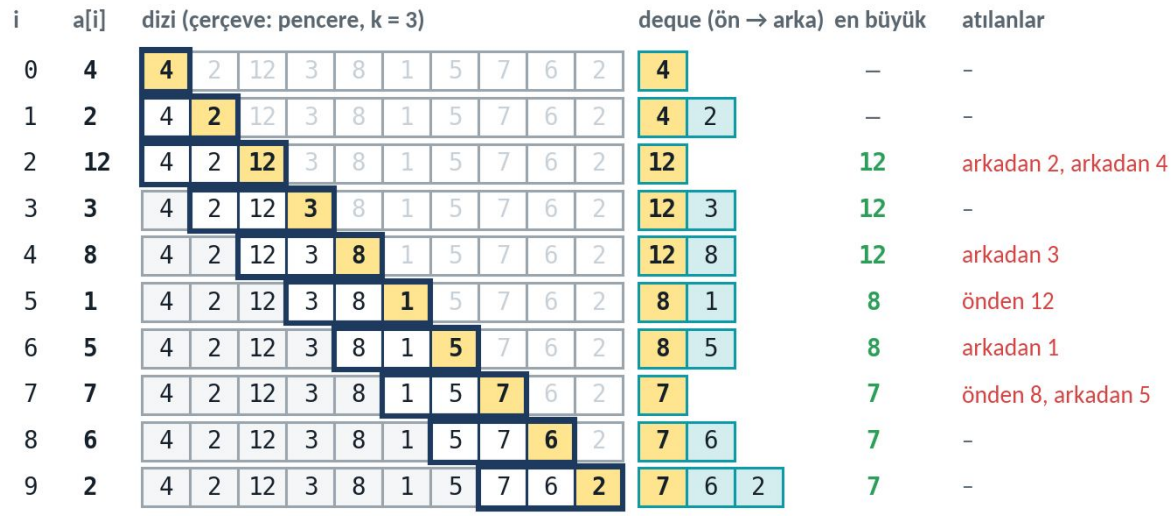
9 · Test. Kaba kuvvet kâhini, eşit değerli 2000 rastgele dizi: `kâhinle uyuşmazlık = 0, it + çıkar > 2n olan dizi = 0` (`<` yerine `<=` eşit değeri "büyük" sayardı). **10 · Deney.** n = 10^5 rastgele girdide it + çıkar 200 000 = 2n, karşılaştırma/n 2,000. **11 · Varyantlar.** Önceki büyük; sonraki küçük; dairesel dizi; histogramdaki en büyük dikdörtgen (Merak Kutusu).

4.7 Kayan pencere en büyüğü: monotonik iki uçlu kuyruk (Örnek 13)

1 · Problem. k uzunluklu her pencerenin (a[i .. i + k − 1]) en büyüğü; kaba kuvvet (n − k + 1)(k − 1) karşılaştırma. **2 · Fikir.** Yeni gelen a[i]'den küçük-eşit eski bir eleman **bir daha hiçbir pencerenin en büyüğü olamaz** (a[i] daha yenidir ve ondan küçük değildir): iki uçlu kuyruğun **arkasından** atılır; pencereden düşen indis **önden** atılır. Kalan değerler önden arkaya kesin azalır, öndeki pencerenin en büyüğüdür. **3 · Elle izleme.** a = [4 2 12 3 8 1 5 7 6 2], k = 3 (Şekil 9.11): 12 gelince 2 ve 4 arkadan atılır;

sonuç `pencere en büyükleri: 12 12 12 8 8 7 7 7`. Kuyruk kapasitesi k olan dairesel tampondur; öne ekleme `bas`'ı geriye sarar.

Şekil 9.11
Kayan pencere en büyüğü (Örnek 13, k = 3): her adımda pencere (çerçeve), iki uçlu kuyruğun değerleri (önden arkaya kesin azalan; sarı: ön, yani en büyük) ve atılanlar



SÖZDE KOD · 4 · Sözde kod: kayan pencere en büyüğü

```
algoritma KAYAN-EN-BÜYÜK(a[0 .. n − 1], k)
    D ← boş iki uçlu kuyruk            ▷ indisler; değerler önden arkaya azalan
    her i ← 0 … n − 1 için
        eğer D boş değil ve ön(D) = i − k ise önden çıkar      ▷ pencereden düştü
        iken D boş değil ve a[arka(D)] ≤ a[i] yap arkadan çıkar
        arkaya i'yi ekle;  eğer i ≥ k − 1 ise en_büyük[i − k + 1] ← a[ön(D)]
```

5 · Doğruluk. Değişmez: Deque'daki indisler artan, değerleri kesin azalan sıradadır ve hepsi geçerli penceredeki konumlardır. Önden atılan pencerenin dışındadır; arkadan atılanın sağında ondan küçük olmayan ve pencerede daha uzun kalacak bir eleman vardır. Değerler kesin azaldığı için en büyük öndedir. **6 · Analiz.** Her indis bir kez eklenir, en çok bir kez çıkarılır: Θ(n), k'dan bağımsız; bellek O(k). n = 10^5 rastgele girdide: k = 10: karşılaştırma/n 1,800, en büyük boy 8, kaba kuvvet 899 919; k = 100: karşılaştırma/n 1,980, en büyük boy 14, kaba kuvvet 9 890 199; k = 1 000: karşılaştırma/n 1,998, en büyük boy 18, kaba kuvvet 98 901 999; k = 10 000: karşılaştırma/n 2,000, en büyük boy 22, kaba kuvvet 899 919 999 (Tablo 9.9).

Tablo 9.9
Kayan pencere en büyüğü kimlik kartı

ALGORİTMA KİMLİK KARTI **Kayan pencere en büyüğü (deque)** *(sliding window maximum; monotonic deque)*	
Strateji	Monotonik iki uçlu kuyruk: yeni değerden küçük-eşit değerler arkadan atılır, pencereden çıkan indis önden atılır; ön eleman pencerenin en büyüğüdür
Girdi önkoşulu	1 ≤ k ≤ n
Zaman (en iyi · ortalama · en kötü)	Θ(n) · Θ(n) · Θ(n)

Ek bellek	O(k)
Özellikler	Örnek 13: a = [4 2 12 3 8 1 5 7 6 2], k = 3 → 12 12 12 8 8 7 7 7
Değişmez / doğruluk	Deque'daki indisler artan, değerleri kesin azalan sıradadır ve hepsi geçerli penceredeki konumlardır
Kütüphane karşılığı	C: yok · C++: `std::deque` · Python: `collections.deque`
Ne zaman kullanılır	Akan veride son k ölçümün en büyüğü/en küçüğü; dinamik programlamada pencere en iyisi
Ne zaman kullanılmaz	Pencere boyu sabit değilse ya da toplam/ortalama gibi çıkarılabilir bir özet isteniyorsa

7 · C gerçekleştirimi. (Örnek 13; iz kodu çıkarıldı):

C `c/H09_13_kayan_pencere.c · satır 131–145`

```c
    for (int i = 0; i < n; i++) {
        atilan[0] = '\0';
        if (!deque_bos_mu(&d) && deque_on(&d) == i - k) {
            onden_cikar(&d, &x);  /* pencereden çıkan indis */
            snprintf(atilan, sizeof atilan, "önden %d", a[x]);
        }
        while (!deque_bos_mu(&d) && kucuk_esit_mi(a[deque_arka(&d)], a[i])) {
            arkadan_cikar(&d, &x);  /* bundan sonra hiç en büyük olamaz */
            snprintf(atilan + strlen(atilan), sizeof atilan - strlen(atilan),
                     "%sarkadan %d", atilan[0] != '\0' ? ", " : "", a[x]);
        }
        arkaya_ekle(&d, i);
        if (i >= k - 1) {
            en_buyuk[i - k + 1] = a[deque_on(&d)];
        }
```

8 · Üç dil karşılaştırması. Öne ekleme: C ve C++'ta `-1 % 4 == -1` olduğu için `(bas - 1 + kapasite) % kapasite` yazılır; Python'da `-1 % 4 == 3`'tür, ama üç dilde aynı ifade korunmuştur:

Python `H09_13_kayan_pencere.py · satır 58–65`

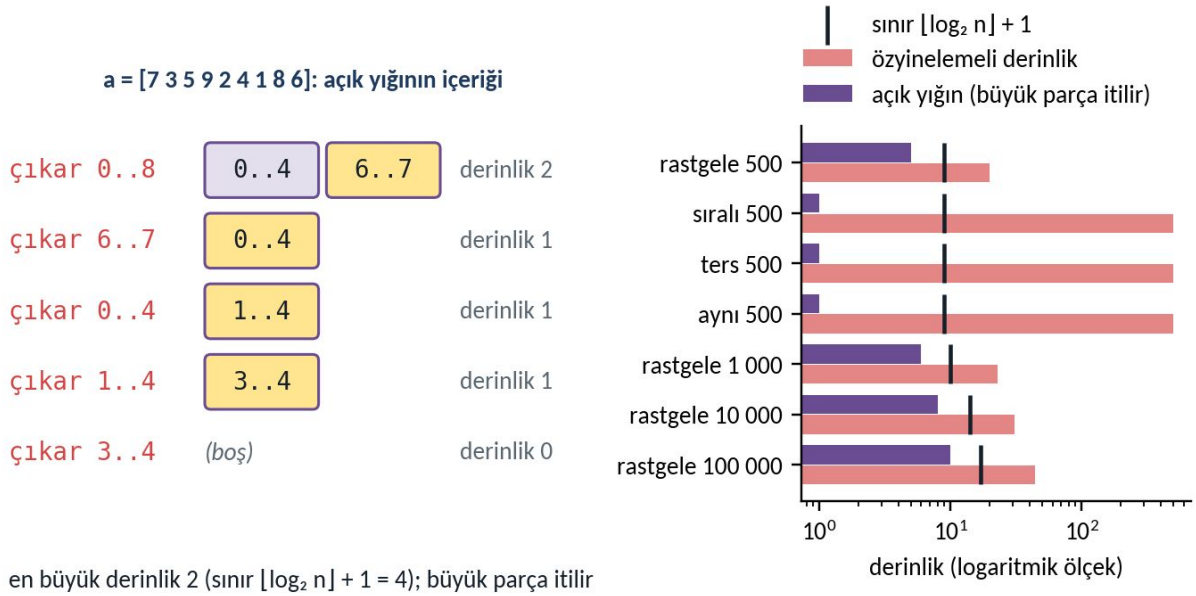
```python
def one_ekle(d, x):
    if d.sayac == d.kapasite:
        return False
    d.bas = (d.bas - 1 + d.kapasite) % d.kapasite  # geriye sarma
    d.veri[d.bas] = x
    d.sayac += 1
    boyut_guncelle(d)
    return True
```

9 · Test. Kaba kuvvet kâhini (2000 girdi, k = 1…n): `kâhinle uyuşmazlık = 0, ekle + çıkar > 2n ya da boy > k = 0`. **10 · Deney.** Yukarıdaki sayılar: kaba kuvvet k ile doğrusal büyür, kuyruk n başına 2 karşılaştırmada kalır. **11 · Varyantlar.** Pencere en küçüğü; `std::deque` ya da `collections.deque`; Alıştırma 9.13.

4.8 Özyinelemeden açık yığına: hızlı sıralama ve ızgara dolaşması (Örnek 12, Kavram 4)

1 · Problem. H6'nın hızlı sıralaması iki özyinelemeli çağrı yapar; Lomuto bölümlemesiyle sıralı girdide derinlik n'e çıkar. Derinliği girdiden bağımsız sınırlamak ve özyinelemeyi açık yığınla yazmak istiyoruz. **2 · Fikir.** Özyinelemeli çağrı "bu aralığı sonra sırala" notudur; notları yığına aralık olarak iteriz. Bölümlemeden sonra **büyük** parça itilir, **küçük** parçayla hemen devam edilir; işlenen parça her seferinde en az yarıya indiği için yığında en çok $\lfloor \log_2 n \rfloor + 1$ aralık bulunur. **3 · Elle izleme.** a = [7 3 5 9 2 4 1 8 6] (Şekil 9.12): ilk bölümlemede program `pivot 6 -> p = 5; it 0..4, sırada 6..8; yığın [0..4]` yazar: pivot p indisine yerleşir, büyük parça itilir, küçüğüyle hemen devam edilir; sonunda `karşılaştırma = 19, takas = 16, it = 5, çıkar = 5, en büyük derinlik = 2 (sınır 4)`.

Şekil 9.12
Açık yığınlı hızlı sıralama (Örnek 12): solda her çıkarmadan sonra yığındaki aralıklar; sağda yığın derinliği (logaritmik ölçek): açık yığın, sınır $\lfloor \log_2 n \rfloor + 1$ ve özyinelemeli sürüm



SÖZDE KOD · 4 · Sözde kod: açık yığınlı hızlı sıralama

```
algoritma YIĞINLI-HIZLI-SIRALA(a[0 .. n − 1])
    Y ← boş yığın (aralıklar);  eğer n > 1 ise Y'ye (0, n − 1) it
    iken Y boş değil yap
        (lo, hi) ← Y'den çıkar
        iken lo < hi yap
            p ← BÖLÜMLE(a, lo, hi)                    ▷ Lomuto, pivot a[hi]
            eğer p − lo < hi − p ise  büyük ← (p + 1, hi);  hi ← p − 1
            değilse  büyük ← (lo, p − 1);  lo ← p + 1
            eğer büyük en az iki eleman ise Y'ye büyük'ü it
```

5 · Doğruluk. Değişmez: yığındaki aralıklar ve işlenen aralık ayrıktır, her biri dizinin geri kalanına göre doğru yerdedir; dışlarındaki konumlar kesinleşmiştir. Derinlik: bir aralık itildiğinde işlenmeye devam eden kardeşi en çok yarı boyuttadır ve üstüne itilecek her aralık onun içinden gelir; tabandan k. aralık en çok $n/2^{k-1}$ elemanlı bir aralıktan geldiği ve en az iki elemanlı olduğu için yığında $\lfloor \log_2 n \rfloor + 1$'den fazla aralık olamaz. **6 · Analiz.** Karşılaştırma sayısı özyinelemeli sürümle aynıdır (ortalama $\Theta(n \log n)$, en kötü

$\Theta(n^2)$); bellek O(log n) **güvenceli.** n = 500 sıralı girdide 124 750 karşılaştırma, açık yığın 1, özyinelemeli derinlik 500; n = 10^5 rastgelede yığın 10 (sınır 17), özyinelemeli 44.

7 · C gerçekleştirimi. (Örnek 12; dış döngü `while (yigin_cikar(&y, &r))` aralığı çıkarır; yığın kapasitesi baştan $\lfloor \log_2 n \rfloor + 1$ ayrılır):

C `c/H09_12_yiginli_hizli_siralama.c · satır 159–172`

```c
        while (lo < hi) {
            int pivot = a[hi];
            int p = bolumle(a, lo, hi);
            Aralik buyuk;
            if (p - lo < hi - p) {  /* sol küçük: sağı it */
                buyuk = (Aralik){p + 1, hi};
                hi = p - 1;
            } else {                /* sağ küçük: solu it */
                buyuk = (Aralik){lo, p - 1};
                lo = p + 1;
            }
            if (buyuk.lo < buyuk.hi) {
                yigin_it(&y, buyuk);
            }
```

8 · Üç dil karşılaştırması. Python'da aralık demettir (`lo, hi = r`); `while tamam:` döngüsü C'nin `while (yigin_cikar(...))` kalıbının karşılığıdır:

Python `H09_12_yiginli_hizli_siralama.py · satır 119–126`

```python
            if p - lo < hi - p:  # sol küçük: sağı it
                buyuk = (p + 1, hi)
                hi = p - 1
            else:                # sağ küçük: solu it
                buyuk = (lo, p - 1)
                lo = p + 1
            if buyuk[0] < buyuk[1]:
                yigin_it(y, buyuk)
```

9 · Test. Kâhin (`qsort`, `std::sort`, `sorted`) dört girdi türünde ve n = 500 … 10^5'te "evet", taşma 0. **10 · Deney.** n = 500 rastgelede açık yığın 5, özyinelemeli 20; sıralı girdide özyinelemeli 500, açık yığın 1 (küçük parça boştur). **11 · Varyantlar.** Küçük parçaya özyinele, büyüğe döngüyle devam et (aynı sınır); introsort.

Izgara dolaşması (Kavram 4). Bitişik açık hücrelerden bölge bulmak ("boya kovası") doğal olarak özyinelemelidir ve derinlik yol uzunluğu kadardır: `özyinelemeli: 3 bölge; en büyük çağrı derinliği = 30`; aynı ızgara açık yığınla `3 bölge; en büyük yığın boyu = 9` (hücre itilirken işaretlenir, böylece her hücre en çok bir kez girer). Yığın yerine **kuyruk** kullanılınca uzaklıklar katman katman çıkar (`ulaşılan 52 hücre, en uzak 18 adım, en büyük kuyruk 6`): H11'deki DFS ile BFS'nin farkı budur. 2000 × 2000 boş ızgarada özyinelemeli sürüm bu makinede, bu çalıştırmada SIGSEGV ile sonlandı; açık yığın çalıştı (`2000 x 2000 boş ızgara, açık yığın: 4000000 hücre, en büyük yığın 1999001`).

5. Çözümlü Örnekler

Bu bölümün dört örneği Bölüm 4'ün yapılarını birleştirir: bir hesap makinesi (iki yığın, Örnek 6), bir düzenleyicinin geri al / yinele düğmeleri (iki yığın, Örnek 7), liste tabanlı kuyruk ve katman katman işleme (Örnek 9) ve bir kayıt gişesinin benzetimi (dairesel tampon kuyruk, Örnek 10).

5.1 Basit hesap makinesi: dönüşüm ve değerlendirme (Örnek 6)

Örnek 6 Bölüm 4.5 ile 4.4'ü birleştirir ve Haftanın Büyük Sorusunu yanıtlar: araek ifade belirteçlere ayrılır, Shunting-yard ile sonek sıraya dizilir ve sonek dizisi işlenen yığınıyla hesaplanır. Belirteçleme sırasında işlenen ve işleç sırası denetlenir (`3 + * 4`: işlenen bekleniyordu); `-` işlenen beklenirken geldiyse tekli eksi (~) sayılır. Hesaplama Örnek 4'ün tanımlarını aynen kullanır: `int` aralığı, sıfıra doğru kesen bölme, negatif üs hatası. Program, ilk hatalı ifadede değil girdi sonunda çıkar; çıkış kodu üç dilde aynıdır: 0 (hepsi geçerli), 1 (hatalı ifade var), 2 (hatalı kullanım).

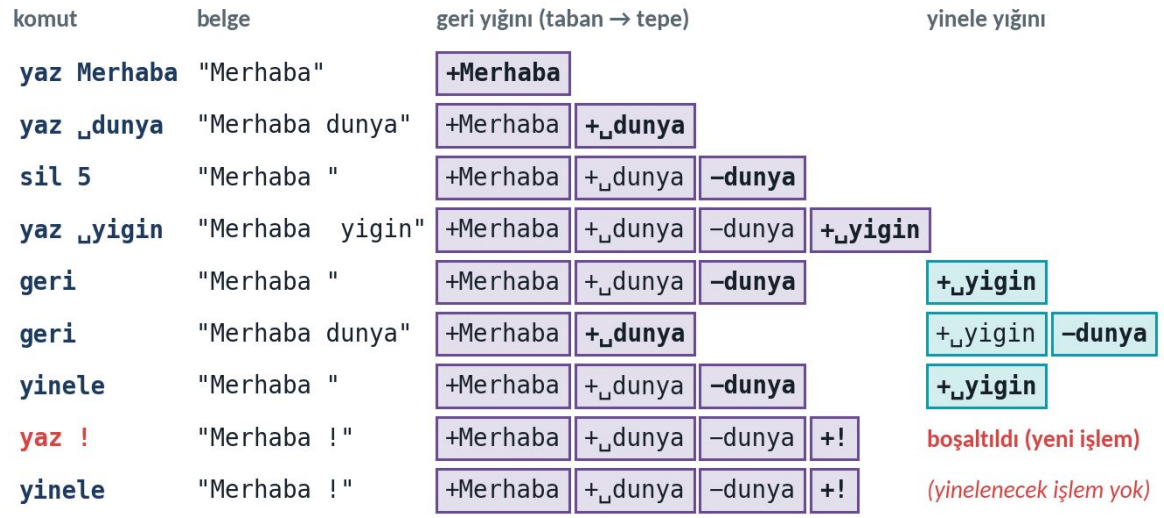
Girdi dosyasından seçilmiş satırlar: `100 / 7 * 7 + 100 - 100 / 7 * 7 = 100`; `-7 / 2 = -3`; `-2 ^ 2 = -4`; `46341 * 46341 -> HATA sütun 7: taşma (int aralığı dışında)`. İlk satır, tamsayı bölmesinin kesmesi yüzünden 100 / 7 * 7 = 98 olduğu hâlde ifadenin yine 100 verdiğini gösterir. Özet: `toplam 21 ifade: geçerli 13, hatalı 8`; aynı satırdaki yığın işlemi sayaçları (`it 39320, çıkar 37414`) yalnız bu 21 ifadeyi değil, programın bütün çalışmasını, aşağıdaki 2000 kâhin ifadesini de kapsar.

Kâhinli sınama. Program tohum 42 ile derinliği en çok 3 olan rastgele ifade ağaçları üretir, her ağacı hem doğrudan (ağaç üzerinde özyinelemeyle) hem de metne yazıp hesap makinesinden geçirerek hesaplar; iki yolun sonucu ya da hata türü aynı olmalıdır. Örnek: `8 / -1 * (-3 / 6) / (-5 / -6 * (-2 - 0)) -> HATA: sıfıra bölme (kâhin: sıfıra bölme)`. Toplam: `2000 ifade: sıfıra bölmeli 210, kâhinle uyuşmazlık 0`. Bu kâhin Örnek 4 ve 5'i de birlikte sınar: belirteçleme, öncelik, birleşme yönü, tekli eksi, parantez ve bölme tanımındaki herhangi bir hata uyuşmazlık olarak görünürdü.

5.2 Geri al / yinele: iki yığın (Örnek 7)

Bir metin düzenleyicide her işlem (yaz, sil) bir kayıt olarak **geri yığınına** itilir. Geri al tepedeki kaydı çıkarır, tersini uygular (yazılanı siler, silineni geri yazar) ve kaydı **yinele yığınına** iter; yinele bunun simetriğidir. Kritik kural şudur: **yeni bir işlem yinele yığınını boşaltır.** Geri alınmış işlemlerden sonra yeni bir şey yazılınca geçmiş dallanır; eski "ileri" artık bu belgeye uygulanamaz. Örnek 7'nin izinde "yaz !" adımı bunu gösterir: ondan önce yinele yığınında [+yigin] vardı, sonra boştur ve hemen ardından gelen yinele `yinelenecek işlem yok` der (Şekil 9.13).

Şekil 9.13
Geri al / yinele (Örnek 7): her komuttan sonra belge, geri yığını ve yinele yığını (tabandan tepeye; + yaz, − sil). Yeni işlem (yaz !) yinele yığınını boşaltır



c/H09_07_geri_al_yinele.c · satır 128–133

```c
static void yeni_islem(Duzenleyici *e, const Islem *i)
{
    uygula(&e->belge, i);
    yigin_it(&e->geri, i);
    temizlenen += yigin_yok_et(&e->yinele);
}
```

Kayıt silinen metni de taşır; geri al onu yeniden yazar. Bu yüzden “sil 100” belgede kalan her şeyi silse bile geri alınabilir (izin sonundaki iki adım). Komut izinin özeti: `sayaçlar: it = 16, çıkar = 10, yeni işlemle temizlenen yinele kaydı = 3, en büyük derinlik = 4`. **Kâhin:** rastgele 20 000 komutta her adımın sonunda belgenin bütün geçmişi bir diziye kaydedilir; geri al bir önceki duruma, yinele bir sonrakine eşit olmalıdır: `durum geçmişi kâhiniyle uyuşmazlık = 0`; `düğüm: ayrılan = 16141, serbest = 16141`. Mini-OBS 2 v2.8 aynı kalıpla kayıt ekle, sil ve güncelle işlemlerini geri alır (Bölüm 10).

5.3 Liste tabanlı kuyruk ve katman katman işleme (Örnek 9)

H8'in baş ve son işaretçili listesi kuyruğun iki işlemini de O(1) yapar: ekleme sona (`son->sonraki = d; son = d`), çıkarma baştan (`bas = bas->sonraki`). Tek ince nokta son düğümün çıkarılmasıdır: liste boşalınca `bas` NULL olur, `son` da NULL yapılmazsa serbest bırakılmış düğümü göstermeye devam eder ve sonraki ekleme oraya yazar (sarkan işaretçi).

c/H09_09_liste_kuyruk.c · satır 81–87

```c
    Dugum *d = q->bas;
    *x = d->deger;
    q->bas = d->sonraki;
    if (q->bas == NULL) {
        q->son = NULL;  /* boşaldı: son da NULL olmalı */
    }
    free(d);
```

Örnek 9'un iki uygulaması kuyruğun düzenini kullanır. **Katman katman işleme:** 1'den başlanır, çıkarılan her x için 2x ve 2x + 1 (≤ 15) eklenir; bir düzeyin bütün elemanları bir sonrakinden önce çıkar (`düzey 3: 8 9 10 11 12 13 14 15`). **Sıcak patates** (Josephus problemi): n kişi bir kuyruktadır; baştaki kuyruktan çıkıp sona girer, her k'ıncı kişi elenir: `n = 7, k = 3: çıkış sırası 3 6 2 7 5 1; kalan 4`; `n = 10, k = 2: çıkış sırası 2 4 6 8 10 3 7 1 9; kalan 5`. Kâhinli rastgele sınama: `100000 işlem: boş reddi 5, en büyük boyut 4462, son boyut 4459, kâhinle uyuşmazlık 0`; `toplam: ekle = 52285, çıkar = 47825; düğüm ayrılan = 52285, serbest = 52285`.

5.4 Kayıt gişesi simülasyonu (Örnek 10)

Model. Bir kayıt gişesinde k memur tek bir FIFO kuyruğuna hizmet verir. Zaman tamsayı saniyedir. Her saniye bir öğrenci p olasılıkla gelir (p = `gelis_milyonda` / 10^6, ortak LCG ile `rastgele_aralik(0, 999999) < gelis_milyonda`) ve kuyruğun sonuna girer; ardından boş her gişe kuyruğun başındakini alır ve ona [60, 180] saniyede düzgün dağılmış bir hizmet süresi çekilir (ortalama E[S] = 120 s). Saniyede ortalama geliş λ = p olduğundan **yük oranı** (her gişenin meşgul olduğu zaman oranı) ρ = λ · E[S] / k'dır; ρ ≥ 1 ise gişeler gelenlere yetişemez ve kuyruk sınırsız büyür. Kuyruk Örnek 8'in sayaçlı dairesel tamponudur; geliş anını taşır, bekleme = hizmete başlama anı − geliş anı. Döngünün üçüncü adımı (aşağıda gösterilmeyen) meşgul gişelerin kalan süresini bir azaltır ve kuyruk uzunluğunu toplar:

`c/H09_10_gise_simulasyonu.c · satır 84–111`

```c
    for (int t = 0; t < sure; t++) {
        if (rastgele_aralik(0, 999999) < gelis_milyonda) {  /* 1) geliş */
            s.gelen++;
            bool tamam = kuyruk_ekle(&q, t);
            s.reddedilen += !tamam;
            if (olay++ < olay_siniri) {
                printf("  t = %4d  öğrenci %lld geldi%s; kuyruk %d\n", t,
                       s.gelen, tamam ? "" : " (kuyruk dolu: geri döndü)",
                       q.sayac);
            }
        }
        for (int g = 0; g < k; g++) {  /* 2) boş gişe sıradakini alır */
            int gelis;
            if (kalan[g] == 0 && kuyruk_cikar(&q, &gelis)) {
                int bekleme = t - gelis;
                s.hizmet_alan++;
                s.toplam_bekleme += bekleme;
                if (bekleme > s.en_uzun_bekleme) {
                    s.en_uzun_bekleme = bekleme;
                }
                kalan[g] = rastgele_aralik(HIZMET_EN_AZ, HIZMET_EN_COK);
                if (olay++ < olay_siniri) {
                    printf("  t = %4d  gişe %d: öğrenci %lld (bekleme %d s,"
                           " hizmet %d s); kuyruk %d\n", t, g + 1,
                           s.hizmet_alan, bekleme, kalan[g], q.sayac);
                }
            }
        }
```

İlk olaylarda (k = 2, p = 0,02, ρ = 1,2) 1. öğrenci t = 119'da gelir ve hemen hizmete girer; 4. öğrenci t = 200'de gelip 110 s bekler. ρ > 1 olduğu için kuyruk büyümeye devam eder.

Tablo 9.10
Bir kayıt günü (8 saat = 28 800 s, saatte ortalama 36 öğrenci, tohum 42): gişe sayısının etkisi (Örnek 10 çıktısının 2. bölümü)

k	ρ	gelen	hizmet	kalan	ort. bekleme (s)	en uzun (s)	ort. kuyruk L	λ·W	doluluk (%)
1	1,20	294	241	53	2 502,20	5 576	25,47	25,54	99,0
2	0,60	292	292	0	29,65	248	0,30	0,30	61,8
3	0,40	289	289	0	2,79	56	0,03	0,03	41,8
4	0,30	290	290	0	0,29	32	0,00	0,00	31,6

Tek gişede (ρ = 1,20) gün sonunda 53 öğrenci kuyruktadır, ortalama bekleme 2 502,20 s'dir ve gişe zamanın %99,0'ında meşguldür; ikinci gişe ρ'yu 0,60'a, beklemeyi 29,65 s'ye indirir (84,4 kat). **Little yasası** (L = λW; Little, 1961) her satırda denetlenir: k = 1'de L = 25,47, λ·W = 25,54 (fark, beklemesi bitmediği için W'ye girmeyen 53 öğrenciden gelir). Tek gişede ρ 1'e yaklaşırken (çıktının 3. bölümü): ρ = 0,50: 74,48 s, ρ = 0,70: 197,35 s, ρ = 0,80: 311,35 s, ρ = 0,90: 359,67 s, ρ = 0,95: 481,02 s. Tek günlük benzetim gürültülüdür (ρ = 0,8'den 0,9'a bekleme yalnızca %15,5 artar); güvenilir eğriler için uzun benzetim ve birden çok tohum gerekir: Haftanın Deneyi.

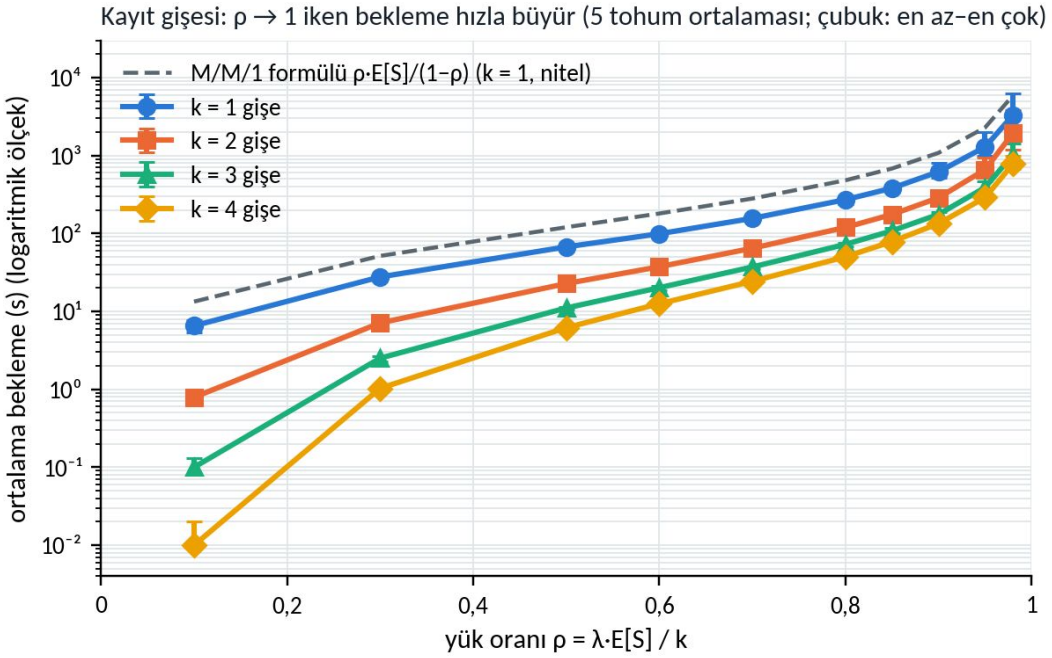
6. Haftanın Deneyi: Gişe Sayısı, Yük Oranı ve Kuyruk Gerçekleştirimi

6.1 Ölçüm düzeni

(a) Kayıt gişesi: Örnek 10'un benzetimi k = 1 … 4 gişe ve on yük oranı (ρ = 0,10 … 0,98) için, her biri 2 000 000 s (≈ 23 gün) ve beş tohumla çalıştırıldı; ayrıca sabit geliş hızlarında (saatte 12 … 48 öğrenci) dört gişe sayısı karşılaştırıldı. Sayılar deterministiktir: C ve C++ CSV'leri bayt bayt aynıdır, Python k = 1 … 4, ρ = 0,90, tohum 1 için aynı satırları üretti. **(b) Kuyruk gerçekleştirimleri:** n ekle + n çıkar ($n = 10^3$ … 10^5): her çıkarmada kalanları sola **kaydıran** dizi, **dairesel** tampon ve Python'da `list.pop(0)` ile `collections.deque.popleft()`; C ve C++ -O2 (C ayrıca -O0), Python 3.11.15. Ortam: Intel(R) Xeon(R) Processor @ 2.80GHz, 2 çekirdekli paylaşımlı sanal makine, gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0. Süreler **işlemci zamanıdır** (duvar saati başka süreçleri bekleme süresini de içerir); her nokta 1 ısınma + 5 tekrarın ortancası, iki oturumdan küçük olanıdır. Tahmini süresi 15 s'yi aşan büyüklükler ölçülmedi, n^2 modeliyle **kestirildi** ve "≈ … (k)" ile işaretlendi. Gişe benzetimleri toplam 25,3 s, kuyruk ölçümleri 77,4 s sürdü.

6.2 Yük oranının etkisi

Şekil 9.14
Ortalama bekleme ve yük oranı (logaritmik y ekseni): k = 1 … 4, beş tohumun en az–en çok aralığı çubukla; kesikli: M/M/1 ortalaması (yalnızca biçim karşılaştırması)



Tablo 9.11
Ortalama bekleme (s): beş tohumun ortalaması, parantezde en az–en çok (her tohum 2 000 000 s; h09_ozet_gise.csv)

ρ	k = 1	k = 2	k = 3	k = 4
0,50	67,20 (64,1–71,7)	22,72 (21,3–23,9)	11,05 (10,5–11,3)	6,17 (5,8–6,3)
0,80	271,00 (238,6–294,9)	119,36 (109,0–125,6)	72,41 (68,7–76,3)	50,08 (46,1–53,5)
0,90	618,00 (514,7–802,6)	286,42 (237,9–308,0)	175,63 (162,1–188,6)	133,75 (125,8–139,0)
0,95	1 284,40 (991,9–1 975,7)	657,70 (519,6–942,1)	388,27 (319,1–462,3)	288,47 (268,6–326,9)
0,98	3 295,04 (1 809,7–6 260,0)	1 939,14 (1 201,4–3 804,6)	1 027,63 (686,2–1 429,2)	783,09 (668,8–998,1)

Bekleme ρ ile doğrusal değil, **1/(1 − ρ)** biçiminde büyür (Şekil 9.14). Tek gişede ρ 0,5'ten 0,9'a çıkınca ortalama bekleme 9,20 katına, 0,9'dan 0,98'e çıkınca (yükte yalnızca %8,9 artış) yeniden 5,33 katına çıkar; 1/(1 − ρ) çarpanı 2, 10 ve 50'dir. Yük arttıkça tohumlar arası fark da büyür (ρ = 0,98, k = 1: en az ile en çok arasında 3,5 kat): sistem uzun süre boşalmaz ve tek bir yoğunluk ortalamayı belirler. “%90 dolu gişe verimlidir” sezgisi gişe için doğrudur, bekleyen için yanlıştır.

6.3 Gişe sayısının etkisi

Tablo 9.12
Aynı geliş hızında gişe sayısı: ortalama bekleme (s) ve yük oranı; † ρ ≥ 1, kuyruk sınırsız büyür (değer benzetim süresine bağlıdır; h09_ozet_gise_sabit.csv)

λ (saatte)	k = 1	k = 2	k = 3	k = 4
12	43,83 (ρ = 0,40)	3,07 (ρ = 0,20)	0,24 (ρ = 0,13)	0,01 (ρ = 0,10)

λ (saatte)	k = 1	k = 2	k = 3	k = 4
24	271,00 (ρ = 0,80)	13,27 (ρ = 0,40)	1,79 (ρ = 0,27)	0,26 (ρ = 0,20)
27	618,00 (ρ = 0,90)	17,54 (ρ = 0,45)	2,51 (ρ = 0,30)	0,40 (ρ = 0,23)
28,8	1 648,19 (ρ = 0,96)	20,63 (ρ = 0,48)	3,01 (ρ = 0,32)	0,51 (ρ = 0,24)
36	167 858,20 (ρ = 1,20) †	37,38 (ρ = 0,60)	5,60 (ρ = 0,40)	1,03 (ρ = 0,30)
48	380 124,14 (ρ = 1,60) †	119,36 (ρ = 0,80)	13,55 (ρ = 0,53)	2,82 (ρ = 0,40)

Saatte 27 öğrencide tek gişe ρ = 0,9 ile 618,00 s bekletir; ikinci gişe beklemeyi 17,54 s'ye, yani 35,2 kat azaltır. Gişe eklemenin getirisi doğrusal değildir, çünkü ρ'yu yarıya indirmek 1/(1 − ρ) eğrisinin dik bölümünden düz bölümüne geçmek demektir. Saatte 36 öğrencide tek gişe kararsızdır (ρ = 1,2: ortalama 167 858 s, benzetim uzadıkça büyür); iki gişe 37,38 s. Aynı ρ'da çok gişeli tek kuyruk da avantajlıdır: ρ = 0,9'da k = 1, 2, 3, 4 için 618,00, 286,42, 175,63, 133,75 s (k = 1 / k = 4 = 4,62 kat). Bir gişe uzun bir işe takıldığında öbürleri sırayı eritir: **havuzlama** (pooling) etkisi.

6.4 Kuramla nitel karşılaştırma ve Little yasası

M/M/1 (Poisson geliş, **üstel** hizmet, tek gişe) modelinde kuyrukta ortalama bekleme $W_q = \rho \cdot E[S] / (1 - \rho)$'dır; genel hizmet dağılımında (**M/G/1**) Pollaczek–Khinchine (P–K) formülü $W_q = \lambda \cdot E[S^2] / (2(1 - \rho))$ verir (Kleinrock, 1975). Bizim hizmetimiz üstel değil, U[60, 180]'dir: $E[S] = 120$ s, $E[S^2] = 15\,620$ s², değişkenlik katsayısının karesi $C_s^2 \approx 0{,}085$ (üstelde 1). Bu yüzden karşılaştırma **yalnızca niteldir**: tek gişede benzetim / M/M/1 oranı 0,49–0,57, yani **biçim aynı** (ikisi de 1/(1 − ρ) ile patlar), **düzey farklı**. Gerçek $E[S^2]$ ile P–K benzetime 0,903–1,055 oranında yaklaşır; P–K / M/M/1 = $(1 + C_s^2)/2 \approx 0{,}542$, aradaki farkın kaynağı budur. En büyük sapma ρ = 0,10'dadır (P–K 7,23 s, benzetim 6,53 s, tohum aralığı 5,40–7,71 s). Sonuç: **hizmet süresinin değişkenliği beklemeyi doğrudan etkiler**; aynı ortalamayla daha düzenli bir gişe daha az bekletir.

Little yasası $L = \lambda W$, varış ve hizmet dağılımından bağımsız olarak kararlı her kuyrukta geçerlidir (Little, 1961). Benzetim L'yi her saniyenin kuyruk uzunluğunun ortalaması olarak, W'yi öğrencilerin ortalama beklemesi olarak ayrı ayrı ölçer; deneyin kararlı bütün satırlarında en büyük bağıl fark %0,03'tür. Kapasite 65 536 olduğu için hiçbir istek reddedilmedi (toplam ret 0).

6.5 Kuyruk gerçekleştirimlerinin maliyeti

Tablo 9.13

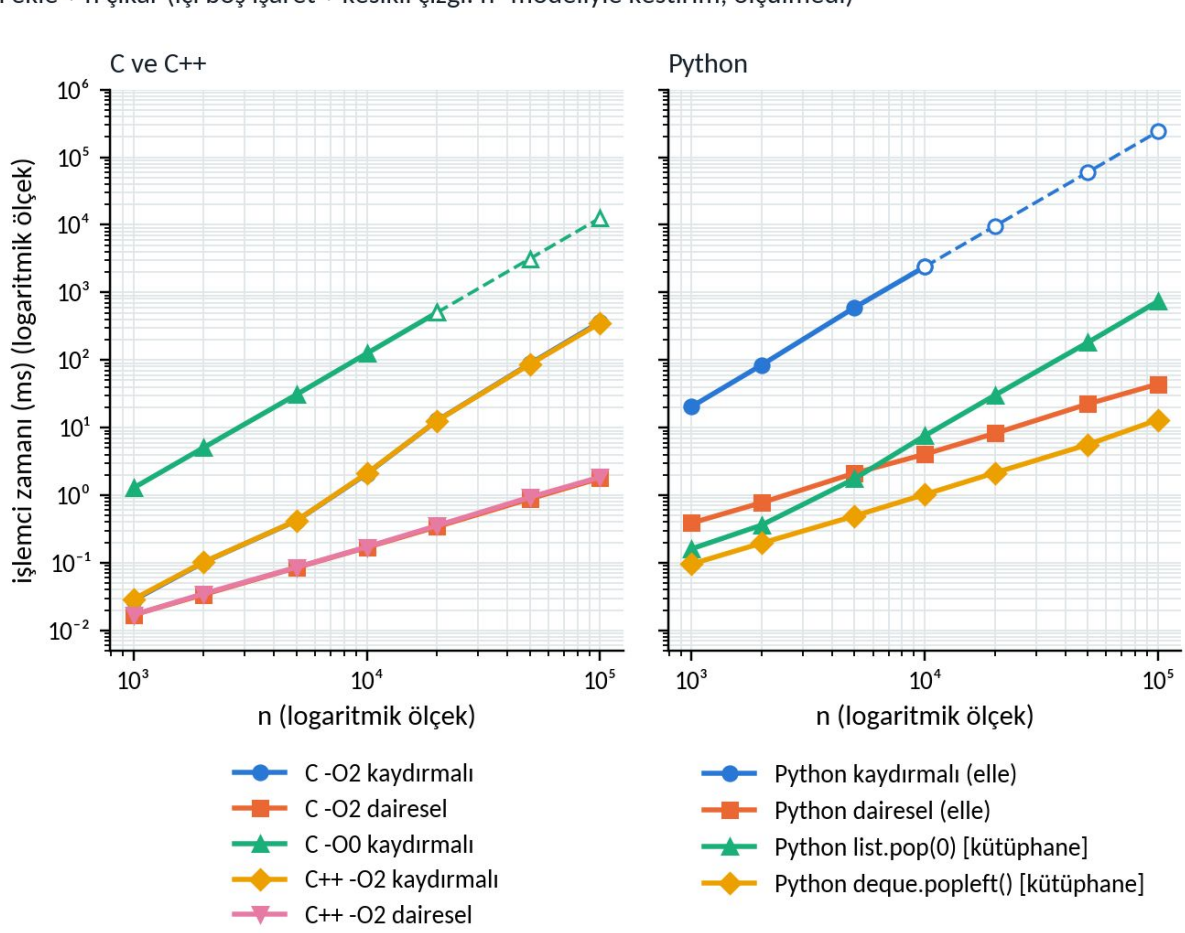
n ekle + n çıkar, işlemci zamanı ortancası (ms; "≈ … (k)": ölçülmedi, n^2 modeliyle kestirim) ve log–log eğim (beklenen: kaydırmalı ve pop(0) 2, ötekiler 1; h09_ozet_kuyruk.csv, h09_ozet_egim.csv)

seri	n = 10^3	n = 10^4	n = 10^5	eğim (n ≥ 10^4)
C -O2 kaydırmalı	0,028	2,07	363,6	2,226
C -O2 dairesel	0,017	0,169	1,78	1,025
C -O0 kaydırmalı	1,28	124,9	≈ 12 591 (k)	2,011
C++ -O2 kaydırmalı	0,029	2,11	351,4	2,205
C++ -O2 dairesel	0,017	0,172	1,85	1,035
Python kaydırmalı	20,38	2 394	≈ 239 422 (k)	2,079 (tümü)
Python dairesel	0,388	4,04	44,07	1,045
Python list.pop(0)	0,161	7,57	737,6	1,985

seri	$n = 10^3$	$n = 10^4$	$n = 10^5$	eğim ($n \geq 10^4$)
Python deque.popleft()	0,096	1,03	13,07	1,098

Kaydırmalı kuyrukta n çıkarma toplam $n(n-1)/2$ kaydırma yapar: $\Theta(n^2)$; dairesel tampon ve deque $\Theta(n)$. log–log eğim büyüme mertebesini verir (Sedgewick & Wayne, 2011) ve bunu doğrular (Tablo 9.13, Şekil 9.15). $n = 10^5$ için kaydırmalı / dairesel C -O2'de **204,03 kat**, C++ -O2'de 189,93 kat (ölçüldü); $n = 10^3$'te yalnızca 1,65 kat, çünkü GCC -O2 kaydırma döngüsünü tek bir `memmove` çağrısına çevirir (bu derleyicide `gcc -O2 -S` çıktısında görüldü) ve küçük bölge önbellekte kalır. Python'da `list.pop(0)` / `deque.popleft()` $n = 10^5$ için **56,44 kat** (ölçüldü), $n = 10^3$ için 1,68 kat; `pop(0)` kaydırmayı C ile yaptığından Python döngüsüyle kaydırmadan $n = 10^4$ için 316 kat hızlıdır, yine de $\Theta(n^2)$'dir. Elle yazılmış dairesel tampon `deque`'dan 3,37 kat yavaştır (ikisi de O(1); fark yorumlayıcı yükü) ve C -O2'deki aynı kodun 24,73 katı sürer.

Şekil 9.15
Kuyruk gerçekleştirimleri: n ekle + n çıkar süresi (log–log). Solda C ve C++, sağda Python; içi boş işaret ve kesikli çizgi: kestirim (ölçülmedi)



Kestirimlerin güvenilirliği. C -O2 kaydırmalı için $n = 2 \cdot 10^4$ ölçümünden n^2 modeliyle $n = 10^5$ kestirildi: 317,45 ms; ölçülen 363,59 ms (oran 1,145). Model %14,5 iyimserdir (bölge büyüdükçe önbellekten taşar); tablodaki kestirimleri alt sınır gibi okuyun. $\Theta(n^2)$ ile $\Theta(n)$ arasındaki fark büyük n'de dil ve derleyici farkını yutar.

6.6 Yorum, sınırlılıklar ve kendi makinenizde

- **Yük oranı ve gişe sayısı:** bekleme 1/(1 − ρ) ile büyür; ρ = 0,9 ile 0,98 arasındaki küçük bir talep artışı beklemeyi beşe katlar. Gişe eklemek doğrusal olmayan bir kazanç sağlar; aynı ρ'da tek kuyruklu çok gişe havuzlama sayesinde daha az bekletir. Kapasite planlamasında hedef ρ = 1 değil, beklemenin kabul edilebildiği ρ'dur.
- **Model ve veri yapısı:** M/M/1'in biçimi doğru, düzeyi yanlıştır; formüller ölçümün yerine değil, onu yorumlamak için kullanılır. Kaydırmalı dizi kuyruğu "çalışır" ama $\Theta(n^2)$'dir; Python'da kuyruk için `collections.deque` (Hata Avcısı 5).
- **Sınırlılıklar:** ayrık zaman ve saniyede en çok bir geliş (Poisson'un yaklaşığı); gelişler ve hizmet süreleri aynı LCG akışından çekilir (k ya da ρ değişince geliş dizisi de değişir); ısınma dönemi atılmadı; ρ ≥ 1 satırları kararlı durum değildir; k > 1 için M/M/k ile karşılaştırılmadı; süreler paylaşımlı makinede ölçüldü, büyük n'deki iki seri kestirimdir.

Kendi makinenizde. `cd kodlar/deney && python3 h09_olcum_calistir.py && python3 h09_cozumle.py` bütün ölçümleri yapar ve özet CSV'leri (`sonuclar/h09_ozet_*.csv`) üretir. Gişe sayıları her makinede aynı çıkmalıdır (ortak LCG, tamsayı aritmetiği); süreleriniz farklı olacak, ama eğimler (2 ve 1) aynı kalmalıdır. Web sayfasındaki gişe oynatıcısı tohum 42'de Örnek 10'un tablosunu birebir üretir.

7. Hata Avcısı

Her satırın hatalı ve düzeltilmiş programı `kodlar/hata_avcisi/` altındadır; belirtiler bu makinede gözlenen çıktılardır (`hata_avcisi/ciktilar/`). Hepsi uyarısız derlenir: hatalar mantıksaldır. Hata Avcısı 1 ve 6'nın C sürümleri sınır dışı bellek erişimidir, yani **tanımsız davranıştır**: aşağıdaki belirtiler bu derleyicide, bu çalıştırmada görülenlerdir; başka bir sistemde program sessizce yanlış sonuç da verebilir.

Tablo 9.14
Hata Avcısı: hata, belirti, neden ve çözüm

#	Hata	Belirti (bu makinede)	Neden	Çözüm
1	Boş yığından çıkarma (alt taşma denetimi yok)	C: `4 eleman itildi; 5 kez çıkarılıyor: 40 30 20 10 0`, `boyut = -1`, sonda `double free or corruption (out)` ve SIGABRT; ASan: heap-buffer-overflow. Python: `4 eleman itildi; 5 kez çıkarılıyor: 40 30 20 10 40` (negatif indis)	`boyut--` denetimsiz: boyut −1 olur, `veri[-1]` okunur; sonraki it öbek bloğunun başlığını bozar	Çıkarmadan önce boşluk denetimi (`yigin_bos_mu`); Python'da negatif indise değil `IndexError`'a güvenin
2	Dairesel kuyrukta dolu/boş ayrımı yalnız `bas == son` ile	`ekle 40 -> bas = 0, son = 0, boyut = 0, boş mu? evet`; `4 eleman eklendi; çıkarılanlar: (hiçbiri)`; 50 eklenince `çıkarılanlar: 50 (10, 20, 30, 40, 50 bekleniyordu)`	Dördüncü eklemede son sarıp bas'a eşit olur: dolu kuyruk boş görünür, sonraki ekleme eskilerin üzerine yazar	Sayaç (dolu: sayaç = kapasite) ya da bir boş hücre: Örnek 8 A/B
3	Shunting-yard'da ^ işlecini soldan birleşmeli işlemek	`2 ^ 3 ^ 2 -> sonek 2 3 ^ 2 ^ = 64`, `4 ^ 3 ^ 2 -> sonek 4 3 ^ 2 ^ = 4096`; düzeltilmiş: `2 ^ 3 ^ 2 -> sonek 2 3 2 ^ ^ = 512`	Eşit öncelikte de tepedeki ^ çıkarılıyor (`>=` her işleçte)	Eşitlikte yalnız gelen soldan birleşmeliyse çıkar (`once_cikar_mi`)
4	Sonek değerlendirmede işlenen sırasını ters almak	`10 3 - = -7 (beklenen 7) <-- YANLIŞ`; `20 4 / = 0 (beklenen 5) <-- YANLIŞ`; doğru görünen: `3 4 + = 7 (beklenen 7)`	İlk çıkan tepe ikinci işlenendir (b); + ve * değişmeli olduğu için hata gizlenir	Önce b, sonra a; testte − / ^ mutlaka olsun

#	Hata	Belirti (bu makinede)	Neden	Çözüm
5	Python'da `list.pop(0)` ile kuyruk	n = 10 000: 7,88 ms; n = 80 000: 515,96 ms (n iki katına çıkınca ≈ 4 kat)	`pop(0)` kalan n − 1 başvuruyu kaydırır: n çıkarma Θ(n²)	`deque.popleft()` (`collections`) ya da dairesel tampon
6	Dizi tabanlı yığında kapasite aşımı (taşma denetimi yok)	C: `10 eleman itildi (kapasite 4); boyut = 10, komşu blok: "Z"`, sonda `munmap_chunk(): invalid pointer` ve SIGABRT; ASan: `heap-buffer-overflow H09_HA6_kapasite_asimi.c:24 in yigin_it`	`veri[boyut] = x` boyut = kapasite iken de yapılıyor: ayrılan 16 baytın ötesine, komşu bloğa yazılır	`if (yigin_dolu_mu(y)) return false;` ya da kapasiteyi iki katına çıkar (Kavram 1)

Hata Avcısı 4'e yakından bakış. İki satırın yeri değişmiştir; düzeltilmiş sürümde önce b, sonra a çıkarılır (Örnek 4, Bölüm 4.4):

C `hata_avcisi/H09_HA4_islenen_sirasi.c · satır 43–45`

```c
        if (strchr("+-*/^", bt[0]) != NULL && bt[1] == '\0') {
            long long a = yigin[--ust];   /* HATA: tepe a değil, b'dir */
            long long b = yigin[--ust];
```

AddressSanitizer ile. Tanımsız davranışın belirtisi rastlantıya bağlıdır; ASan ise ilk geçersiz erişimde programı durdurur ve yerini söyler. Hata Avcısı 1'in raporu (sadeleştirilmiş: adresler `0x…`, süreç numarası `PID`, sistem kitaplığı çerçeveleri ve gölge bellek dökümü çıkarıldı) okunan yerin ayrılan 16 baytlık bölgenin **4 bayt öncesi**, yani `veri[-1]` olduğunu gösterir:

ÇIKTI

```
$ ./H09_HA1_bos_yigindan_cikar   # -fsanitize=address ile derlendi
==PID==ERROR: AddressSanitizer: heap-buffer-overflow on address 0x… at pc 0x… bp 0x…
sp 0x…
READ of size 4 at 0x… thread T0
    #0 0x… in yigin_cikar H09_HA1_bos_yigindan_cikar.c:29
    #1 0x… in main H09_HA1_bos_yigindan_cikar.c:43
0x… is located 4 bytes before 16-byte region [0x…,0x…)
allocated by thread T0 here:
    #0 0x… in malloc ../../../../src/libsanitizer/asan/asan_malloc_linux.cpp:69
    #1 0x… in main H09_HA1_bos_yigindan_cikar.c:34
SUMMARY: AddressSanitizer: heap-buffer-overflow H09_HA1_bos_yigindan_cikar.c:29 in
yigin_cikar
```

8. Dil Penceresi: C • C++ • Python

Tablo 9.15
Dil karşılıkları: kütüphane yapıları, kapsülleme, hata bildirimi ve tamsayı bölmesi

Öğe	C	C++	Python
Yığın	yok; elle (dizi ya da liste)	`std::stack<T>`: kap uyarlayıcısı, varsayılan kap `std::deque<T>`	`list`: `append` / `pop()`

Öğe	C	C++	Python
Kuyruk	yok; dairesel tampon ya da liste	`std::queue<T>`: uyarlayıcı, varsayılan `std::deque<T>`	`collections.deque`: `append` / `popleft()`; `queue.Queue` (iş parçacığı güvenli)
İki uçlu kuyruk	elle (Örnek 13)	`std::deque<T>`: iki uçta O(1), indisle erişim	`collections.deque` (`appendleft`, `maxlen`)
Kapsülleme	opak `struct` (başlıkta yalnız ad)	`class` + `private` üyeler	gelenek: `_alan`; `__alan` ad bozma
Hata bildirimi	dönüş kodu (`bool`), önkoşul	dönüş kodu ya da istisna (`std::out_of_range`)	istisna: `IndexError`, `queue.Empty`
Baştan çıkarma	kaydırma `O(n)`; dairesel `O(1)`	`queue::pop` `O(1)`; `vector` başından silme `O(n)`	`list.pop(0)` `O(n)`; `popleft()` `O(1)`
Tamsayı bölmesi	`/` sıfıra doğru keser: `-7 / 2 = -3`	C ile aynı	`//` aşağı yuvarlar: `-7 // 2 = -4`
Kalan	işareti bölünenin: `-7 % 2 = -1`	C ile aynı	işareti bölenin: `-7 % 2 = 1`

Kütüphane yapıları. C++'ta `std::stack` ve `std::queue` **kap uyarlayıcısıdır** (container adaptor): bir kabın (varsayılan `std::deque`) arayüzünü yığın ya da kuyruk işlemlerine daraltır; ikinci şablon parametresiyle başka bir kap da verilebilir: yığına `std::vector` ya da `std::list`, kuyruğa `std::list` (`std::vector`'de `pop_front` olmadığı için kuyruğun kabı olamaz) (ISO/IEC, 2017). Python'da `list` iyi bir yığın, kötü bir kuyruktur; `collections.deque` iki uçta O(1)'dir ve `maxlen` ile dolunca öbür uçtan atar; `queue.Queue` iş parçacıkları arasında güvenlidir ve boşken `get()` bekler (Python Software Foundation, t.y.).

Kavram 7'nin çıktısından: `std::stack<int, std::vector<int>> çıkış sırası: 30 20 10`; `std::queue<int, std::list<int>> çıkış sırası: 10 20 30`; `d.at(5) -> std::out_of_range istisnası (d[5] ise denetimsizdir)`; Python'da `boş listede pop() -> IndexError: pop from empty list`; `deque(maxlen=3) içine 1..5 eklenince: [3, 4, 5]`; `boş Queue'da get_nowait() -> queue.Empty (get() ise bekler)`.

Kapsülleme ve hata bildirimi (Kavram 6). C++'ta `private` alanlara sınıf dışından erişim derleme hatasıdır; aynı sınıf taşmayı dönüş koduyla, boş yığından çıkarmayı istisnayla bildirir: `it 1: true, it 2: true, it 3: false (false = taşma)`; `istisna yakalandı: std::out_of_range: boş yığından çıkarma`. Python'da erişim denetimi yoktur, yalnızca gelenek vardır: `_veri dışarıdan okunabilir (yalnızca gelenek): [1, 2]`; çift alt çizgili ad sınıf adıyla "bozulur" (name mangling), ama yine erişilebilir: `y.__boyut -> AttributeError: 'Yigin' object has no attribute '__boyut'`; `ama bozulmuş adla erişilebilir: y._Yigin__boyut = 2`. C'nin opak tipi (Bölüm 3.1) en katısıdır: alanlar kullanıcı kodunu derleyen derleyiciye bile görünmez. Nesneye Yönelik Programlama dersi bu ayrımı genelleştirir.

Tamsayı bölmesi ve kalan (Kavram 10). C standardına göre tamsayı bölümünün kesirli kısmı atılır, yani bölüm **sıfıra doğru** kesilir ve (a / b) · b + a % b = a özdeşliği korunur (ISO/IEC, 2018); C++ da aynı tanımı kullanır. Python'un `//` işleci **aşağı** (−∞'a doğru) yuvarlar ve `%` bölenin işaretini alır; özdeşlik iki dilde de korunur.

Kavram 10: C'de `-7 / 2` = -3, `-7 % 2` = -1, `7 / -2` = -3; Python'da `-7 // 2` = -4, `-7 % 2` = 1, `7 // -2` = -4. Hesap makinesinin üç dilde aynı sonucu vermesi için Python sürümü C'nin tanımını `kesik_bol` ile kurar (`-7 / 2` üç dilde −3). `int(a / b)` kullanılmaz: kayan nokta bölmesi büyük sayılarda kesinliği yitirir (`x = 10**17 + 1: kesik_bol(x, 1) = 100000000000000001, int(x / 1) = 100000000000000000`). Negatif sayıda `%` farkı dairesel tamponda geriye sarmayı da etkiler (Bölüm 3.5, 4.7).

9. Mühendis Gözüyle: Arayüz, Sözleşme ve Taşma Politikası

- **Arayüzü sözleşmeyle yazın, gerçekleştirimi saklayın.** Kavram 1'de aynı `main.c` iki gerçekleştirimle bayt bayt aynı çıktıyı verdi. Sözleşme yalnız imza değildir: boş yığından çıkarmada ne olacağı, sahiplik ve maliyet (O(1) mi, amortize O(1) mi?) de sözleşmedir; sözleşmede yazmayan bir davranışa güvenen kod, gerçekleştirim değişince bozulur.
- **Hata bildirimini bilinçli seçin.** Dönüş kodu ucuzdur ama göz ardı edilebilir; istisna göz ardı edilemez ama maliyetlidir; önkoşul en hızlısıdır ama C'de ihlali tanımsız davranıştır (Hata Avcısı 1 ve 6).
- **Sınırlı tamponda taşma politikası bir tasarım kararıdır. Reddet:** Örnek 8 ve Örnek 10 dolu kuyruğa eklemeyi reddeder; Mini-OBS 2 v2.8'in 64 kapasiteli kayıt kuyruğu 2 gişede 1 601 isteği reddeder (v2.8 belgesi § 10). **En eskinin üzerine yaz:** `deque(maxlen=3)` içine 1 … 5 eklenince [3, 4, 5] kalır; "son değerler önemlidir" durumları (ses, günlük kaydı) için. **Bekle:** üretici yer açılana kadar bekler (`queue.Queue.put`); veri kaybı yoktur, üretici yavaşlar. Seçim belgelenmeli ve sayılmalıdır (reddedilen istek, üzerine yazılan örnek).
- **Sınırları sınayın; derinliği sınırlayın.** Dairesel tamponun hataları sınırlardadır (kapasite 1, tam dolu, tam boş, sarma anı): Örnek 8 bu yüzden küçük kapasitelerle sınandı. Girdiye bağlı özyineleme derinliği üretimde çöker (Kavram 3 ve 4); açık yığınla yazın ya da derinliği sınırlayın (Örnek 12).

10. Mini-OBS 2 Adımı: v2.8 — Geri Al / Yinele ve Ders Kayıt Kuyruğu

Mini-OBS'ye iki yığınla **geri al / yinele** ekliyoruz: kayıt ekle, sil, güncelle ve geçen haftanın ders kaydı işlemleri (kaydol, bırak) geri alınabilir ve yinelenebilir. Ders kayıt günü için **FIFO istek kuyruğunu dairesel tamponla** kuruyor, kontenjanı dolu derse gelen isteği geçen haftanın bekleme listesine düşürüyor ve ortak LCG ile bir **kayıt günü simülasyonu** yapıyoruz. (Kaynak: proje klasöründeki `mini_obs_v2_8.md`, "Ders notu için özet", v2.8 belgesi § 10.) Örneklerle bağı: geri al / yinele Örnek 7, istek kuyruğu Örnek 8'in sayaçlı tamponu, kayıt günü Örnek 10'dur. Fark: v2.8'in kuyruğu küçüktür ve doluysa reddeder (varsayılanlarla 1 601 ret, kuyrukta ortalama bekleme 183,43 s; deneyde kapasite 65 536, ret yok) ve gelişler ile hizmet için ayrı tohumlu alt akışlar kullanır (Örnek 10 tek akış).

10.1 Gereksinimler ve ipuçları

- `kuyruk.h/.c`: sabit kapasiteli dairesel tampon — `kuyruk_olustur`, `kuyruk_sona_ekle` (doluysa false: reddet), `kuyruk_bastan_al`, `kuyruk_sondan_al`, `kuyruk_bos_mu`, `kuyruk_dolu_mu`, `kuyruk_yok_et`; dolu/boş ayrımı **sayaçla**; hiçbir işlem öğe kaydırmaz.
- `geri_al.h/.c`: `IslemKaydi` (tür, numara, `Ogrenci *once`, `Ogrenci *sonra`, konum, aktarılan) ve `Gecmis` (geri ve yinele yığınları, sınır); `islem_ekle`, `islem_sil`, `islem_guncelle`, `islem_kaydol`, `islem_birak` (uygula + kaydet; **yeni işlem yinele yığınını boşaltır**); `geri_al`, `yinele`; sınır aşılınca **en eski kayıt bırakılır** (varsayılan 100).
- Bekleme listesine `bekleme_basa_ekle` ve `bekleme_konuma_ekle` (geri almada kullanılır).

- `kayit` programına `ekle`, `sil`, `guncelle`, `goster`, `geri`, `yinele`, `gecmis` komutları ve `--sinir`.
- `kayit_gunu` programı: N istek, G gişe, kapasitesi C olan istek kuyruğu (doluysa ret), tamsayı zaman; çıktı: istek, ret, kabul, bekleme listesi, otomatik aktarma, ortalama ve en uzun bekleme, en uzun kuyruk; deterministik.
- Testler: geri al ∘ işlem = kimlik ve yinele ∘ geri al = işlem, **rastgele işlem dizileriyle**, her adımda bütün tablonun anlık görüntüsüyle karşılaştırarak; kuyruk sarma ve dolu/boş sınırları.
- Ölçüm: geri al/yinele süresi (işlem başına) ve bellek tüketimi (kayıt kopyası boyutu × işlem sayısı; sınırla); simülasyon sonuçları tablosu (en az gişe sayısı × geliş aralığı).

> MİNİ-OBS 2 PROJESİ · **İpuçları (v2.8 belgesinden)**
>
> - Kuyrukta yalnız `bas` ve `n` tutun; son konum `(bas + n) % kapasite`'dir. "Dolu" `n == kapasite`, "boş" `n == 0`.
> - Geri al yığınında en eski kaydı düşürmek için yığını **iki uçlu** kullanın: itme ve çekme sondan, düşürme baştan.
> - İşlem kaydını işlemden **önce** ayırın: bellek yoksa işlem hiç yapılmasın (yarım durum kalmasın).
> - Silmenin tersi için kaydın **kopyasını** saklayın (işaretçisini değil): sınıf dizisi `realloc` ile taşınabilir, silinen kaydın yeri başkası tarafından kullanılabilir.
> - Bırakmanın tersini kâğıtta çizin: bırakan eski konumuna, aktarılan öğrenci bekleme listesinin **başına** döner. Sıra önemlidir: önce bellek isteyen adım (diziye ekleme), sonra diğerleri.
> - Yeni işlemde yinele yığınındaki kayıtları **serbest bırakmayı** unutmayın (yalnızca sayacı sıfırlamak sızıntıdır; Valgrind "definitely lost" der).
> - Kâhin için her adımın sonunda bütün tablonun kopyasını bir diziye koyun: geri al bir önceki kopyaya, yinele bir sonrakine eşit olmalıdır.
> - Simülasyonda istekleri ve hizmet sürelerini ayrı tohumlu alt akışlardan üretin: gişe sayısını değiştirmek istek dizisini değiştirmesin.

10.2 Kabul ölçütleri ve test senaryoları

1. `make` ve `make test` sıfır uyarıyla derlenir; v1.0'ın 99 doğrulaması ve önceki sürümlerin testleri dahil bütün birim testleri ve `test_kahin` (300 deneme) geçer.
2. Kapasite 4'lük kuyrukta 1, 2, 3, 4 eklenince `kuyruk_dolu_mu` doğru ve 5 reddedilir; 1 baştan alınıp 5 eklenince içerik baştan sona 2 3 4 5 olur ve sarma sayısı 1'dir; kapasite 1'lik kuyruğa bir öğe girer.
3. `./kayit ornek_veri/ogrenciler.csv --komutlar ornek_veri/islem_komutlari.txt` şu satırı basar: `GERİ ALINDI: birak 2301010048 · kayıtlı listede 2. sıraya döndü; 2301010046 MEHMET ALİ KAYA bekleme listesinin başına döndü · geri 5, yinele 1`; özet satırı: `Özet: 23 komut · kabul 3 · bekleme 3 · aktarma 1 · bırakma 1 · ret 1 · hata 0 · geri al 4 · yinele 2 · son durum: kayıtlı 3/3, bekleyen 2, kayıt 9`; çıkış 0.
4. Aynı oturumda `kaydol 2301010051`'den sonraki `yinele` "YİNELENECEK İŞLEM YOK" der (yeni işlem yinele yığınını boşalttı).
5. `--sinir 2` ile aynı oturumda `gecmis` "geri 2 (sınır 2)" ve "düşürülen 6" gösterir.
6. Güncellemenin geri alınması kaydı bayt bayt eski hâline döndürür (`goster` aynı ortalamaları ve harf notlarını basar).

7. `./kayit_gunu` (varsayılanlar: 2 gişe, kuyruk 64, geliş aralığı 2 s) `Kuyruk dolu: ret  : 1601` ve `Kuyrukta bekleme  : ortalama 183.43 s` satırlarını basar; `--gise 4` ile ret 0; aynı parametrelerle iki çalıştırma bayt bayt aynıdır.
8. `./test_kahin --hata-goster dolu` ve `--hata-goster yinele` uyuşmazlık basar ve 1 ile çıkar; `make asan` ve `make valgrind` temizdir.
9. Ölçüm raporunda geri al ve yinele işlem başına süresi ölçülür (bu makinede 10^5 işlemde 0,81 µs ve 0,73 µs) ve geçmiş belleği sınırla O(sınır)'da kalır (sınır 100: 31 520 bayt).

Tablo 9.16
Mini-OBS 2 v2.8 test senaryoları (v2.8 belgesi § 10)

#	Girdi	Beklenen çıktı / davranış
1	kapasite 4: ekle 1..4, ekle 5	5 reddedilir; kuyruk değişmez
2	baştan al, ekle 5, sondan al	1; sarma 1; 5; içerik 2 3 4
3	boş kuyrukta baştan/sondan al	false
4	ekle (yeni numara) → geri → yinele	kayıt yok olur, sonra aynı konuma geri gelir
5	güncelle → geri	eski not ve ortalama (bayt bayt)
6	sil → geri	kayıt aynı sıraya döner
7	geri → yeni işlem → yinele	"YİNELENECEK İŞLEM YOK"
8	kontenjan 2: kaydol A, B, C; birak A; geri	kayıtlı A B; C bekleme listesinin başında
9	bekleyen B, C; birak B; geri	B bekleme listesinde yine 1. sırada
10	sınır 3: 5 güncelleme, 5 kez geri	3 geri al başarılı, 4.'sü "yok"; düşürülen 2
11	kaydol (ZATEN)	geçmişe kayıt eklenmez
12	`kayit_gunu --gise 1 --kuyruk 4`	ret > 0; en uzun kuyruk 4; sarma > 0

Verilir: kendi v2.7'niz (ya da referans v2.7), `kuyruk.h`, `geri_al.h` (bildirimler, tersler tablosu, sahiplik), `ornek_veri/islem_komutlari.txt`, `kayit_gunu`'nun model açıklaması ve istek üretimi (tohumlar), `olcer_yapi`'nın geçmiş ölçüm düzeneği. **Yazılır:** `kuyruk.c`, `geri_al.c`, `bekleme.c`'ye iki ekleme, `kayit.c`'ye yeni komutlar, `kayit_gunu.c`'nin simülasyon döngüsü; testler (birim + anlık görüntülü kâhin); ölçüm ve yorum. **Değiştirilmez:** v1.0 kuralları ve menü, kayıt dizisinin numara sırası, v2.7'nin kontenjan kuralları, ortak modüller. **Meydan okuma:** (a) `Kuyruk`'u opak tip yapın (başlıkta yalnız `typedef struct Kuyruk Kuyruk;`); (b) kontenjan değişikliğini de geri alınabilir yapın (aktarılanların listesi kayıtta); (c) simülasyonu olay güdümlü yazın (gişe bitiş anları H7'nin öbeğinde) ve zaman adımlı sürümle aynı sonucu verdiğini gösterin; (d) "geri al" yerine anlık görüntü tutan sürümü yazın, bellek ve süresini ölçüp komut kaydıyla karşılaştırın.

11. Alıştırmalar ve Atölyeler

Alıştırmalar yedi türdedir: A) izleme ve çıktı tahmini, B) karmaşıklık, C) değişmez yazma, D) hata bulma, E) Parsons bulmacası (Parsons & Haden, 2006), F) kod yazma, G) diller arası okuma. Kodu verilen her alıştırmanın dosyası `kodlar/alistirmalar/` altındadır; önce **çalıştırmadan** tahmin edin, sonra çalıştırıp karşılaştırın. Çözümler öğretim üyesindedir.

11.1 İzleme ve çıktı tahmini (A)

Alıştırma 9.1 ★☆☆ *Çıktı tahmini*

Kapasitesi 4 olan bir yığına ve sayaçlı bir dairesel kuyruğa aynı beş değer ekleniyor (beşincisi ikisine de sığmaz). Önce kâğıtta yığını ve kuyruğun dört hücresini çizin; program dört satırda ne yazdırır? `bas` hangi değerde biter ve neden?

alistirmalar/H09_A01_cikti_tahmini.c · satır 49–67

```
49      for (int i = 1; i <= 5; i++) {
50          it(10 * i);
51          ekle(10 * i);
52      }
53      int y = cikar();
54      int q = al();
55      printf("%d %d\n", y, q);
56      it(y + 1);
57      ekle(q + 1);
58      printf("boyut = %d, bas = %d, sayac = %d\n", boyut, bas, sayac);
59      printf("yığın:");
60      while (boyut > 0) {
61          printf(" %d", cikar());
62      }
63      printf("\nkuyruk:");
64      while (sayac > 0) {
65          printf(" %d", al());
66      }
67      printf("\n");
```

Alıştırma 9.2 ★★☆ *İzleme*

`(5 - 1) * 3 + 64 / 2 ^ 3 ^ 2 - 7` ifadesini Shunting-yard ile adım adım dönüştürün (her belirteçte yığın, çıktı, eylem; Örnek 5'in biçiminde), sonra sonek ifadeyi değerlendirin. İşleç yığını en çok kaç derinliğe çıkar? ^ soldan birleşmeli olsaydı sonuç ne olurdu? Örnek 5 ve 6 ile doğrulayın.

Alıştırma 9.3 ★☆☆ *İzleme*

Kapasitesi 5 olan sayaçlı dairesel tamponda (başta bas = son = sayaç = 0): ekle 1, 2, 3; çıkar; çıkar; ekle 4, 5, 6, 7, 8; çıkar; çıkar. Her adımda bas, son, sayaç ve hücreleri yazın. Hangi adımlarda sarma olur, 8 eklenebilir mi? Yöntem B (6 hücre) ile sonuç hangi adımda farklılaşır?

11.2 Karmaşıklık (B)

Alıştırma 9.4 ★★☆ *Karmaşıklık*

Örnek 11'de tek bir adımda iç döngü n − 1 kez dönebilir. (a) Hangi girdi bunu yaptırır, toplam karşılaştırma kaçtır? (b) it + çıkar ≤ 2n'i her indisin yaşamı üzerinden kanıtlayın. (c) Azalan girdide karşılaştırma neden yalnızca n − 1, rastgele girdide neden ≈ 2n'dir?

Alıştırma 9.5 ★★☆ *Karmaşıklık*

İki yığınla kuyruk: ekle(x) giriş yığınına iter; çıkar() çıkış yığını boşsa giriş yığınındakileri tek tek çıkış yığınına aktarır, sonra çıkış yığınından çıkarır. (a) Tek bir çıkarmanın en kötü maliyeti nedir? (b) n işlemde toplam it + çıkar ≤ 4n, yani amortize O(1) olduğunu gösterin. (c) Alıştırma 9.12'deki programla ölçün.

11.3 Değişmez yazma (C)

Alıştırma 9.6 ★★☆ *Değişmez yaz*

Tek tür ayraç için sayaçla denetimin (açılışta +1, kapanışta −1) değişmezini yazın ve yöntemin doğruluğunu kanıtlayın. Sonra `[()]` için iki ayrı sayacın (biri ( ) için, biri [ ] için) neden yanlışlıkla "dengeli" dediğini, Örnek 3'ün yığın değişmeziyle (Tablo 9.5) karşılaştırarak açıklayın.

Alıştırma 9.7 ★★☆ *Değişmez yaz*

Kayan pencere en büyüğünde (Örnek 13) iki uçlu kuyruğun değişmezini (indisler artan, değerler kesin azalan, hepsi geçerli pencerede) döngünün her adımı için koruduğunu gösterin ve ön elemanın pencerenin en büyüğü olduğunu bu değişmezden çıkarın. Arkadan atma koşulunda ≤ yerine < yazılsaydı hangi değişmez değişir, sonuç değişir mi, kuyruk boyu k'yı aşabilir mi?

11.4 Hata bulma (D)

Alıştırma 9.8 ★★☆ *Hata bul*

`parantez_denetle` altı satırdan bazıları için yanlış sonuç veriyor. Önce her satır için doğru yanıtı (Örnek 3) ve bu programın vereceği yanıtı tahmin edin; sonra hatalı satırı bulun. Hangi hata türünü hiç yakalayamaz?

alistirmalar/H09_A02_hata_bul_parantez.c · satır 24–37

```
24      for (int i = 0; s[i] != '\0'; i++) {
25          char c = s[i];
26          if (c == '(' || c == '[' || c == '{') {
27              if (boyut == MAKS) {
28                  return false;
29              }
30              yigin[boyut++] = c;
31          } else if (c == ')' || c == ']' || c == '}') {
32              if (boyut == 0 || yigin[--boyut] != acilis_karsiligi(c)) {
33                  return false;
34              }
35          }
36      }
37      return true;
```

Alıştırma 9.9 ★★☆ *Hata bul*

Sayaçlı dairesel kuyruk 10, 20, 30, 40'ı aynı sırayla vermeli; vermiyor. Program ne yazdırır? Hatalı satırları bulun ve düzeltin. Hata neden ilk eklemede değil de çıkarmada görünür?

alistirmalar/H09_A03_hata_bul_dairesel.c · satır 21–29

```c
static bool kuyruk_ekle(Kuyruk *q, int x)
{
    if (q->sayac == KAPASITE) {
        return false;
    }
    q->sayac++;
    q->veri[(q->bas + q->sayac) % KAPASITE] = x;
    return true;
}
```

11.5 Parsons (E)

Alıştırma 9.10 ★★☆ *Parsons*

Aşağıdaki satırlar tek basamaklı işlenenli bir sonek ifadeyi (ör. `512+4*+3-`) değerlendiren fonksiyonun karıştırılmış satırlarıdır; iki satır fazladır (dosya bu hâliyle derlenmez). Doğru sırayı ve girintiyi yazın; fazla iki satır neden yanlıştır? İşlenenlerin hangi sırayla çıkarıldığına dikkat edin.

`alistirmalar/H09_A04_parsons_sonek.c · satır 7–22`

```c
            int a = yigin[--n];
    return yigin[n];
        if (c >= '0' && c <= '9') {
    int yigin[64], n = 0;
            yigin[n++] = c - '0';
static int sonek(const char *s)
    return yigin[n - 1];
            yigin[++n] = c - '0';
        } else if (c == '+' || c == '-' || c == '*') {
    for (int i = 0; s[i] != '\0'; i++) {
            int b = yigin[--n];
        char c = s[i];
            yigin[n++] = c == '+' ? a + b : c == '-' ? a - b : a * b;
        }
    }
}
```

11.6 Kod yazma (F)

Alıştırma 9.11 ★☆☆ *Kod yaz*

Bir satırın harflerini (yalnızca ASCII harfler; büyük-küçük harf farkı ve harf olmayan karakterler yok sayılır) hem bir yığına hem bir kuyruğa ekleyip ikisinden aynı anda çıkararak satırın palindrom olup olmadığını bulan bir program yazın (“Never odd or even” evet, “stack and queue” hayır). Yığını ve kuyruğu Örnek 1 ve 8'deki gibi elle yazın; sonucu, satırın ters çevrilmiş kopyasıyla karşılaştıran bir kâhinle rastgele dizgilerde sınayın. C++ ve Python sürümlerini de yazın.

Alıştırma 9.12 ★★☆ *Kod yaz*

Alıştırma 9.5'teki iki yığınla kuyruğu yazın (iki dizi tabanlı yığın; çıkış yığını boşken aktarma). Tohum 42 ile 10^6 rastgele işlemde (%55 ekle) her çıkarmayı Örnek 8'in dairesel tamponuna karşı sınayın; toplam aktarılan eleman sayısını ve tek bir çıkarmadaki en büyük aktarmayı yazdırın. İşlem başına it + çıkar sayısı 4'ü aşıyor mu?

Alıştırma 9.13 ★★★ *Kod yaz*

Bir dizide **en büyük − en küçük ≤ d** koşulunu sağlayan en uzun kesintisiz alt dizinin uzunluğunu Θ(n)'de bulun: iki uç (sol, sağ) ilerlerken biri en büyük, biri en küçük için iki monotonik iki uçlu kuyruk tutun (Örnek 13). O(n^2) kaba kuvvet kâhiniyle n = 0 … 60'lık 2000 rastgele dizide sınayın, ardından n = 10^5 rastgele dizide (değerler 0 … 10^6, d = 1000) sonucu ve deque işlemlerinin toplamını yazdırın.

11.7 Diller arası okuma (G)

Alıştırma 9.14 ★★☆ *Diller arası okuma*

Python programı ne yazdırır? `deque(maxlen=3)` hangi taşma politikasını uygular? `y.pop()` ile `y.pop(0)`'ın maliyeti nedir? C'de `7 / -2` ve `-7 % 2` ne verirdi?

alistirmalar/H09_A05_deque_okuma.py · satır 9–21

```python
def main():
    d = deque(maxlen=3)
    for x in [1, 2, 3, 4, 5]:
        d.append(x)
    print(list(d))
    d.appendleft(0)
    print(list(d), d.pop())
    y = [10, 20, 30, 40]
    print(y.pop(), y.pop(0), y)
    q = deque([10, 20, 30, 40])
    q.rotate(1)
    print(q.popleft(), list(q))
    print(7 // -2, -7 % 2, 7 - 2 * (7 // 2))
```

Alıştırma 9.15 ★★☆ *Diller arası okuma*

C++ programının çıktısı nedir? `std::stack<int, std::vector<int>>` ile `std::queue<int>`'in alttaki kapları nedir? Programı Python'da `list` ve `collections.deque` ile yazın; son satır ne verir?

alistirmalar/H09_A06_stack_queue_okuma.cpp · satır 15–33

```cpp
    std::stack<int, std::vector<int>> s;
    std::queue<int> q;
    for (int i = 1; i <= 4; i++) {
        s.push(i * i);
        q.push(i * i);
    }
    s.pop();
    q.pop();
    std::cout << s.top() << " " << q.front() << " " << q.back() << "\n";
    while (!s.empty()) {
        q.push(s.top());
        s.pop();
    }
    std::cout << q.size() << ":";
    while (!q.empty()) {
        std::cout << " " << q.front();
        q.pop();
    }
    std::cout << "\n" << 7 / -2 << " " << -7 % 2 << "\n";
```

11.8 Atölye 1 · Ray manevra oyunu: Shunting-yard kartlarla (sınıf, ≈ 15 dk)

ALGORİTMA ATÖLYESİ · **Sınıf bir demiryolu kavşağı olur**

- Yere bir T kavşağı çizilir: sağda **girdi rayı**, solda **çıktı rayı**, ortada kör bir yan hat, yani **işleç yığını** (Şekil 9.9). Her öğrenci bir belirteç kartı taşır; işleç kartlarında öncelik ve birleşme yönü yazılıdır (+ − 1 soldan, * / 2 soldan, ^ 4 sağdan). Yan hatta yalnızca en öndeki (tepe) hareket edebilir.
- Girdi `a + b * c ^ d ^ e - f`. Her "tren" için sınıf karar verir: **çıktıya yaz**, **yan hatta it** ya da **önce yan hattan çıkar**; sonda çıktı rayı Örnek 5'inkiyle karşılaştırılır (`a b c d e ^ ^ * + f -`).
- **İkinci tur:** kural "eşitse çıkar" yapılınca ^ zinciri ne olur (Hata Avcısı 3)? **Üçüncü tur:** `-(a + b) * c` ve hatalı `3 + 4) * 2`. Her tur web sayfasındaki adım adım Shunting-yard oynatıcısıyla doğrulanır.

11.9 Atölye 2 · Mini derleyici adımı: dört işlem, parantez ve üs destekli hesap makinesi (eşli, ≈ 30 dk)

ALGORİTMA ATÖLYESİ · **Bir derleyicinin ilk üç aşaması: belirteçleme, ayrıştırma, değerlendirme**

Eşler bir satırlık araek ifadeyi hesaplayan programı yazar: (1) **belirteçleme** (çok basamaklı sayılar, + − * / ^, parantez, tekli eksi), (2) Shunting-yard, (3) sonek değerlendirme. Başlangıç Örnek 4 ve 5'tir; Örnek 6'ya bakmadan yazın, 15 dakikada roller (klavye / sınama) değişir. Sonra program Örnek 6'nın girdi dosyasıyla (`kodlar/girdiler/H09_06_hesap_makinesi.txt`) çalıştırılıp çıktılar karşılaştırılır. İnceleme listesi:

- `-` ikili mi tekli mi (başta, `(`'den ya da işleçten sonra)? `-2 ^ 2` = −4 ve `(-2) ^ 2` = 4 mü; `2 ^ 3 ^ 2` = 512 mi?
- Bölme sıfıra doğru mu kesiyor (`-7 / 2` = −3); sıfıra bölme ve `46341 * 46341` taşması hata mı?
- Her hata sütunuyla mı bildiriliyor (eşleşmeyen `)`, kapanmamış `(`, işlenen/işleç bekleniyordu, boş ifade)?
- Rastgele ifade ağaçlarıyla kâhinli sınama (Örnek 6) için ne gerekir?

11.10 Atölye 3 · Kuyruk simülasyonu: gişe sayısının ortalama beklemeye etkisi (gruplar, ≈ 20 dk)

ALGORİTMA ATÖLYESİ · **Kayıt günü planlaması: kaç gişe açmalıyız?**

Kayıt günü 8 saattir, öğrenciler saatte ortalama λ kişi gelir, bir kayıt 60–180 s sürer. Her grup bir λ alır (saatte 12, 24, 27, 36 ya da 48) ve **önce tahmin eder**: k = 1 … 4 için ρ = λ · 120 / (3600 k) nedir, hangi k'da kuyruk sınırsız büyür, bekleme nasıl değişir? Sonra web sayfasındaki gişe oynatıcısıyla (tohum 42) benzetim yapılır; saatte 36 öğrenci ve k = 2 için oynatıcı Örnek 10'un satırını vermelidir (ortalama bekleme 29,65 s).

- Tahmin ve benzetim tahtada tek tabloda: λ, k, ρ, ortalama bekleme, en uzun kuyruk, doluluk. En çok şaşırtan hangisi?
- Little yasası: ortalama kuyruk ≈ λ × ortalama bekleme mi; ρ ≥ 1'de neden tutmaz? "Ortalama bekleme 5 dakikayı geçmesin" ölçütüyle her λ için en az kaç gişe gerekir?
- Hizmet süresi 110–130 s yapılınca (ortalama aynı) bekleme nasıl değişir? Bölüm 6.4'teki P–K yorumuyla karşılaştırın.

12. Kendini Değerlendirme

Aşağıdakileri kitaba bakmadan yapabiliyorsanız işaretleyin; yapamadıklarınız için parantez içindeki bölüme dönün. Web sayfasındaki on soruluk quiz ve oynatıcılar bu listeyi sınamanın en hızlı yoludur.

- ☐ Bir soyut veri tipini işlemleri ve sözleşmeleriyle tanımlayabiliyor; C'de opak tiple arayüzü gerçekleştirimden ayırabiliyorum. (3.1, Kavram 1)
- ☐ Yığını dizi ve bağlı listeyle gerçekleyip taşma ve alt taşmayı sözleşmeye uygun bildirebiliyorum. (4.1, Hata Avcısı 1 ve 6)
- ☐ Dairesel tamponda bas, son ve sayacı adım adım izleyebiliyor; dolu ve boş durumunu iki yöntemle ayırabiliyorum. (4.2, Hata Avcısı 2)
- ☐ Parantez denetiminin üç hata türünü açıklayabiliyor; bir sonek ifadeyi işlenen sırasını doğru alarak değerlendirebiliyorum. (4.3, 4.4, Hata Avcısı 4)
- ☐ Shunting-yard'ı öncelik ve birleşme yönüyle adım adım uygulayabiliyor; ^ işlecinin neden sağdan birleştiğini gösterebiliyorum. (4.5, Hata Avcısı 3)
- ☐ Monotonik yığın ve monotonik iki uçlu kuyrukla Θ(n) algoritma yazabiliyor ve amortize argümanını kurabiliyorum. (4.6, 4.7)
- ☐ Özyinelemeli bir algoritmayı açık yığınla yazabiliyor, yığın derinliğini $\lfloor \log_2 n \rfloor + 1$ ile sınırlayabiliyorum. (3.5, 4.8)
- ☐ Kuyrukla katman katman işleme ve gişe benzetimi yazıp ρ ve Little yasasıyla yorumlayabiliyor; üç dilin karşılıklarını ve `list.pop(0)` tuzağını biliyorum. (5.3, 5.4, 6, 8)

13. İleri Bakış ve Merak Kutusu

İLERİ BAKIŞ · Bu haftanın konusu nerede derinleşiyor?

- Yığın ve kuyruk: H11'de DFS (yığın) ve BFS (kuyruk) dolaşmalarının motorudur. Kavram 4'teki iki döngü (yığınla bölge, kuyrukla uzaklık) H11'de bir grafta çalışır.
- İfade ayrıştırma: Biçimsel Diller ve Özdevinirler Kuramı ile derleyici tasarımında (yığınlı özdevinirler, ayrıştırıcılar) genelleşir. Bağlamdan bağımsız dilbilgilerini tanıyan makine **yığınlı özdevinir**dir (pushdown automaton): sonlu durumlu denetim + yığın.
- Kuyruk kuramı ve simülasyon: olasılık–istatistik ve işletim sistemi zamanlayıcıları konularında sürer. M/M/1 ve P–K formülleri (Bölüm 6.4) oradan gelir; zamanlayıcı hazır süreçleri kuyruklarda tutar.

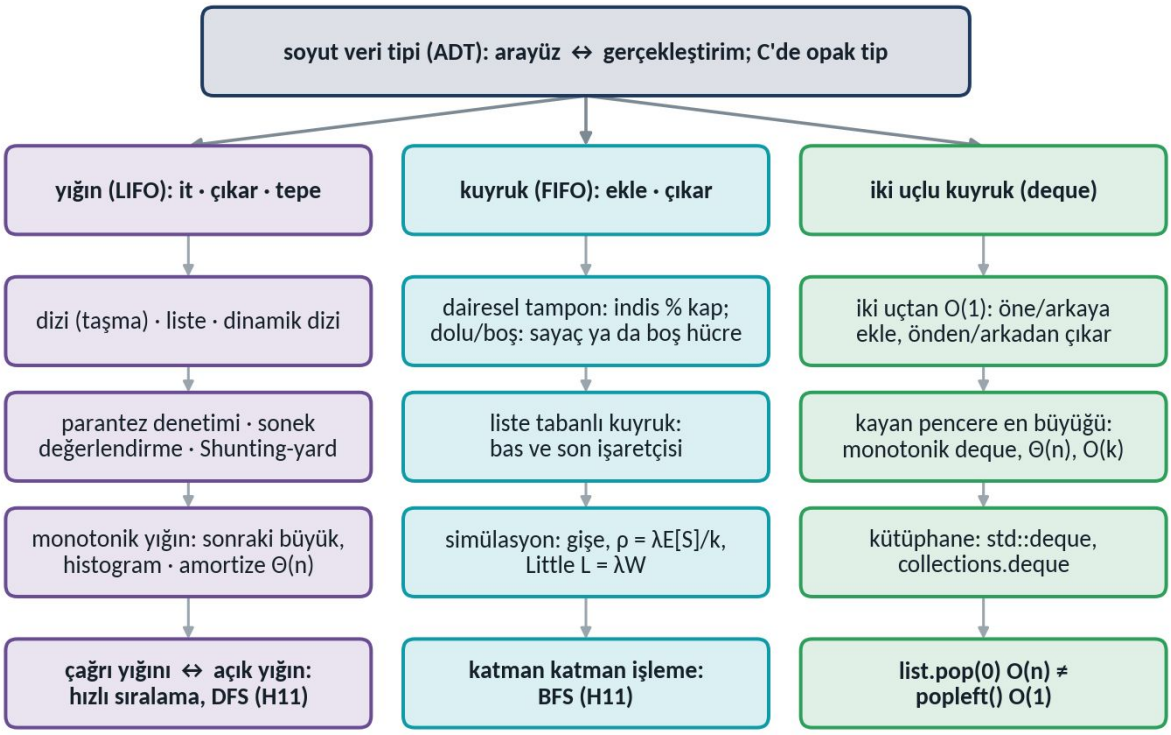
MERAK KUTUSU · Histogramdaki en büyük dikdörtgen (Örnek 14)

Genişliği 1 olan çubuklardan oluşan bir histogramda, çubukların içine sığan en büyük dikdörtgenin alanı nedir? Kaba kuvvet her (i, j) çifti için en kısa çubuğu bulur: $O(n^2)$. **Monotonik yığın** bunu Θ(n)'e indirir: yığında yükseklikleri tabandan tepeye **artan** çubuklar bekler. Yeni çubuk tepedekinden kısaysa tepedeki çubuk artık sağa uzayamaz: çıkarılır ve "kendi yüksekliğinde, sağda bu çubuğa, solda yığında altındakine kadar" dikdörtgenin alanı hesaplanır; eşitse de çıkarılır, dikdörtgenini yeni çubuk devralır. Sonda yüksekliği 0 olan sanal bir çubuk yığını boşaltır. h = [2 4 3 5 1 4 4 2] için `en büyük alan = 9 (kâhin 9); it = 8, çıkar = 8, karşılaştırma = 14` (3 çubuk genişliğinde, yüksekliği 3).

Kâhinli sınama: `kâhinle uyuşmazlık = 0, it + çıkar > 2n = 0`. n = 10^5 rastgele yükseklikte (0 … 10^4) en büyük alan 124 374, karşılaştırma/n = 2,000; artan girdide en büyük alan 2 500 050 000'dir ve 32 bitlik `int`'i aşar: C ve C++ sürümleri alanı `long long` ile hesaplar (Python tamsayısı sınırsızdır). Soru: 0/1 matristeki en büyük dikdörtgen (her satır bir histogram)?

14. Özet, Kavram Haritası ve Terim Sözlüğü

Şekil 9.16
Hafta 9 kavram haritası



- Soyut veri tipi işlemleri ve sözleşmeleri tanımlar, gerçekleştirimi saklar (C'de opak tip). Yığın (LIFO) ve kuyruk (FIFO) işlemleri O(1)'dir; taşma ve alt taşma sözleşmeyle bildirilir.
- Dairesel tamponda bas == son hem boşta hem doluda doğrudur: sayaç ya da bir boş hücre gerekir. Kaydırmalı dizi kuyruğu n = 10^5'te C'de 204 kat, Python `list.pop(0)` `deque`'dan 56 kat yavaştı.
- Parantez denetimi, sonek değerlendirme ve Shunting-yard tek geçişli Θ(n) yığın algoritmalarıdır; öncelik ve birleşme yönü sonekte işleç sırasına dönüşür (^ sağdan, − soldan).
- Monotonik yığın ve iki uçlu kuyrukta her eleman bir kez girer, en çok bir kez çıkar: amortize Θ(n). Özyineleme örtük bir yığındır.
- Kuyruk benzetiminde bekleme 1/(1 − ρ) ile büyür, gişe eklemenin getirisi doğrusal değildir ve L = λW her kararlı kuyrukta geçerlidir.

Tablo 9.17
Terim sözlüğü (Yol Haritası Bölüm 6.8 ile uyumlu)

Türkçe	İngilizce	Not
soyut veri tipi; opak tip; eksik tip	abstract data type (ADT); opaque type; incomplete type	sözleşme: önkoşul, sonkoşul, hata
yığın; it, çıkar, tepe; taşma, alt taşma	stack; push, pop, peek; overflow, underflow	"yığın" her zaman stack; heap = öbek
kuyruk; ekle, çıkar; baş, son	queue; enqueue, dequeue; front (head), rear (tail)	FIFO
dairesel tampon; sarma	circular (ring) buffer; wrap-around	mod ya da bit maskesi
iki uçlu kuyruk; kayan pencere	double-ended queue (deque); sliding window	`std::deque`, `collections.deque`

Türkçe	İngilizce	Not
araek; önek; sonek	infix; prefix; postfix	(ters) Polonya gösterimi; RPN
işleç önceliği; birleşme yönü; manevra sahası	operator precedence; associativity; shunting-yard	soldan / sağdan; Dijkstra (1961)
monotonik yığın; yük oranı; Little yasası	monotonic stack; utilization; Little's law	ρ = λE[S]/k; L = λW
amortize maliyet; kâhin; çağrı yığını	amortized cost; oracle; call stack	kap uyarlayıcısı: container adaptor

15. Kaynaklar

Backus, J. W., Bauer, F. L., Green, J., Katz, C., McCarthy, J., Perlis, A. J., Rutishauser, H., Samelson, K., Vauquois, B., Wegstein, J. H., van Wijngaarden, A., & Woodger, M. (1963). Revised report on the algorithmic language ALGOL 60. *Communications of the ACM*, *6*(1), 1–17. https://doi.org/10.1145/366193.366201

Bauer, F. L., & Samelson, K. (1962). *Automatic computing machines and method of operation* (U.S. Patent No. 3,047,228). U.S. Patent and Trademark Office.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

Dijkstra, E. W. (1961). *Algol 60 translation: An Algol 60 translator for the X1 and making a translator for Algol 60* (Report MR 35/61). Mathematisch Centrum.

ISO/IEC. (2017). *Programming languages — C++* (ISO/IEC Standard No. 14882:2017). International Organization for Standardization.

ISO/IEC. (2018). *Information technology — Programming languages — C* (ISO/IEC Standard No. 9899:2018). International Organization for Standardization.

Kleinrock, L. (1975). *Queueing systems: Vol. 1. Theory*. Wiley.

Knuth, D. E. (1997). *The art of computer programming: Vol. 1. Fundamental algorithms* (3rd ed.). Addison-Wesley.

Little, J. D. C. (1961). A proof for the queuing formula: L = λW. *Operations Research*, *9*(3), 383–387. https://doi.org/10.1287/opre.9.3.383

Łukasiewicz, J. (1957). *Aristotle's syllogistic from the standpoint of modern formal logic* (2nd ed.). Clarendon Press.

Parsons, D., & Haden, P. (2006). Parson’s programming puzzles: A fun and effective learning tool for first programming courses. In *Proceedings of the 8th Australasian Conference on Computing Education* (Vol. 52, pp. 157–163). Australian Computer Society.

Python Software Foundation. (t.y.). *The Python standard library*. Erişim tarihi 13 Ekim 2026, https://docs.python.org/3/library/

Samelson, K., & Bauer, F. L. (1960). Sequential formula translation. *Communications of the ACM*, *3*(2), 76–83. https://doi.org/10.1145/366959.366968

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.

Whitney, T. M., Rodé, F., & Tung, C. C. (1972). The “powerful pocketful”: An electronic calculator challenges the slide rule. *Hewlett-Packard Journal*, *23*(10), 2–9.

Ek A. Dil Karşılıkları

Bu ekte örnek merdiveninin on dört basamağının C++ ve Python sürümlerinden çekirdek fonksiyonlar, dosyadaki satır numaralarıyla verilir; tam dosyalar `kodlar/cpp/` ve `kodlar/python/` altındadır. Hardal satırlar C sürümünden **anlam olarak** farklılaşan satırlardır (`kodlar/ornekler.json` → `farklar`); fonksiyon dışındaki farklar not kutusunda açıklanır. Fonksiyon ve değişken adları üç dilde aynıdır. Bütün örneklerin çıktısı üç dilde aynıdır (=); Örnek 6 hatalı ifade içeren girdide üç dilde de 1 çıkış koduyla biter.

Tablo 9.18
Derleme ve çalıştırma komutları (X: örneğin kök adı, ör. H09_05_shunting_yard)

Dil	Derleme / çalıştırma	Not
C	`gcc -std=c17 -Wall -Wextra -pedantic -O2 X.c ortak_rastgele.c -o X · ./X`	Örnek 4 ve 5'te ortak_rastgele.c gerekmez
C++	`g++ -std=c++17 -Wall -Wextra -pedantic -O2 X.cpp -o X · ./X`	ortak başlıklar (.hpp) yalnızca #include
Python	`python3 X.py (Windows: py X.py)`	Python 3.10 ve üstü; yalnızca standart kütüphane
Girdili	`./X < ../girdiler/X.txt`	Örnek 3, 4, 5, 6 ve 7

A.1 Örnek 1 · Dizi tabanlı yığın

C++ **cpp/H09_01_dizi_yigin.cpp · satır 54–73**

```cpp
bool yigin_cikar(Yigin& y, int& x)
{
    if (yigin_bos_mu(y)) {
        return false;
    }
    y.boyut--;
    x = y.veri[y.boyut];
    cikar_sayisi++;
    return true;
}

// Tepedeki elemanı çıkarmadan okur
bool yigin_tepe(const Yigin& y, int& x)
{
    if (yigin_bos_mu(y)) {
        return false;
    }
    x = y.veri[y.boyut - 1];
    return true;
}
```

Python **python/H09_01_dizi_yigin.py · satır 51–65**

```python
def yigin_cikar(y):
    """Alt taşma: yığın boşsa (False, 0); değilse (True, tepe)"""
    global cikar_sayisi
    if yigin_bos_mu(y):
```

```
55          return False, 0
56      y.boyut -= 1
57      cikar_sayisi += 1
58      return True, y.veri[y.boyut]
59
60
61  def yigin_tepe(y):
62      """Tepedeki elemanı çıkarmadan okur"""
63      if yigin_bos_mu(y):
64          return False, 0
65      return True, y.veri[y.boyut - 1]
```

DİL PENCERESİ · Farklılaşan satırlar

C'de veri `malloc` ile ayrılır ve `yigin_yok_et` içinde `free` edilir; C++'ta `std::vector` belleği yıkıcısında kendisi bırakır (RAII), Python'da `[0] * kapasite` ile sabit uzunluklu liste ayrılır (yerleşik `append` büyümesine güvenilmez). Çıkarılan değer C'de çıktı parametresiyle (`int *x`), C++'ta başvuruyla (`int& x`), Python'da `(True, değer)` demetiyle döner; Python'da fonksiyon içinde değişen sayaçlar `global` bildirimi ister. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 17, 28, 31; Python 24, 40.)

A.2 Örnek 2 · Liste tabanlı yığın

C++ **cpp/H09_02_liste_yigin.cpp · satır 62–73**

```
62  bool yigin_it(Yigin& y, int x)
63  {
64      Dugum *d = new Dugum{x, y.tepe};  // yeni düğüm eski tepenin önüne
65      ayrilan_dugum++;
66      y.tepe = d;
67      y.boyut++;
68      it_sayisi++;
69      if (y.boyut > en_buyuk_derinlik) {
70          en_buyuk_derinlik = y.boyut;
71      }
72      return true;
73  }
```

Python **python/H09_02_liste_yigin.py · satır 55–65**

```
55  def yigin_it(y, x):
56      """Kapasite sınırı yok: bellek yetmezse MemoryError oluşur"""
57      global it_sayisi, en_buyuk_derinlik, ayrilan_dugum
58      d = Dugum(x, y.tepe)  # yeni düğüm eski tepenin önüne
59      ayrilan_dugum += 1
60      y.tepe = d
61      y.boyut += 1
62      it_sayisi += 1
63      if y.boyut > en_buyuk_derinlik:
64          en_buyuk_derinlik = y.boyut
65      return True
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> C'de her düğüm `malloc`/`free` ile, C++'ta `new`/`delete` ile ayrılır; C++ yığını bir RAII yapısıdır: yıkıcı kalan düğümleri döngüyle siler, kopyalama `= delete` ile yasaklanır (yoksa iki nesne aynı düğümleri silerdi). Python'da düğüm `__slots__`lu bir sınıftır; serbest bırakma yoktur — `yigin_yok_et` başvuruları bırakır, belleği başvuru sayımı geri alır. Kâhin C'de ayrı bir dizi, C++'ta `std::stack`, Python'da `list`'tir. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 34, 36, 123; Python 12, 44, 104, 116.)

A.3 Örnek 3 · Parantez denetimi

C++ **cpp/H09_03_parantez_denetimi.cpp · satır 99–135**

```cpp
Sonuc parantez_denetle(const std::string& s, bool iz)
{
    int n = static_cast<int>(s.size());
    Sonuc r{DENGELI, 0, 0, 0, 0, 0};
    Yigin y;
    yigin_olustur(y, n > 0 ? n : 1);
    for (int i = 0; i < n && r.durum == DENGELI; i++) {
        char c = s[i];
        islenen++;
        if (c == '(' || c == '[' || c == '{') {
            yigin_it(y, Acilis{c, i + 1});
        } else if (c == ')' || c == ']' || c == '}') {
            Acilis a;
            if (!yigin_cikar(y, a)) {
                r = Sonuc{ESLESMEYEN_KAPANIS, i + 1, c, 0, 0, 0};
            } else if (a.tur != acilis_karsiligi(c)) {
                r = Sonuc{TUR_UYUSMAZLIGI, i + 1, c,
                          kapanis_karsiligi(a.tur), a.sutun, 0};
            }
        } else {
            continue;  // parantez olmayan karakterler atlanır
        }
        if (iz) {
            std::cout << "    sütun " << std::setw(2) << i + 1 << " '" << c
                      << "' -> yığın: ";
            for (int k = 0; k < y.boyut; k++) {
                std::cout << y.veri[k].tur;
            }
            std::cout << (y.boyut == 0 ? "(boş)" : "") << "\n";
        }
    }
    if (r.durum == DENGELI && y.boyut > 0) {
        Acilis a = y.veri[y.boyut - 1];  // en içteki açık parantez
        r = Sonuc{ACIK_KALAN, n + 1, a.tur, 0, a.sutun, y.boyut};
    }
    return r;
}
```

Python **python/H09_03_parantez_denetimi.py · satır 94–124**

```python
def parantez_denetle(s, iz):
    """s taranır; ilk hatada durulur (erken çıkış)"""
```

```
    global islenen
    n = len(s)
    r = Sonuc(DENGELI, 0, "", "", 0, 0)
    y = yigin_olustur(n if n > 0 else 1)
    i = 0
    while i < n and r.durum == DENGELI:
        c = s[i]
        islenen += 1
        if c in "([{":
            yigin_it(y, Acilis(c, i + 1))
        elif c in ")]}":
            tamam, a = yigin_cikar(y)
            if not tamam:
                r = Sonuc(ESLESMEYEN_KAPANIS, i + 1, c, "", 0, 0)
            elif a.tur != acilis_karsiligi(c):
                r = Sonuc(TUR_UYUSMAZLIGI, i + 1, c,
                          kapanis_karsiligi(a.tur), a.sutun, 0)
        else:
            i += 1
            continue  # parantez olmayan karakterler atlanır
        if iz:
            icerik = "".join(y.veri[k].tur for k in range(y.boyut))
            print(f"    sütun {i + 1:2d} '{c}' -> yığın: "
                  + (icerik if y.boyut > 0 else "(boş)"))
        i += 1
    if r.durum == DENGELI and y.boyut > 0:
        a = y.veri[y.boyut - 1]  # en içteki açık parantez
        r = Sonuc(ACIK_KALAN, n + 1, a.tur, "", a.sutun, y.boyut)
    return r
```

DİL PENCERESİ · Farklılaşan satırlar

Satırlar C'de `fgets` + `strcspn` ile, C++'ta `std::getline` ile, Python'da stdin bayt olarak okunup `\n`'den bölünerek alınır; dosya sonundaki boş parça atılır (C'deki `fgets` davranışı). Python'daki `c in "([{"` üyelik sınaması C'deki üç karşılaştırmanın karşılığıdır. Büyük girdi C'de `malloc`la ayrılan `char` dizisi, C++'ta `std::string`, Python'da karakter listesi + `join` ile kurulur; Python dizgileri değişmez olduğundan bozuk sürümler dilimlerle yeniden oluşturulur. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 163, 207; Python 160, 173, 176.)

A.4 Örnek 4 · Sonek değerlendirme

C++ **cpp/H09_04_sonek_degerlendirme.cpp · satır 91–116**

```
static int islem_uygula(char op, int a, int b, int& sonuc)
{
    long long r;
    if (op == '+') {
        r = static_cast<long long>(a) + b;
    } else if (op == '-') {
        r = static_cast<long long>(a) - b;
    } else if (op == '*') {
        r = static_cast<long long>(a) * b;
    } else if (op == '/') {
```

```
        if (b == 0) {
            return SIFIRA_BOLME;
        }
        r = static_cast<long long>(a) / b;  // C++: sıfıra doğru kesme
    } else {
        if (b < 0) {
            return NEGATIF_US;
        }
        return us_al(a, b, sonuc);
    }
    if (r < INT_MIN || r > INT_MAX) {
        return TASMA;
    }
    sonuc = static_cast<int>(r);
    return HATA_YOK;
}
```

Python `python/H09_04_sonek_degerlendirme.py · satır 76–100`

```
def kesik_bol(a, b):
    """C'deki gibi sıfıra doğru kesen bölme (// aşağı yuvarlar)"""
    q = abs(a) // abs(b)
    return q if (a < 0) == (b < 0) else -q


def islem_uygula(op, a, b):
    """a op b: (hata kodu, sonuç); int aralığı elle denetlenir"""
    if op == "+":
        r = a + b
    elif op == "-":
        r = a - b
    elif op == "*":
        r = a * b
    elif op == "/":
        if b == 0:
            return SIFIRA_BOLME, 0
        r = kesik_bol(a, b)  # 7 / -2 = -3 (C gibi); 7 // -2 = -4 olurdu
    else:
        if b < 0:
            return NEGATIF_US, 0
        return us_al(a, b)
    if r < INT_MIN or r > INT_MAX:
        return TASMA, 0
    return HATA_YOK, r
```

DİL PENCERESİ · **Farklılaşan satırlar**

Tamsayı bölmesi C ve C++'ta sıfıra doğru keser (`7 / -2 = -3`); Python'un `//` işleci aşağı yuvarlar (`7 // -2 = -4`), bu yüzden `kesik_bol` aynı tanımı açıkça kodlar (`int(a / b)` kayan noktaya gittiği için kullanılmaz). Python tamsayıları sınırsızdır: C'deki 32 bitlik `int` taşması `INT_MIN`/`INT_MAX` sabitleriyle elle denetlenir; C/C++'ta ara sonuç `long long`'da hesaplanıp aralık denetlenir. C'de `||` kısa devresiyle yazılan iki çıkarma Python'da iki ayrı deyimdir. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 14, 148; Python 21, 148.)

A.5 Örnek 5 · Shunting-yard

C++ cpp/H09_05_shunting_yard.cpp · satır 85–103

```cpp
static int oncelik(char op)
{
    if (op == '+' || op == '-') {
        return 1;
    }
    if (op == '*' || op == '/') {
        return 2;
    }
    return op == '~' ? 3 : 4;  // ~ tekli eksi, ^ üs
}

static bool sag_birlesmeli(char op) { return op == '^' || op == '~'; }

// Yığının tepesindeki ust, gelen işleçten önce çıktıya gitmeli mi?
static bool once_cikar_mi(char ust, char gelen)
{
    return oncelik(ust) > oncelik(gelen) ||
           (oncelik(ust) == oncelik(gelen) && !sag_birlesmeli(gelen));
}
```

Python python/H09_05_shunting_yard.py · satır 77–93

```python
def oncelik(op):
    """Öncelik ve birleşme yönü tablosu"""
    if op in "+-":
        return 1
    if op in "*/":
        return 2
    return 3 if op == "~" else 4  # ~ tekli eksi, ^ üs


def sag_birlesmeli(op):
    return op == "^" or op == "~"


def once_cikar_mi(ust, gelen):
    """Yığının tepesindeki ust, gelen işleçten önce çıktıya gitmeli mi?"""
    return (oncelik(ust) > oncelik(gelen) or
            (oncelik(ust) == oncelik(gelen) and not sag_birlesmeli(gelen)))
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> C'de belirteçler sabit boyutlu diziye, metinler `strcat`/`memcpy` ile tampona yazılır; C++'ta `std::vector<Belirtec>` ve `std::string` birleştirmesi, Python'da liste ve `join` kullanılır. Hizalı sütunlar C'de `%-14s`, C++'ta `std::setw` + `std::left`, Python'da `:<14` ile verilir; C ve C++ bayt, Python karakter saydığı için hizalanan sütunlarda yalnızca ASCII bulunur (Türkçe `eylem` metni satır sonundadır). Python'da birden çok değer demetle döner: `(hata, belirteçler, sütun)`. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 149, 160, 224; Python 120, 146.)

A.6 Örnek 6 · Basit hesap makinesi

C++ **cpp/H09_06_hesap_makinesi.cpp · satır 302–322**

```cpp
int hesapla(const std::string& s, int& sonuc, int& sutun, std::string& sonek,
            bool iz)
{
    std::vector<Belirtec> b;
    std::vector<int> cikti;
    sonek.clear();
    int hata = belirtecle(s, b, sutun);
    if (hata == HATA_YOK) {
        hata = sonege_cevir(b, cikti, sutun);
    }
    if (hata != HATA_YOK) {
        return hata;
    }
    for (std::size_t i = 0; i < cikti.size(); i++) {
        const Belirtec& t = b[cikti[i]];
        sonek += (i > 0 ? " " : "") + (t.tur == SAYI
                                           ? std::to_string(t.deger)
                                           : std::string(1, t.islec));
    }
    return sonek_hesapla(b, cikti, sonuc, sutun, iz);
}
```

Python **python/H09_06_hesap_makinesi.py · satır 256–266**

```python
def hesapla(s, iz):
    """Bir ifadeyi hesaplar: (hata, sonuç, sütun, sonek metni)"""
    hata, b, sutun = belirtecle(s)
    if hata == HATA_YOK:
        hata, cikti, sutun = sonege_cevir(b)
    if hata != HATA_YOK:
        return hata, 0, sutun, ""
    sonek = " ".join(str(b[j].deger) if b[j].tur == SAYI else b[j].islec
                     for j in cikti)
    hata, sonuc, sutun = sonek_hesapla(b, cikti, iz)
    return hata, sonuc, sutun, sonek
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Komut satırı argümanları C/C++'ta `argc`/`argv`, Python'da `sys.argv` ile okunur; çıkış kodu C/C++'ta `main`'in dönüş değeri, Python'da `sys.exit(main())` ile verilir — üç dilde 0 (tümü geçerli), 1 (hatalı ifade), 2 (hatalı kullanım). Kullanım iletisi stderr'e yazılır (`fprintf(stderr, …)`, `std::cerr`, `print(…, file=sys.stderr)`). Rastgele ifade metni C'de `sprintf` ile tampona, C++'ta `std::string` birleştirmesiyle, Python'da f-dizgiyle kurulur; Python'un `eval`'i `^`'yi XOR, `/`'yi gerçek bölme saydığı için kâhin olarak kullanılmadı. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 355, 397; Python 97, 284, 318, 346.)

A.7 Örnek 7 · Geri al / yinele

C++ **cpp/H09_07_geri_al_yinele.cpp · satır 103–126**

```cpp
static void uygula(std::string& b, const Islem& i)
{
    if (i.tur == '+') {
        b += i.metin;
    } else {
        b.erase(b.size() - i.metin.size());
    }
}

// İşlemin tersini uygular (geri al)
static void ters_uygula(std::string& b, const Islem& i)
{
    Islem ters = i;
    ters.tur = (i.tur == '+') ? '-' : '+';
    uygula(b, ters);
}

// Yeni işlem: uygula, geri yığınına it, yinele yığınını boşalt
static void yeni_islem(Duzenleyici& e, const Islem& i)
{
    uygula(e.belge, i);
    yigin_it(e.geri, i);
    temizlenen += yigin_yok_et(e.yinele);
}
```

Python **python/H09_07_geri_al_yinele.py · satır 90–107**

```python
def uygula(belge, i):
    """İşlemi uygular, yeni belgeyi döner: yaz → sona ekle, sil → sondan"""
    if i.tur == "+":
        return belge + i.metin
    return belge[:len(belge) - len(i.metin)]


def ters_uygula(belge, i):
    """İşlemin tersini uygular (geri al)"""
    return uygula(belge, Islem("-" if i.tur == "+" else "+", i.metin))


def yeni_islem(e, i):
    """Yeni işlem: uygula, geri yığınına it, yinele yığınını boşalt"""
    global temizlenen
    e.belge = uygula(e.belge, i)
    yigin_it(e.geri, i)
    temizlenen += yigin_yok_et(e.yinele)
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> İşlem kaydı C'de sabit uzunluklu `char` dizisi içeren bir `struct`'tır; `d->islem = *i` yapı ataması metni de kopyalar. C++'ta `std::string` alanı kopyalamayı kendisi yapar; Python'da dizgiler değişmez olduğundan paylaşılmaları güvenlidir, `Islem` nesnesi C'deki kopya anlamını korumak için yeniden kurulur. Sayı ayrıştırma C'de `strtol` + `errno`, C++'ta `std::stol` + istisna; Python'un `int()`'i boşluk ve `_` kabul ettiği için C ile aynı kuralı uygulayan `tamsayi_oku` elle yazılmıştır. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 22, 217, 249; Python 69, 172, 215.)

A.8 Örnek 8 · Dairesel tampon kuyruk

C++ **cpp/H09_08_dairesel_kuyruk.cpp · satır 54–76**

```cpp
static int ilerle(const Kuyruk& q, int i)
{
    if (i + 1 == q.kapasite) {
        sarma_sayisi++;
    }
    if (q.maskeli) {
        return (i + 1) & (q.kapasite - 1);
    }
    return (i + 1) % q.kapasite;
}

// Dolu kuyrukta false döner (taşma politikası: reddet)
bool kuyruk_ekle(Kuyruk& q, int x)
{
    if (kuyruk_dolu_mu(q)) {
        return false;
    }
    q.veri[q.son] = x;
    q.son = ilerle(q, q.son);
    q.sayac++;
    ekle_sayisi++;
    return true;
}
```

Python **python/H09_08_dairesel_kuyruk.py · satır 45–64**

```python
def ilerle(q, i):
    """Sonraki indis: son hücreden sonra 0'a sarar"""
    global sarma_sayisi
    if i + 1 == q.kapasite:
        sarma_sayisi += 1
    if q.maskeli:
        return (i + 1) & (q.kapasite - 1)
    return (i + 1) % q.kapasite


def kuyruk_ekle(q, x):
    """Dolu kuyrukta False döner (taşma politikası: reddet)"""
    global ekle_sayisi
    if kuyruk_dolu_mu(q):
        return False
```

```
60      q.veri[q.son] = x
61      q.son = ilerle(q, q.son)
62      q.sayac += 1
63      ekle_sayisi += 1
64      return True
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Kâhin C'de sarmayan büyük bir dizi ve iki indis, C++'ta `std::queue`, Python'da `collections.deque`'dur (yalnızca karşılaştırma için). `%` burada yalnız negatif olmayan indislerle kullanıldığından üç dilde aynı sonucu verir; negatif sayıda C'de `-1 % 8 == -1`, Python'da `7` olurdu. Fiziksel dizi metni C'de çağıranın tamponuna `snprintf`/`strcat` ile yazılır, C++ ve Python'da dizgi olarak döndürülür. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 22, 218, 239; Python 10, 116, 172, 189.)

A.9 Örnek 9 · Liste tabanlı kuyruk

C++ **cpp/H09_09_liste_kuyruk.cpp · satır 78–94**

```
78  bool kuyruk_cikar(Kuyruk& q, int& x)
79  {
80      if (kuyruk_bos_mu(q)) {
81          return false;
82      }
83      Dugum *d = q.bas;
84      x = d->deger;
85      q.bas = d->sonraki;
86      if (q.bas == nullptr) {
87          q.son = nullptr;  // boşaldı: son da nullptr olmalı
88      }
89      delete d;
90      serbest_dugum++;
91      q.boyut--;
92      cikar_sayisi++;
93      return true;
94  }
```

Python **python/H09_09_liste_kuyruk.py · satır 69–82**

```
69  def kuyruk_cikar(q):
70      """Baştan çıkarır; boş kuyrukta (False, -1)"""
71      global serbest_dugum, cikar_sayisi
72      if kuyruk_bos_mu(q):
73          return False, -1
74      d = q.bas
75      x = d.deger
76      q.bas = d.sonraki
77      if q.bas is None:
78          q.son = None  # boşaldı: son da None olmalı
79      serbest_dugum += 1
80      q.boyut -= 1
81      cikar_sayisi += 1
82      return True, x
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Boş kuyruk C'de `NULL`, C++'ta `nullptr`, Python'da `None` ile gösterilir (çıktı metni üç dilde aynı kalsın diye `NULL` yazar). Son düğüm çıkarılınca `son` da boşaltılmazsa C/C++'ta serbest bırakılmış düğümü gösteren sarkan işaretçi kalır; Python'da aynı hata, kuyruktan çıkmış eski düğüme bağlanıp elemanları sessizce kaybetmek demektir. C++ kuyruğu RAII'dir: yıkıcı düğümleri döngüyle siler. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 37, 64, 167; Python 125, 145.)

A.10 Örnek 10 · Kayıt gişesi simülasyonu

C++ **cpp/H09_10_gise_simulasyonu.cpp · satır 66–116**

```cpp
Sonuc gise_benzet(int k, int gelis_milyonda, int sure, int kapasite,
                  int olay_siniri)
{
    Sonuc s;
    Kuyruk q;
    std::vector<int> kalan(k, 0);  // gişedeki işin kalan süresi (0: boş)
    int olay = 0;
    kuyruk_olustur(q, kapasite);
    for (int t = 0; t < sure; t++) {
        if (rastgele_aralik(0, 999999) < gelis_milyonda) {  // 1) geliş
            s.gelen++;
            bool tamam = kuyruk_ekle(q, t);
            s.reddedilen += !tamam;
            if (olay++ < olay_siniri) {
                std::printf("  t = %4d  öğrenci %lld geldi%s; kuyruk %d\n",
                            t, s.gelen,
                            tamam ? "" : " (kuyruk dolu: geri döndü)",
                            q.sayac);
            }
        }
        for (int g = 0; g < k; g++) {  // 2) boş gişe sıradakini alır
            int gelis;
            if (kalan[g] == 0 && kuyruk_cikar(q, gelis)) {
                int bekleme = t - gelis;
                s.hizmet_alan++;
                s.toplam_bekleme += bekleme;
                if (bekleme > s.en_uzun_bekleme) {
                    s.en_uzun_bekleme = bekleme;
                }
                kalan[g] = rastgele_aralik(HIZMET_EN_AZ, HIZMET_EN_COK);
                if (olay++ < olay_siniri) {
                    std::printf("  t = %4d  gişe %d: öğrenci %lld (bekleme "
                                "%d s, hizmet %d s); kuyruk %d\n", t, g + 1,
                                s.hizmet_alan, bekleme, kalan[g], q.sayac);
                }
            }
        }
        for (int g = 0; g < k; g++) {  // 3) saat ilerler
            if (kalan[g] > 0) {
                kalan[g]--;
                s.mesgul++;
            }
        }
```

```
        s.toplam_kuyruk += q.sayac;
        if (q.sayac > s.en_uzun_kuyruk) {
            s.en_uzun_kuyruk = q.sayac;
        }
    }
    s.kuyrukta_kalan = q.sayac;
    return s;
}
```

Python **python/H09_10_gise_simulasyonu.py · satır 61–100**

```python
def gise_benzet(k, gelis_milyonda, sure, kapasite, olay_siniri):
    """k gişe, sure saniye; ilk olay_siniri olay yazdırılır"""
    s = Sonuc()
    q = kuyruk_olustur(kapasite)
    kalan = [0] * k  # gişedeki işin kalan süresi (0: boş)
    olay = 0
    for t in range(sure):
        if rastgele_aralik(0, 999999) < gelis_milyonda:  # 1) geliş
            s.gelen += 1
            tamam = kuyruk_ekle(q, t)
            s.reddedilen += 0 if tamam else 1
            if olay < olay_siniri:
                print(f"  t = {t:4d}  öğrenci {s.gelen} geldi"
                      + ("" if tamam else " (kuyruk dolu: geri döndü)")
                      + f"; kuyruk {q.sayac}")
            olay += 1
        for g in range(k):  # 2) boş gişe sıradakini alır
            if kalan[g] == 0:
                tamam, gelis = kuyruk_cikar(q)
                if tamam:
                    bekleme = t - gelis
                    s.hizmet_alan += 1
                    s.toplam_bekleme += bekleme
                    if bekleme > s.en_uzun_bekleme:
                        s.en_uzun_bekleme = bekleme
                    kalan[g] = rastgele_aralik(HIZMET_EN_AZ, HIZMET_EN_COK)
                    if olay < olay_siniri:
                        print(f"  t = {t:4d}  gişe {g + 1}: öğrenci "
                              f"{s.hizmet_alan} (bekleme {bekleme} s, "
                              f"hizmet {kalan[g]} s); kuyruk {q.sayac}")
                    olay += 1
        for g in range(k):  # 3) saat ilerler
            if kalan[g] > 0:
                kalan[g] -= 1
                s.mesgul += 1
        s.toplam_kuyruk += q.sayac
        if q.sayac > s.en_uzun_kuyruk:
            s.en_uzun_kuyruk = q.sayac
    s.kuyrukta_kalan = q.sayac
    return s
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Ortalama C'de `(double)toplam / sayi`, Python'da `toplam / sayi` ile hesaplanır; iki tamsayının bölümü üç dilde de doğru yuvarlanmış tek bir IEEE 754 işlemidir, bu yüzden `%.2f` çıktıları bayt bayt aynıdır. Uzun tablo satırı C++'ta `std::printf` ile yazılmıştır (`std::cout` + `std::setw`/`std::setprecision` zinciri aynı sonucu verir ama okunması zordur). Python'da tamsayı bölmesi `//` (pozitif sayılarda C'deki `/` ile aynı), gerçek bölme `/` işlecidir. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 28, 135; Python 105, 110, 114.)

A.11 Örnek 11 · Sonraki büyük eleman

C++ **cpp/H09_11_sonraki_buyuk.cpp · satır 66–94**

```cpp
void sonraki_buyuk(const std::vector<int>& a, std::vector<int>& sonuc,
                   bool iz)
{
    int n = static_cast<int>(a.size());
    Yigin y;
    yigin_olustur(y, n);
    sonuc.assign(n, 0);
    for (int i = 0; i < n; i++) {
        std::string coz;
        while (y.boyut > 0 && kucuk_mu(a[y.veri[y.boyut - 1]], a[i])) {
            int j = yigin_cikar(y);
            sonuc[j] = a[i];  // a[j]'nin cevabı bulundu
            coz += (coz.empty() ? "" : " ") + std::to_string(a[j]);
        }
        yigin_it(y, i);
        if (iz) {
            std::cout << "  " << std::setw(2) << i << " " << std::setw(5)
                      << a[i] << "   " << std::left << std::setw(13)
                      << (coz.empty() ? "-" : coz) << std::right << " [";
            for (int k = 0; k < y.boyut; k++) {
                std::cout << (k > 0 ? " " : "") << a[y.veri[k]];
            }
            std::cout << "]\n";
        }
    }
    while (y.boyut > 0) {
        sonuc[yigin_cikar(y)] = -1;  // sağda büyüğü olmayanlar
    }
}
```

Python **python/H09_11_sonraki_buyuk.py · satır 59–78**

```python
def sonraki_buyuk(a, iz):
    """sonuc[i] = a[i]'nin sağındaki ilk büyük değer; yoksa -1 (a[i] >= 0).
    iz: her adımda cevabı bulunanları (çıkarılanları) ve yığını yazar"""
    n = len(a)
    y = yigin_olustur(n)
    sonuc = [0] * n
    for i in range(n):
        coz = []
        while y.boyut > 0 and kucuk_mu(a[y.veri[y.boyut - 1]], a[i]):
```

```
            j = yigin_cikar(y)
            sonuc[j] = a[i]  # a[j]'nin cevabı bulundu
            coz.append(str(a[j]))
        yigin_it(y, i)
        if iz:
            icerik = " ".join(str(a[y.veri[k]]) for k in range(y.boyut))
            print(f"  {i:2d} {a[i]:5d}   {' '.join(coz) or '-':<13} "
                  f"[{icerik}]")
    while y.boyut > 0:
        sonuc[yigin_cikar(y)] = -1  # sağda büyüğü olmayanlar
    return sonuc
```

DİL PENCERESİ · Farklılaşan satırlar

Sonuç dizilerinin eşitliği C'de döngülü `ayni_mi` fonksiyonuyla, C++'ta `std::vector`'ün `==` işleciyle, Python'da listelerin `==`'siyle denetlenir. Çıkarılan değerlerin metni C'de `snprintf(coz + strlen(coz), …)` ile tampona eklenir, C++'ta `std::string` birleştirmesi, Python'da liste + `join` kullanılır (`' '.join(coz) or '-'`: boş dizgi yanlış sayılır). (Gösterilen fonksiyonların dışındaki fark satırları: C++ 161, 177; Python 97, 140.)

A.12 Örnek 12 · Açık yığınlı hızlı sıralama

C++ **cpp/H09_12_yiginli_hizli_siralama.cpp · satır 137–170**

```
void yiginli_hizli_sirala(std::vector<int>& a, bool iz)
{
    int n = static_cast<int>(a.size());
    Yigin y;
    Aralik r;
    yigin_olustur(y, taban_log2(n > 0 ? n : 1) + 1);
    if (n > 1) {
        yigin_it(y, Aralik{0, n - 1});
    }
    while (yigin_cikar(y, r)) {
        int lo = r.lo, hi = r.hi;
        if (iz) {
            std::cout << "  çıkar " << lo << ".." << hi << "\n";
        }
        while (lo < hi) {
            int pivot = a[hi];
            int p = bolumle(a, lo, hi);
            Aralik buyuk;
            if (p - lo < hi - p) {  // sol küçük: sağı it
                buyuk = Aralik{p + 1, hi};
                hi = p - 1;
            } else {                // sağ küçük: solu it
                buyuk = Aralik{lo, p - 1};
                lo = p + 1;
            }
            if (buyuk.lo < buyuk.hi) {
                yigin_it(y, buyuk);
            }
            if (iz) {
```

```
                izle(a, y, pivot, p, buyuk, lo, hi);
            }
        }
    }
}
```

Python `python/H09_12_yiginli_hizli_siralama.py · satır 105–129`

```python
def yiginli_hizli_sirala(a, iz):
    """Açık yığınlı hızlı sıralama: büyük parçayı it, küçüğü hemen işle"""
    n = len(a)
    y = yigin_olustur(taban_log2(n if n > 0 else 1) + 1)
    if n > 1:
        yigin_it(y, (0, n - 1))
    tamam, r = yigin_cikar(y)
    while tamam:
        lo, hi = r
        if iz:
            print(f"  çıkar {lo}..{hi}")
        while lo < hi:
            pivot = a[hi]
            p = bolumle(a, lo, hi)
            if p - lo < hi - p:  # sol küçük: sağı it
                buyuk = (p + 1, hi)
                hi = p - 1
            else:                # sağ küçük: solu it
                buyuk = (lo, p - 1)
                lo = p + 1
            if buyuk[0] < buyuk[1]:
                yigin_it(y, buyuk)
            if iz:
                izle(a, y, pivot, p, buyuk, lo, hi)
        tamam, r = yigin_cikar(y)
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Aralık C ve C++'ta iki alanlı bir `struct`, Python'da `(lo, hi)` demetidir; demet açma (`lo, hi = r`) yapı alanlarına erişimin karşılığıdır. Kâhin C'de `qsort`, C++'ta `std::sort`, Python'da `sorted`'dır. Özyinelemeli sürüm Python'da varsayılan özyineleme sınırına (1000) takılmasın diye en kötü durum girdileri n = 500 ile sınırlıdır; açık yığınlı sürüm ise her n'de yalnızca $\lfloor \log_2 n \rfloor + 1$ yuvalık yığın kullanır. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 85, 209, 215; Python 70, 159.)

A.13 Örnek 13 · Kayan pencere en büyüğü

C++ `cpp/H09_13_kayan_pencere.cpp · satır 116–155`

```cpp
void kayan_en_buyuk(const std::vector<int>& a, int k,
                    std::vector<int>& en_buyuk, bool iz)
{
    int n = static_cast<int>(a.size());
    Deque d;
    int x;
    deque_olustur(d, k);
```

```
    en_buyuk.assign(n - k + 1, 0);
    for (int i = 0; i < n; i++) {
        std::string atilan;
        if (!deque_bos_mu(d) && deque_on(d) == i - k) {
            onden_cikar(d, x);  // pencereden çıkan indis
            atilan = "önden " + std::to_string(a[x]);
        }
        while (!deque_bos_mu(d) && kucuk_esit_mi(a[deque_arka(d)], a[i])) {
            arkadan_cikar(d, x);  // bundan sonra hiç en büyük olamaz
            atilan += (atilan.empty() ? "" : ", ") + std::string("arkadan ") +
                      std::to_string(a[x]);
        }
        arkaya_ekle(d, i);
        if (i >= k - 1) {
            en_buyuk[i - k + 1] = a[deque_on(d)];
        }
        if (iz) {
            std::string ic = "[", eb = "-";
            for (int q = 0; q < d.sayac; q++) {
                ic += (q > 0 ? " " : "") +
                      std::to_string(a[d.veri[(d.bas + q) % k]]);
            }
            ic += "]";
            if (i >= k - 1) {
                eb = std::to_string(en_buyuk[i - k + 1]);
            }
            std::cout << "  " << std::setw(2) << i << " " << std::setw(4)
                      << a[i] << "   " << std::left << std::setw(12) << ic
                      << std::right << " " << std::setw(5) << eb << "   "
                      << (atilan.empty() ? "-" : atilan) << "\n";
        }
    }
}
```

Python `python/H09_13_kayan_pencere.py · satır 102–124`

```
def kayan_en_buyuk(a, k, iz):
    """en_buyuk[i] = max(a[i..i+k-1]), i = 0..n-k (1 <= k <= n)"""
    n = len(a)
    d = deque_olustur(k)
    en_buyuk = [0] * (n - k + 1)
    for i in range(n):
        atilan = []
        if not deque_bos_mu(d) and deque_on(d) == i - k:
            x = onden_cikar(d)[1]  # pencereden çıkan indis
            atilan.append(f"önden {a[x]}")
        while not deque_bos_mu(d) and kucuk_esit_mi(a[deque_arka(d)], a[i]):
            x = arkadan_cikar(d)[1]  # bundan sonra hiç en büyük olamaz
            atilan.append(f"arkadan {a[x]}")
        arkaya_ekle(d, i)
        if i >= k - 1:
            en_buyuk[i - k + 1] = a[deque_on(d)]
        if iz:
            ic = "[" + " ".join(str(a[d.veri[(d.bas + q) % k]])
                                for q in range(d.sayac)) + "]"
```

```
            eb = str(en_buyuk[i - k + 1]) if i >= k - 1 else "-"
            print(f"  {i:2d} {a[i]:4d}   {ic:<12} {eb:>5}   "
                  + (", ".join(atilan) if atilan else "-"))
    return en_buyuk
```

DİL PENCERESİ · Farklılaşan satırlar

Öne eklemede `bas` geriye sararken `(bas - 1 + kapasite) % kapasite` yazılır: C ve C++'ta `-1 % 4 == -1` olduğundan `+ kapasite` zorunludur; Python'da `-1 % 4 == 3` olduğu için gerekmezdi ama üç dilde aynı ifade korunmuştur. Kaba kuvvetin döngü sınırı C'deki `i + k <= n` yerine Python'da `range(n - k + 1)` ile verilir. Python'daki çoklu atama (`e1, e0 = …`) iki çağrıyı soldan sağa yapar; C'de tek bildirimdeki iki ilk değer de soldan sağa değerlendirilir (her tam bildirimcinin sonu bir sıralama noktasıdır). (Gösterilen fonksiyonların dışındaki fark satırları: C++ 22, 237; Python 132, 153.)

A.14 Örnek 14 · Histogramdaki en büyük dikdörtgen

C++ **cpp/H09_14_histogram_dikdortgen.cpp · satır 66–106**

```
66  long long en_buyuk_dikdortgen(const std::vector<int>& h, bool iz)
67  {
68      int n = static_cast<int>(h.size());
69      Yigin y;
70      long long en_iyi = 0;
71      yigin_olustur(y, n);
72      for (int i = 0; i <= n; i++) {
73          int yuk = (i == n) ? 0 : h[i];  // i == n: tüm yığını boşaltan bekçi
74          std::string cikan;
75          while (y.boyut > 0 && buyuk_esit_mi(h[y.veri[y.boyut - 1]], yuk)) {
76              int t = yigin_cikar(y);
77              int sol = (y.boyut > 0) ? y.veri[y.boyut - 1] : -1;
78              long long alan = static_cast<long long>(h[t]) * (i - sol - 1);
79              if (alan > en_iyi) {
80                  en_iyi = alan;
81              }
82              cikan += (cikan.empty() ? "" : " ") + std::to_string(h[t]) + "x" +
83                       std::to_string(i - sol - 1) + "=" + std::to_string(alan);
84          }
85          if (i < n) {
86              yigin_it(y, i);
87          }
88          if (iz) {
89              std::string ic = "[";
90              for (int q = 0; q < y.boyut; q++) {
91                  ic += (q > 0 ? " " : "") + std::to_string(h[y.veri[q]]);
92              }
93              ic += "]";
94              if (i < n) {
95                  std::cout << "  " << std::setw(2) << i << " " << std::setw(4)
96                            << yuk << "   ";
97              } else {
98                  std::cout << "  son  (0)  ";
99              }
100             std::cout << std::left << std::setw(14) << ic << std::right << " "
```

```
                        << std::setw(4) << en_iyi << "   "
                        << (cikan.empty() && i < n ? "-" : cikan) << "\n";
            }
        }
        return en_iyi;
    }
```

Python **python/H09_14_histogram_dikdortgen.py · satır 60–83**

```python
def en_buyuk_dikdortgen(h, iz):
    """h histogramındaki en büyük dikdörtgenin alanı"""
    n = len(h)
    y = yigin_olustur(n)
    en_iyi = 0
    for i in range(n + 1):
        yuk = 0 if i == n else h[i]  # i == n: tüm yığını boşaltan bekçi
        cikan = []
        while y.boyut > 0 and buyuk_esit_mi(h[y.veri[y.boyut - 1]], yuk):
            t = yigin_cikar(y)
            sol = y.veri[y.boyut - 1] if y.boyut > 0 else -1
            alan = h[t] * (i - sol - 1)
            if alan > en_iyi:
                en_iyi = alan
            cikan.append(f"{h[t]}x{i - sol - 1}={alan}")
        if i < n:
            yigin_it(y, i)
        if iz:
            ic = "[" + " ".join(str(h[y.veri[q]])
                                for q in range(y.boyut)) + "]"
            bas = f"  {i:2d} {yuk:4d}   " if i < n else "  son  (0)  "
            son = " ".join(cikan) if cikan or i == n else "-"
            print(bas + f"{ic:<14} {en_iyi:4d}   {son}")
    return en_iyi
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Alan C/C++'ta `long long` ile hesaplanır: artan girdide en büyük alan 2 500 050 000'dir ve 32 bitlik `int` sınırını (2 147 483 647) aşar — `int` çarpımı işaretli taşma, yani tanımsız davranış olurdu. Python tamsayısı sınırsız olduğundan dönüşüm gerekmez. Büyük girdiler C'de aynı diziye yerinde yazılır, Python'da her tür için liste üreteciyle yeniden kurulur. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 160; Python 135.)